DEVOPS ENGINEERING MASTERY

# DEVSECOPS, SRE & PRODUCTION RELIABILITY ENGINEERING

Secure Software Delivery, Application Security, Infrastructure Security, Site Reliability Engineering, Incident Management, Disaster Recovery, FinOps & Enterprise Production Operations

VOLUME 15 • FIRST EDITION

```
nahid@technova:~$ trivy image technova/api:1.4.2
✓ 0 CRITICAL, 0 HIGH — image clean
nahid@technova:~$ cosign verify technova/api:1.4.2
✓ Signature verified — Sigstore keyless
nahid@technova:~$ kubectl get slo technova-api -n prod
```

DevSecOps • SRE • OWASP • Kubernetes Security • SLI/SLO

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Capstone

# DevSecOps/SRE Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 15 (Companion Workbook). First Edition, 9 November 2026. DevSecOps, SRE and production reliability, built around the TechNova MERN example application. This workbook collects all 24 guided labs, 15 assignments, the 3 major projects and the capstone, so you can work through them beside your terminal without carrying the whole book. Do the labs in order; each lab states its prerequisites. All security labs run only in authorized, isolated environments — local VMs, disposable containers, OWASP Juice Shop, kind/Minikube clusters. Never scan or test a system you do not own or have explicit authorization to test. The instructor-style solution guide for the capstone is at the end — attempt the capstone before reading it.

ল্যাব ১

# Lab 1 — MERN Application Threat Modeling

এই ল্যাবে আপনি TechNova-র MERN (MongoDB, Express.js, React, Node.js) অ্যাপ্লিকেশনের জন্য একটি পূর্ণাঙ্গ থ্রেট মডেল (Threat Model — সম্ভাব্য আক্রমণ ও ঝুঁকির কাঠামোবদ্ধ চিত্র) তৈরি করবেন। থ্রেট মডেলিং মানে কোড লেখার আগেই ভাবা — আক্রমণকারী কোথা দিয়ে ঢুকতে পারে, কী চুরি করতে পারে, আর কোন নিয়ন্ত্রণ (control) তা ঠেকাবে।

## ১. উদ্দেশ্য (Objectives)

- MERN অ্যাপের ডেটা ফ্লো ডায়াগ্রাম (Data Flow Diagram, DFD) আঁকতে পারা।
- STRIDE পদ্ধতিতে (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) থ্রেট চিহ্নিত করা।
- প্রতিটি থ্রেটের ঝুঁকি (likelihood × impact) মূল্যায়ন করে অগ্রাধিকার দেওয়া।
- প্রতিটি উচ্চ-ঝুঁকির থ্রেটের জন্য mitigation (প্রতিকার) লেখা।

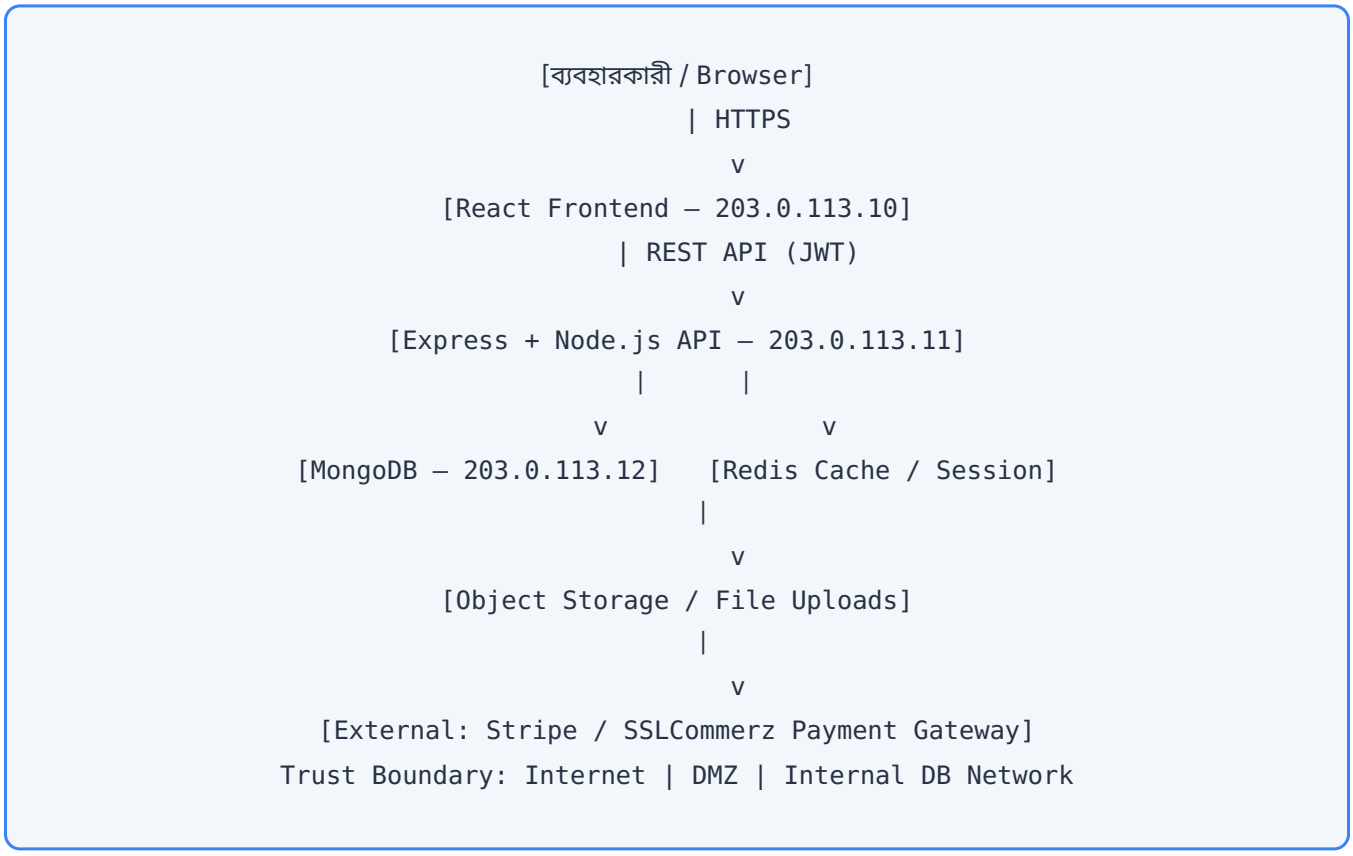
## ২. পূর্বশর্ত (Prerequisites)

- MERN স্ট্যাকের মৌলিক ধারণা; TechNova অ্যাপের আর্কিটেকচার সম্পর্কে ধারণা।
- Ubuntu Server 24.04 LTS ল্যাব VM বা কনটেইনার; draw.io বা কাগজ-কলম।
- OWASP Threat Modeling Cheat Sheet পড়া থাকলে ভালো।

## ৩. আনুমানিক সময়

৩ ঘণ্টা — DFD ৪৫ মিনিট, STRIDE বিশ্লেষণ ১ ঘণ্টা, ঝুঁকি মূল্যায়ন ও mitigation ১ ঘণ্টা, পর্যালোচনা ১৫ মিনিট।

## ৪. আর্কিটেকচার




## ৫. নিরাপদ ল্যাব পরিবেশ

এই ল্যাব সম্পূর্ণ কাগজ/ডায়াগ্রাম ও নিজের ল্যাব VM-এ সীমাবদ্ধ। কোনো বাস্তব বা তৃতীয় পক্ষের সিস্টেম স্ক্যান বা আক্রমণ করবেন না। শুধু technova.example (ল্যাব ডোমেইন) ও 203.0.113.x ল্যাব নেটওয়ার্ক ব্যবহার করুন।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab01-threat-model
cd ~/technova-lab/lab01-threat-model
# অ্যাসেট তালিকা তৈরি করুন
cat > assets.md <<'EOF'
Assets: ব্যবহারকারীর PII, পাসওয়ার্ড হ্যাশ, JWT secret,
পেমেন্ট টোকেন, MongoDB ডেটা, আপলোড করা ফাইল, অ্যাডমিন প্যানেল
EOF
cat assets.md
```

## ৭. কমান্ড ও কনফিগারেশন

থ্রেট মডেলিং মূলত বিশ্লেষণের কাজ; নিচের টেমপ্লেটটি একটি ফাইলে পূরণ করুন:

```
# threat-model.md টেমপ্লেট
## ১. সিস্টেম বিবরণ
## ২. DFD (উপরের ডায়াগ্রাম)
## ৩. STRIDE টেবিল
```

```
| # | উপাদান | STRIDE | থ্রেট | Likelihood | Impact | ঝুঁকি | Mitigation |
| 1 | Login API | Spoofing | Credential stuffing | মাঝারি | উচ্চ | উচ্চ | Rate limit + MFA + bcrypt |
| 2 | JWT | Tampering | টোকেন জালিয়াতি | নিম্ন | উচ্চ | মাঝারি | শক্তিশালী secret, স্বল্প expiry |
| 3 | MongoDB | Info Disclosure | NoSQL injection | মাঝারি | উচ্চ | উচ্চ | Parameterized query, validation |
## ৪. অবশিষ্ট ঝুঁকি (Residual Risk) ও স্বীকৃতি
```

> **CONCEPT:** STRIDE-এর প্রতিটি অক্ষর এক ধরনের আক্রমণ: Spoofing (পরিচয় জালিয়াতি), Tampering (ডেটা বদলে দেওয়া), Repudiation (কাজ অস্বীকার), Information Disclosure (তথ্য ফাঁস), Denial of Service (সেবা বন্ধ), Elevation of Privilege (অনুমতির ঊর্ধ্বে ওঠা)।

## ৮. প্রত্যাশিত আউটপুট

- একটি সম্পূর্ণ DFD, কমপক্ষে ১২টি STRIDE থ্রেট, ঝুঁকি-র‍্যাঙ্ক করা তালিকা এবং প্রতিটির mitigation।
- শীর্ষ ৫ ঝুঁকির জন্য সুনির্দিষ্ট নিয়ন্ত্রণ (যেমন: Helmet, rate limiting, input validation)।

## ৯. যাচাইকরণ (Verification)

- প্রতিটি trust boundary অতিক্রমকারী ডেটা ফ্লো কি DFD-তে আছে? যাচাই করুন।
- শীর্ষ ঝুঁকিগুলো কি Lab 2-এর OWASP পরীক্ষার সঙ্গে মেলে?
- সহপাঠী বা পর্যালোচক আপনার মডেলে অন্তত একটি অনুপস্থিত থ্রেট খুঁজে পান কি না দেখুন।

## ১০. সমস্যা সমাধান (Troubleshooting)

- **থ্রেট খুঁজে পাচ্ছি না:** প্রতিটি API endpoint ধরে ধরে ভাবুন — কে কল করে, কী ইনপুট যায়, কী ফেরত আসে।
- **সবকিছুই উচ্চ ঝুঁকি মনে হচ্ছে:** likelihood ও impact আলাদা করে বিচার করুন; সবকিছু উচ্চ হতে পারে না।

## ১১. নিরাপত্তা সতর্কতা

> **WARNING:** থ্রেট মডেলে আসল সিক্রেট, পাসওয়ার্ড বা প্রোডাকশন ডেটা লিখবেন না। শুধু প্লেসহোল্ডার (যেমন CHANGE-ME-JWT-SECRET) ব্যবহার করুন। থ্রেট মডেলিং আক্রমণের অনুমতি দেয় না — এটি প্রতিরক্ষার পরিকল্পনা।

## ১২. ক্লিনআপ (Cleanup)

```
cd ~/technova-lab/lab01-threat-model
# ফাইলগুলো আপনার ল্যাব ফোল্ডারে সংরক্ষণ করুন; কোনো সার্ভিস চালু করা হয়নি, তাই বন্ধ করার কিছু নেই
ls -l
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. কোন থ্রেটটি সবচেয়ে বেশি অবহেলিত হয় এবং কেন?
2. থ্রেট মডেল কত ঘন ঘন হালনাগাদ করা উচিত?
3. আপনার শীর্ষ ঝুঁকিটি ঠেকাতে সবচেয়ে সস্তা নিয়ন্ত্রণ কোনটি?

ল্যাব ২

# Lab 2 — OWASP Top 10 Security Assessment

OWASP Top 10 হলো ওয়েব অ্যাপ্লিকেশনের সবচেয়ে গুরুতর ১০টি নিরাপত্তা ঝুঁকির তালিকা। এই ল্যাবে আপনি OWASP Juice Shop (একটি ইচ্ছাকৃতভাবে দুর্বল, শেখার জন্য তৈরি অ্যাপ) ব্যবহার করে এই ঝুঁকিগুলো চিনতে ও পরীক্ষা করতে শিখবেন — নিজের ল্যাবেই, নিরাপদে।

## ১. উদ্দেশ্য (Objectives)

- OWASP Top 10 (2021) এর প্রতিটি শ্রেণি ব্যাখ্যা করতে পারা।
- Juice Shop-এ Broken Access Control, Injection, XSS-এর মতো দুর্বলতা খুঁজে বের করা।
- প্রতিটি ফলাফলের জন্য প্রমাণ, প্রভাব ও প্রতিকার লেখা।

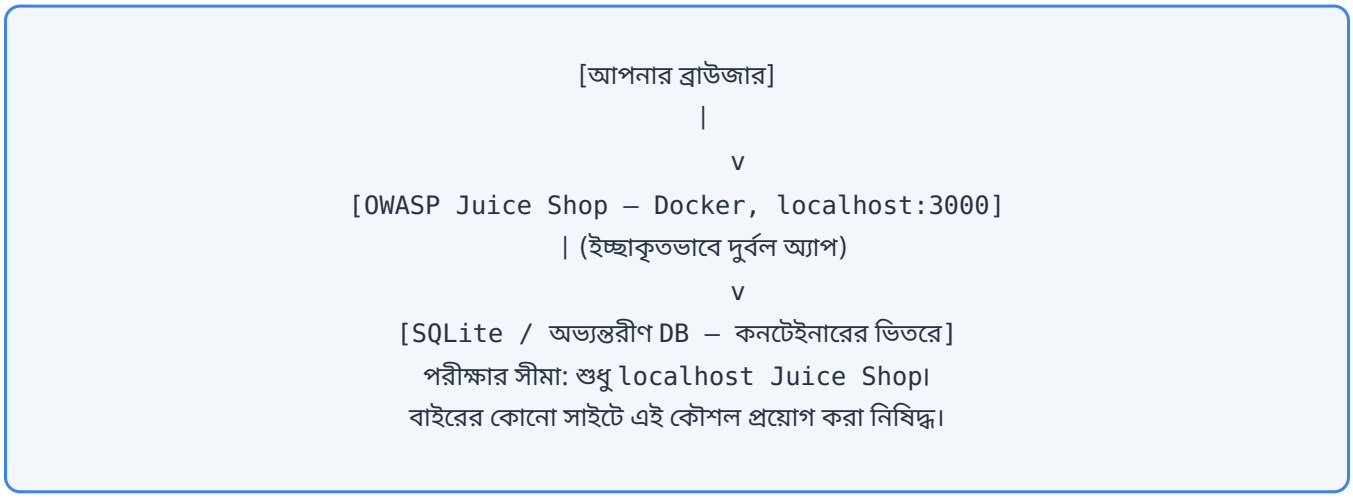
## ২. পূর্বশর্ত (Prerequisites)

- Lab 1 সম্পন্ন; Docker ইনস্টল করা Ubuntu 24.04 ল্যাব VM।
- ব্রাউজার ও Burp Suite Community বা ব্রাউজার DevTools সম্পর্কে প্রাথমিক ধারণা।

## ৩. আনুমানিক সময়

৪ ঘণ্টা।

## ৪. আর্কিটেকচার




## ৫. নিরাপদ ল্যাব পরিবেশ

**WARNING:** Juice Shop শুধু আপনার নিজের মেশিনে চালান। এই ল্যাবে শেখা injection বা XSS কৌশল কোনো বাস্তব ওয়েবসাইটে, এমনকি কৌতূহলবশতও, প্রয়োগ করবেন না — তা অননুমোদিত আক্রমণ এবং আইনত দণ্ডনীয় হতে পারে।

## ৬. সেটআপ

```
docker pull bkimminich/juice-shop
docker run -d --name juice-shop -p 3000:3000 bkimminich/juice-shop
sleep 10
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/
```

## ৭. কমান্ড ও কনফিগারেশন

```
# কনটেইনার চলছে কি না দেখুন
docker ps --filter name=juice-shop
docker logs juice-shop --tail 20

# লগইন ফর্মে SQL injection ধারণা যাচাই (শুধু Juice Shop-এ):
# ইমেইল ঘরে: ' OR 1=1--  (শিক্ষামূলক উদাহরণ; ফলাফল লিপিবদ্ধ করুন)
# XSS পরীক্ষা: সার্চ বাক্সে <script>alert(1)</script> লিখে প্রতিক্রিয়া দেখুন

# আপনার ফলাফল নোট করুন
cat > owasp-findings.md <<'EOF'
| OWASP শ্রেণি | উদাহরণ | প্রভাব | প্রতিকার |
| A01 Broken Access Control | অ্যাডমিন পেজ সরাসরি URL-এ | অননুমোদিত প্রবেশ | সার্ভার-সাইড authorization |
| A03 Injection | SQL injection | ডেটা চুরি | Parameterized query |
| A07 XSS | Reflected XSS | সেশন চুরি | Output encoding + CSP |
EOF
cat owasp-findings.md
```

## ৮. প্রত্যাশিত আউটপুট

Juice Shop http://localhost:3000 -এ খুলবে (HTTP 200)। অন্তত ৫টি OWASP শ্রেণির বাস্তব উদাহরণসহ ফলাফল-তালিকা তৈরি হবে।

## ৯. যাচাইকরণ (Verification)

- Juice Shop-এর Score Board-এ আপনার সমাধান করা চ্যালেঞ্জগুলো দেখা যাচ্ছে কি?
- প্রতিটি ফলাফলে কি প্রমাণ (স্ক্রিনশট/অনুরোধ) ও প্রতিকার আছে?
- আপনার তালিকা কি Lab 1-এর থ্রেট মডেলের সঙ্গে সঙ্গতিপূর্ণ?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **পোর্ট 3000 ব্যস্ত:** `docker ps` দেখে পুরোনো কনটেইনার থামান, বা `-p 3001:3000` ব্যবহার করুন।
- **পেজ খুলছে না:** `docker logs juice-shop` দেখুন; কনটেইনার পুরোপুরি চালু হতে ২০–৩০ সেকেন্ড লাগতে পারে।

## ১১. নিরাপত্তা সতর্কতা

Juice Shop-এ কোনো আসল তথ্য ব্যবহার করবেন না। পরীক্ষা শেষে কনটেইনার মুছে ফেলুন যাতে দুর্বল অ্যাপটি চালু না থাকে।

## ১২. ক্লিনআপ (Cleanup)

```
docker stop juice-shop
docker rm juice-shop
docker ps -a --filter name=juice-shop
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. কোন OWASP শ্রেণিটি MERN অ্যাপে সবচেয়ে বেশি দেখা যায়?
2. একটি দুর্বলতা খুঁজে পাওয়া আর সেটি ঠিক করা — কোনটি কঠিন এবং কেন?
3. কীভাবে এই মূল্যায়নকে CI/CD-তে স্বয়ংক্রিয় করা যায়? (Lab 11-এর পূর্বাভাস)

ল্যাব ৩

# Lab 3 — Linux Server Security Audit

একটি প্রোডাকশন সার্ভারের নিরাপত্তা শুরু হয় অপারেটিং সিস্টেম থেকে। এই ল্যাবে আপনি Ubuntu Server 24.04 LTS-এর একটি ল্যাব VM অডিট করবেন — ব্যবহারকারী, SSH, ফায়ারওয়াল, আপডেট ও লগ পরীক্ষা করে দুর্বলতা খুঁজবেন ও ঠিক করবেন।

## ১. উদ্দেশ্য (Objectives)

- Lynis বা ম্যানুয়াল চেক দিয়ে সার্ভার অডিট চালাতে পারা।
- SSH hardening (কী-ভিত্তিক লগইন, root লগইন বন্ধ) করতে পারা।
- UFW ফায়ারওয়াল ও স্বয়ংক্রিয় আপডেট কনফিগার করা।
- অডিট ফলাফলের প্রতিবেদন লেখা।

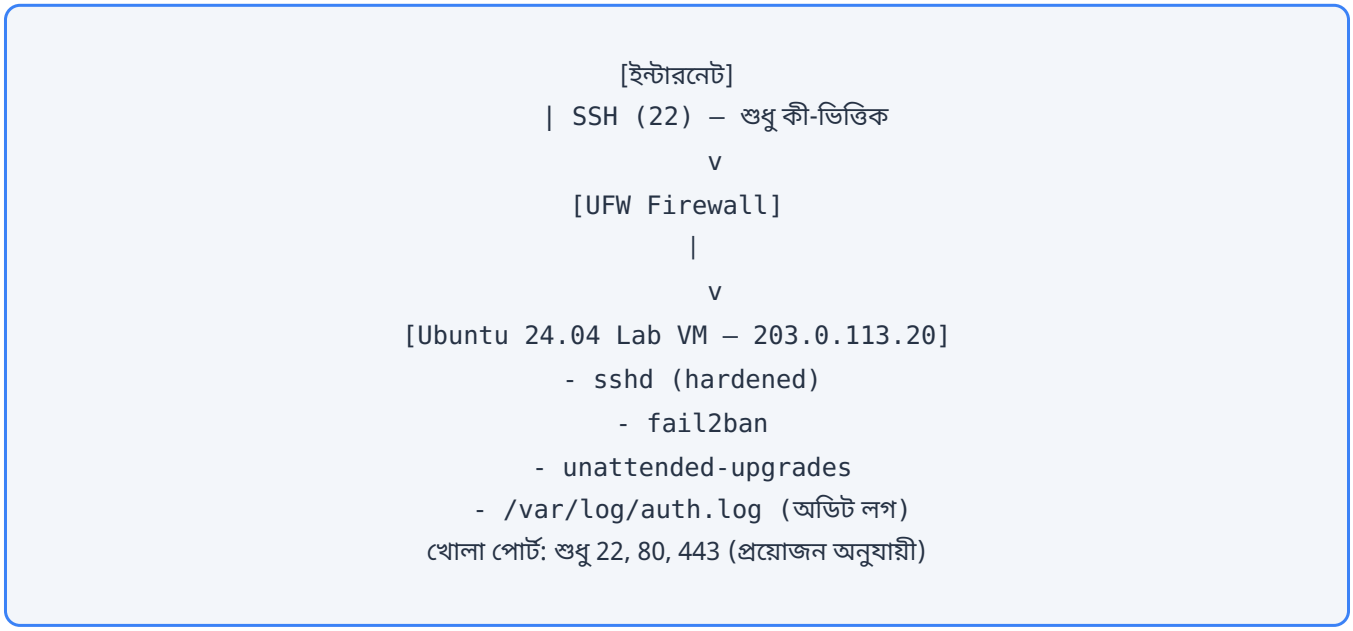
## ২. পূর্বশর্ত (Prerequisites)

- Ubuntu Server 24.04 LTS ল্যাব VM-এ sudo অধিকার।
- Linux কমান্ড লাইনের মৌলিক ধারণা।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

শুধু আপনার নিজের ল্যাব VM-এ কাজ করুন। প্রোডাকশন সার্ভারে পরীক্ষা করবেন না। পরিবর্তনের আগে sshd কনফিগারেশনের ব্যাকআপ নিন, যাতে নিজেকে লক-আউট না করেন।

## ৬. সেটআপ

```
sudo apt update && sudo apt install -y lynis ufw fail2ban unattended-upgrades
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
sudo cp /etc/passwd /tmp/passwd.bak
whoami; hostname; lsb_release -a
```

## ৭. কমান্ড ও কনফিগারেশন

```
# ১. অডিট চালান
sudo lynis audit system --quick 2>&1 | tee ~/lynis-report.txt

# ২. ব্যবহারকারী ও sudo পরীক্ষা
cat /etc/passwd | cut -d: -f1,3,7
grep -E "sudo|admin" /etc/group
sudo lastlog | head -20

# ৩. খোলা পোর্ট
sudo ss -tuln

# ৪. SSH hardening — /etc/ssh/sshd_config-এ যোগ করুন:
# PermitRootLogin no
# PasswordAuthentication no
# PubkeyAuthentication yes
# MaxAuthTries 3
sudo nano /etc/ssh/sshd_config
sudo sshd -t && sudo systemctl reload sshd

# ৫. UFW ফায়ারওয়াল
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
sudo ufw status verbose

# ৬. স্বয়ংক্রিয় নিরাপত্তা আপডেট
sudo dpkg-reconfigure -plow unattended-upgrades
```

## ৮. প্রত্যাশিত আউটপুট

Lynis রিপোর্টে hardening index ও সতর্কতার তালিকা; `ss -tuln`-এ শুধু প্রত্যাশিত পোর্ট; `ufw status`-এ সক্রিয় নিয়ম; root লগইন প্রত্যাখ্যাত হবে।

## ৯. যাচাইকরণ (Verification)

```
sudo lynis audit system --quick 2>&1 | grep -i "hardening index"
sudo ufw status verbose
sudo sshd -T | grep -i -E "permitrootlogin|passwordauthentication"
sudo grep "Failed password" /var/log/auth.log | tail -5
```

## ১০. সমস্যা সমাধান (Troubleshooting)

- **SSH-এ ঢুকতে পারছি না:** VM কনসোল থেকে `sshd_config.bak` ফিরিয়ে আনুন; `sshd -t` দিয়ে সিনট্যাক্স যাচাই করুন।
- **UFW চালু করলে সংযোগ বিচ্ছিন্ন:** আগে 22/tcp allow করেছেন কি না নিশ্চিত করুন; কনসোল থেকে `ufw disable` করুন।

> **TROUBLESHOOTING:** PasswordAuthentication বন্ধ করার আগে নিশ্চিত করুন আপনার SSH কী কাজ করছে — নতুন একটি টার্মিনালে লগইন পরীক্ষা না করে পুরোনো সেশন বন্ধ করবেন না।

## ১১. নিরাপত্তা সতর্কতা

SSH প্রাইভেট কী কখনো শেয়ার করবেন না, Git-এ কমিট করবেন না। fail2ban লগ নিয়মিত দেখুন।

## ১২. ক্লিনআপ (Cleanup)

```
# ল্যাব VM-এ পরিবর্তনগুলো ইচ্ছাকৃতভাবে রেখে দিন (এগুলোই hardening);
# শুধু অস্থায়ী ব্যাকআপ পরিষ্কার করুন
rm -f /tmp/passwd.bak
sudo ufw status
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. কোন Lynis সুপারিশটি সবচেয়ে বেশি ঝুঁকি কমায়?
2. স্বয়ংক্রিয় আপডেটের ঝুঁকি কী? কখন তা বন্ধ রাখা উচিত?
3. ফায়ারওয়াল নিয়ম কীভাবে সংস্করণ নিয়ন্ত্রণে (version control) রাখা যায়?

ল্যাব ৪

# Lab 4 — IAM and Least-Privilege Policy

IAM (Identity and Access Management) হলো কে কী করতে পারে তা নিয়ন্ত্রণের ব্যবস্থা। Least privilege (সর্বনিম্ন অনুমতি) নীতি মানে প্রত্যেক ব্যবহারকারী, সার্ভিস ও অ্যাপ্লিকেশনকে শুধু তার কাজের জন্য যতটুকু দরকার ততটুকুই অনুমতি দেওয়া — একটুও বেশি নয়। এই ল্যাবে আপনি TechNova-র জন্য ভূমিকা (role) ও নীতি (policy) ডিজাইন করবেন।

## ১. উদ্দেশ্য (Objectives)

- ভূমিকা-ভিত্তিক অ্যাক্সেস (RBAC) ডিজাইন করতে পারা।
- Least-privilege নীতি JSON/YAML-এ লেখা।
- অতিরিক্ত অনুমতি (over-privileged) অ্যাকাউন্ট শনাক্ত করা।
- অ্যাক্সেস পর্যালোচনার (access review) প্রক্রিয়া লেখা।

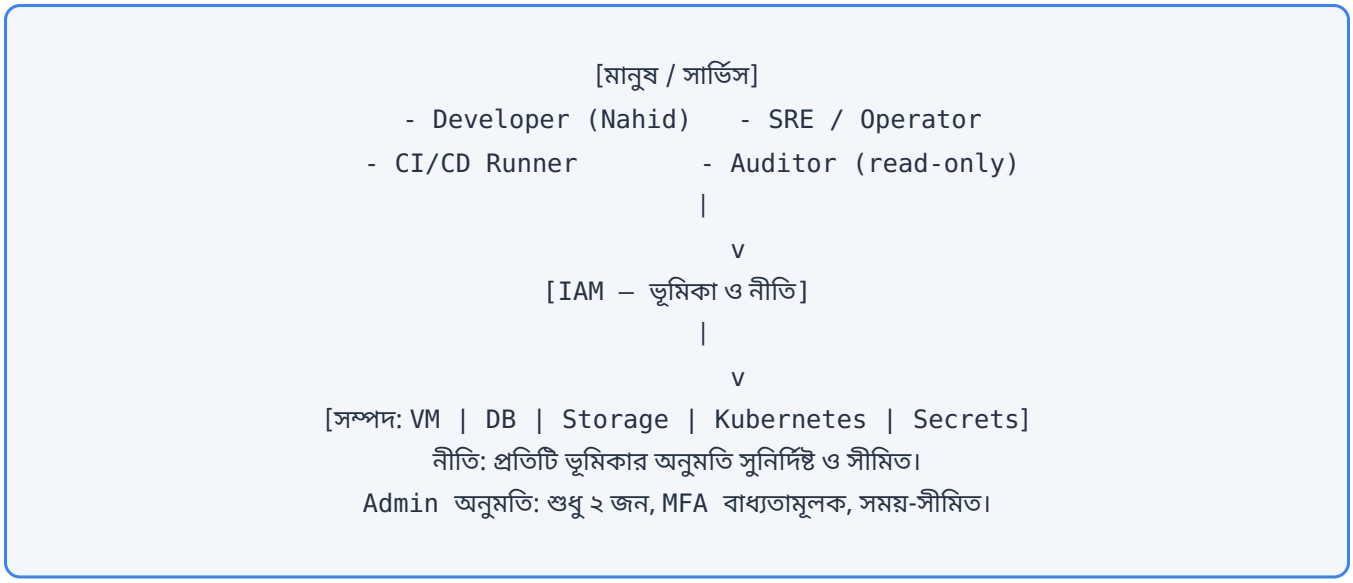
## ২. পূর্বশর্ত (Prerequisites)

- Lab 1 ও Lab 3 সম্পন্ন; ক্লাউড IAM-এর মৌলিক ধারণা।
- ল্যাব পরিবেশে একটি ক্লাউড অ্যাকাউন্ট বা স্থানীয় Keycloak/MinIO অনুকরণ — কোনোটিই না থাকলে কাগজে নীতি ডিজাইন করুন।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

ল্যাব ক্লাউড অ্যাকাউন্ট বা স্থানীয় অনুকরণে কাজ করুন। আসল প্রোডাকশন অ্যাকাউন্টে admin নীতি পরীক্ষা করবেন না। সব পরীক্ষামূলক অ্যাকাউন্টের নামে `lab-` উপসর্গ ব্যবহার করুন।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab04-iam
cd ~/technova-lab/lab04-iam
cat > roles.md <<'EOF'
Roles:
- developer: কোড পড়া/লেখা, dev পরিবেশে deploy, লগ পড়া
- sre-operator: প্রোডাকশন লগ/মেট্রিক পড়া, restart, সীমিত deploy
- ci-runner: শুধু build + নির্দিষ্ট registry-তে push
- auditor: সবকিছু read-only
- admin: সবকিছু, MFA বাধ্যতামূলক, break-glass পদ্ধতিতে
EOF
cat roles.md
```

## ৭. কমান্ড ও কনফিগারেশন

```
# উদাহরণ: ডেভেলপারের least-privilege নীতি (AWS IAM ধাঁচের JSON, ল্যাবের জন্য)
cat > policy-developer.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": ["arn:aws:s3:::technova-lab-*"]
    },
    {
      "Effect": "Deny",
      "Action": ["iam:*", "s3:DeleteBucket"],
      "Resource": ["*"]
    }
  ]
}
EOF
cat policy-developer.json

# স্থানীয়ভাবে নীতি যাচাই (jq থাকলে)
jq . policy-developer.json

# অ্যাক্সেস কী-এর বয়স পরীক্ষার ধারণা (ল্যাব অ্যাকাউন্টে):
# 90 দিনের পুরোনো কী তালিকাভুক্ত করে ঘোরানোর (rotate) পরিকল্পনা লিখুন।
```

> **CONCEPT:** Break-glass অ্যাকাউন্ট মানে জরুরি অবস্থায় ব্যবহারের জন্য আলাদা করে রাখা উচ্চ-অনুমতির অ্যাকাউন্ট — সাধারণত বন্ধ থাকে, ব্যবহার করলে তাৎক্ষণিক সতর্কবার্তা (alert) যায়।

## ৮. প্রত্যাশিত আউটপুট

- ৫টি ভূমিকার সংজ্ঞা, প্রতিটির জন্য নীতি ফাইল, এবং একটি অ্যাক্সেস-পর্যালোচনা চেকলিস্ট।
- অন্তত ৩টি over-privileged অনুমতি চিহ্নিত করে সংশোধনের প্রস্তাব।

## ৯. যাচাইকরণ (Verification)

- developer ভূমিকা কি admin কাজ (যেমন বাকেট মোছা) করতে পারে না? নীতিতে Deny আছে কি?
- প্রতিটি নীতিতে কি wildcard (`*`) অনুমতি এড়ানো হয়েছে?
- প্রতিটি অ্যাকাউন্টে MFA বাধ্যতামূলক কি না যাচাই করুন।

## ১০. সমস্যা সমাধান (Troubleshooting)

- **নীতি কাজ করছে না:** JSON সিনট্যাক্স (`jq .`) যাচাই করুন; Allow ও Deny-র ক্রম মনে রাখুন — স্পষ্ট Deny সবসময় জয়ী হয়।
- **CI runner ব্যর্থ হচ্ছে:** runner-এর ভূমিকায় শুধু প্রয়োজনীয় push অনুমতি আছে কি না দেখুন।

## ১১. নিরাপত্তা সতর্কতা

> **WARNING:** অ্যাক্সেস কী বা টোকেন কখনো চ্যাট, ইমেইল বা Git-এ পাঠাবেন না। ল্যাব কী-ও ব্যবহারের পর মুছে ফেলুন।

## ১২. ক্লিনআপ (Cleanup)

```
# ল্যাব পরীক্ষামূলক অ্যাকাউন্ট/কী মুছে ফেলুন (ক্লাউড কনসোল বা CLI থেকে)
# নীতি ফাইলগুলো সংরক্ষণ করুন — এগুলোই ডেলিভারেবল
ls -l ~/technova-lab/lab04-iam/
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. কেন ডেভেলপারকে প্রোডাকশন ডেটাবেসে লেখার অনুমতি দেওয়া ঝুঁকিপূর্ণ?
2. অ্যাক্সেস পর্যালোচনা কত ঘন ঘন করা উচিত?
3. সার্ভিস অ্যাকাউন্টের কী ঘোরানো (rotation) কেন মানুষের পাসওয়ার্ড বদলানোর চেয়ে গুরুত্বপূর্ণ?

ল্যাব ৫

# Lab 5 — Secrets Management Setup

Secrets (সিক্রেট) মানে পাসওয়ার্ড, API কী, JWT secret, ডেটাবেস ক্রেডেনশিয়াল — যা ফাঁস হলে আক্রমণকারী সিস্টেমে ঢুকতে পারে। এই ল্যাবে আপনি HashiCorp Vault (বা Docker Secrets) দিয়ে সিক্রেট নিরাপদে সংরক্ষণ, পড়া ও ঘোরানো শিখবেন। কোডে কখনো সিক্রেট হার্ডকোড করা যাবে না।

## ১. উদ্দেশ্য (Objectives)

- Vault dev মোডে চালিয়ে সিক্রেট সংরক্ষণ ও পড়া।
- অ্যাপ্লিকেশন থেকে পরিবেশ ভেরিয়েবল (environment variable) দিয়ে সিক্রেট পড়া।
- সিক্রেট rotation (ঘোরানো) ও lease (মেয়াদ) বোঝা।
- Git-এ সিক্রেট ফাঁস হয়েছে কি না পরীক্ষা করা।

## ২. পূর্বশর্ত (Prerequisites)

- Docker ও Linux CLI-র ধারণা; Lab 4-এর IAM ধারণা।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার




## ৫. নিরাপদ ল্যাব পরিবেশ

Vault dev মোড শুধু শেখার জন্য — এতে ডেটা মেমরিতে থাকে, প্রোডাকশনে ব্যবহার করবেন না। সব সিক্রেটের মান CHANGE-ME-... প্লেসহোল্ডার ব্যবহার করুন।

## ৬. সেটআপ

```
docker run -d --cap-add=IPC_LOCK --name vault \
  -p 8200:8200 \
  -e 'VAULT_DEV_ROOT_TOKEN_ID=CHANGE-ME-VAULT-ROOT-TOKEN' \
  hashicorp/vault:latest
sleep 5
export VAULT_ADDR='http://127.0.0.1:8200'
export VAULT_TOKEN='CHANGE-ME-VAULT-ROOT-TOKEN'
docker logs vault --tail 10
```

## ৭. কমান্ড ও কনফিগারেশন

```
# Vault CLI ইনস্টল (প্রয়োজনে) — অথবা docker exec ব্যবহার করুন
docker exec vault vault kv put secret/technova/app \
  DB_PASSWORD='CHANGE-ME-DB-PASSWORD' \
  JWT_SECRET='CHANGE-ME-JWT-SECRET' \
  STRIPE_KEY='CHANGE-ME-STRIPE-KEY'

docker exec vault vault kv get secret/technova/app

# অ্যাপের জন্য শুধু-পড়ার নীতি
cat > app-policy.hcl <<'EOF'
path "secret/data/technova/app" {
  capabilities = ["read"]
}
EOF
docker cp app-policy.hcl vault:/tmp/app-policy.hcl
docker exec vault vault policy write app-read-only /tmp/app-policy.hcl

# Git ইতিহাসে সিক্রেট আছে কি না পরীক্ষা (gitleaks ধাঁচে)
# আপনার রেপোতে চালান (শুধু নিজের রেপোতে):
# gitleaks detect --source . --verbose
grep -r "CHANGE-ME" ~/technova-lab/ | head
```

> **REAL WORLD:** বাস্তবে বহু নিরাপত্তা-ঘটনার শুরু GitHub-এ ফাঁস হওয়া একটি API কী থেকে। .env ফাইল .gitignore-এ রাখা এবং pre-commit hook-এ gitleaks চালানো একটি সস্তা কিন্তু শক্তিশালী প্রতিরক্ষা।

## ৮. প্রত্যাশিত আউটপুট

`vault kv get`-এ সংরক্ষিত কীগুলো দেখা যাবে; অ্যাপ-নীতিতে লেখার চেষ্টা প্রত্যাখ্যাত হবে; Git স্ক্যানে কোনো আসল সিক্রেট পাওয়া যাবে না।

## ৯. যাচাইকরণ (Verification)

```
docker exec vault vault kv get secret/technova/app
docker exec vault vault policy read app-read-only
docker exec vault vault token lookup | head -20
```

## ১০. সমস্যা সমাধান (Troubleshooting)

- **Vault-এ সংযোগ ব্যর্থ:** `VAULT_ADDR` ও পোর্ট 8200 যাচাই করুন; `docker ps` দেখুন।
- **Permission denied:** টোকেনের নীতি (policy) ঠিক আছে কি না দেখুন; root টোকেন দিয়ে পরীক্ষা করুন।

## ১১. নিরাপত্তা সতর্কতা

> **WARNING:** Dev মোডের root টোকেন ও unseal কী কখনো প্রোডাকশনে বা শেয়ার করা পরিবেশে ব্যবহার করবেন না। প্রোডাকশন Vault-এ TLS, auto-unseal ও অডিট লগ বাধ্যতামূলক।

## ১২. ক্লিনআপ (Cleanup)

```
docker stop vault
docker rm vault
unset VAULT_TOKEN VAULT_ADDR
# .env বা পরীক্ষামূলক সিক্রেট ফাইল থাকলে মুছে ফেলুন
rm -f ~/technova-lab/lab05-secrets/.env 2>/dev/null; echo "cleaned"
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. সিক্রেট ঘোরানো (rotation) না করলে কী ঝুঁকি থাকে?
2. পরিবেশ ভেরিয়েবল বনাম Vault — কখন কোনটি যথেষ্ট?
3. একটি সিক্রেট ফাঁস হলে প্রথম ৩টি পদক্ষেপ কী হওয়া উচিত?

ল্যাব ৬

# Lab 6 — SAST Source Code Scanning

SAST (Static Application Security Testing) মানে কোড চালানো ছাড়াই, শুধু সোর্স কোড পড়ে দুর্বলতা খোঁজা। এই ল্যাবে আপনি Semgrep বা SonarQube দিয়ে TechNova MERN কোডবেস স্ক্যান করবেন, ফলাফল ত্রিয়াজ (triage — কোনটি আসল, কোনটি ভুল সতর্কতা) করবেন ও ঠিক করবেন।

## ১. উদ্দেশ্য (Objectives)

- Semgrep দিয়ে Node.js/Express কোড স্ক্যান করা।
- ফলাফলের তীব্রতা (severity) অনুযায়ী অগ্রাধিকার দেওয়া।
- অন্তত ৩টি আসল দুর্বলতা ঠিক করে পুনঃস্ক্যানে তা দূর হওয়া দেখানো।
- False positive (ভুল সতর্কতা) চিহ্নিত করে দমন (suppress) করা।

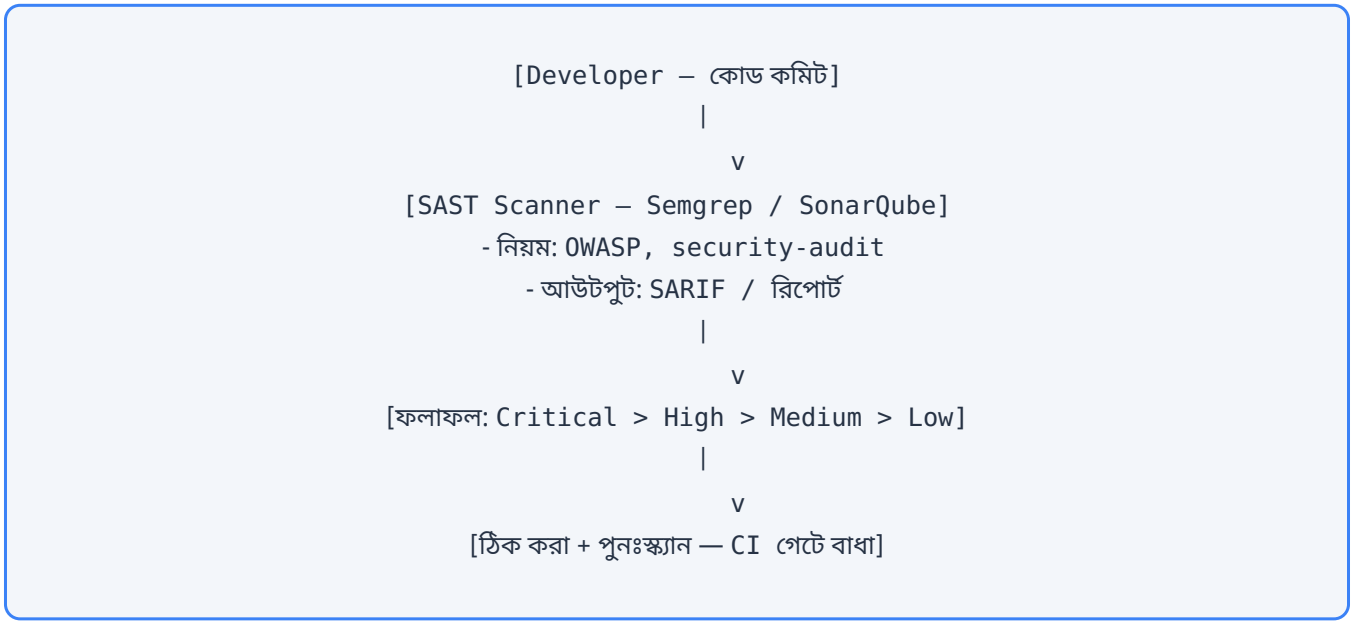
## ২. পূর্বশর্ত (Prerequisites)

- Python 3 ও pip; একটি নমুনা MERN রেপো (নিজের বা ল্যাবের)।
- Lab 2-এর OWASP ধারণা।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

শুধু নিজের বা ল্যাবের দেওয়া কোডবেস স্ক্যান করুন। অন্যের ব্যক্তিগত রেপো স্ক্যান করবেন না। স্ক্যান রিপোর্টে আসল সিক্রেট থাকলে তা মুছে ফেলুন।

## ৬. সেটআপ

```
pip install semgrep
semgrep --version
mkdir -p ~/technova-lab/lab06-sast/sample-app
cd ~/technova-lab/lab06-sast/sample-app
cat > app.js <<'EOF'
// ইচ্ছাকৃতভাবে দুর্বল নমুনা — শুধু ল্যাবের জন্য
const express = require('express');
const app = express();
app.get('/user', (req, res) => {
  const q = "SELECT * FROM users WHERE id = '" + req.query.id + "'";
  res.send("query: " + q);
});
app.listen(3000);
EOF
ls -l
```

## ৭. কমান্ড ও কনফিগারেশন

```
cd ~/technova-lab/lab06-sast/sample-app
semgrep --config auto --json --output semgrep-report.json .
semgrep --config auto . 2>&1 | tee semgrep-output.txt

# ফলাফল গুনুন
grep -c "severity" semgrep-report.json || true
cat semgrep-output.txt | head -60

# ঠিক করা: parameterized query ও validation ব্যবহার করুন
cat > app.fixed.js <<'EOF'
const express = require('express');
const app = express();
app.get('/user', (req, res) => {
  const id = Number(req.query.id);
  if (!Number.isInteger(id)) return res.status(400).send('Invalid id');
  // Parameterized query ব্যবহার করুন (যেমন db.query('SELECT ... WHERE id = ?', [id]))
  res.send('ok: ' + id);
});
app.listen(3000);
EOF
semgrep --config auto app.fixed.js
```

## ৮. প্রত্যাশিত আউটপুট

প্রথম স্ক্যানে SQL injection-ধাঁচের সতর্কতা (High/Critical) দেখা যাবে; ঠিক করা ফাইলে সেই সতর্কতা থাকবে না। রিপোর্ট JSON ফাইলে সংরক্ষিত হবে।

## ৯. যাচাইকরণ (Verification)

- ঠিক করার আগে ও পরে ফলাফলের সংখ্যা তুলনা করুন।
- প্রতিটি High ফলাফলের জন্য কি সিদ্ধান্ত লেখা আছে — ঠিক করা, স্বীকার করা, নাকি false positive?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **Semgrep নিয়ম ডাউনলোড হচ্ছে না:** ইন্টারনেট সংযোগ যাচাই করুন; `--config p/javascript` দিয়ে নির্দিষ্ট নিয়ম চেষ্টা করুন।
- **অনেক false positive:** নিয়ম সীমিত করুন ( `--config p/security-audit` ) এবং দলের সঙ্গে ত্রিয়াজ করুন।

## ১১. নিরাপত্তা সতর্কতা

SAST রিপোর্ট নিজেই সংবেদনশীল — এতে কোডের দুর্বলতার মানচিত্র থাকে। রিপোর্ট শেয়ার করার আগে অনুমতি নিন ও অ্যাক্সেস সীমিত রাখুন।

## ১২. ক্লিনআপ (Cleanup)

```
rm -rf ~/technova-lab/lab06-sast/sample-app/node_modules 2>/dev/null
ls ~/technova-lab/lab06-sast/
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. SAST কী ধরনের দুর্বলতা ধরতে পারে না? (উত্তরের ইঙ্গিত: Lab 7-এর DAST)
2. CI-তে SAST ব্যর্থ হলে ডেভেলপাররা কীভাবে প্রতিক্রিয়া জানাবে?
3. False positive বেশি হলে দল স্ক্যানারকে উপেক্ষা করতে শুরু করে কেন?

ল্যাব ৭

# Lab 7 — DAST Running Application Testing

DAST (Dynamic Application Security Testing) মানে চলমান অ্যাপ্লিকেশনকে বাইরে থেকে আক্রমণকারীর মতো পরীক্ষা করা। SAST কোড পড়ে, DAST অ্যাপ চালিয়ে পরীক্ষা করে। এই ল্যাবে আপনি OWASP ZAP দিয়ে নিজের ল্যাব অ্যাপ স্ক্যান করবেন।

## ১. উদ্দেশ্য (Objectives)

- OWASP ZAP দিয়ে baseline স্ক্যান চালানো।
- ZAP রিপোর্ট পড়ে আসল দুর্বলতা ও ভুল সতর্কতা আলাদা করা।
- অনুপস্থিত নিরাপত্তা হেডার (security header) চিহ্নিত ও ঠিক করা।

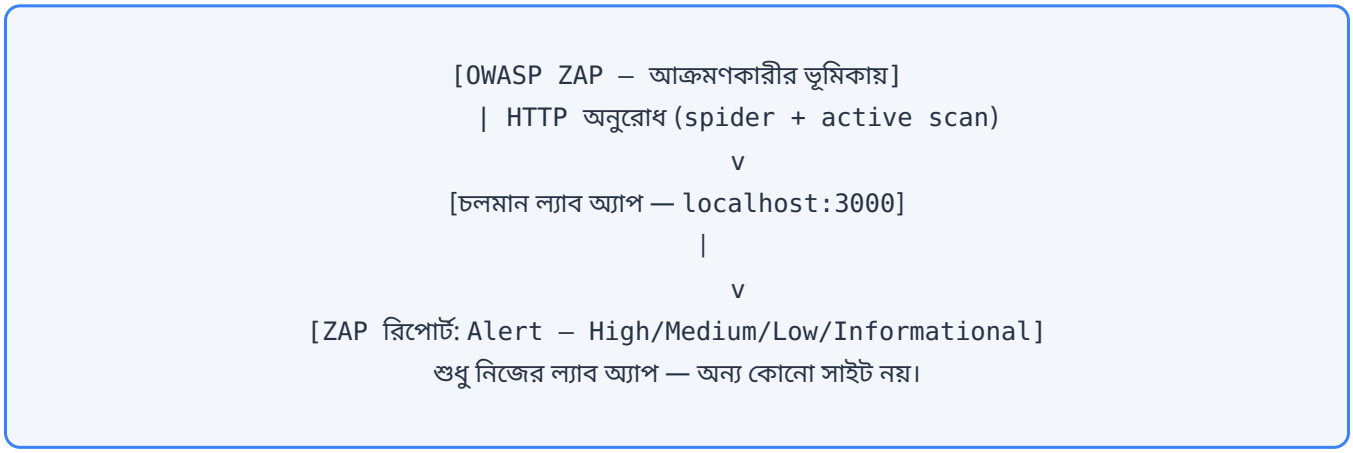
## ২. পূর্বশর্ত (Prerequisites)

- Docker; Lab 2 ও Lab 6 সম্পন্ন। একটি চলমান ল্যাব অ্যাপ (Juice Shop বা নিজের MERN অ্যাপ)।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার




## ৫. নিরাপদ ল্যাব পরিবেশ

**WARNING:** DAST স্ক্যান একটি চলমান সাইটে অনেক অনুরোধ পাঠায় — এটি প্রোডাকশন সাইটে সেবা ব্যাহত করতে পারে। শুধু নিজের ল্যাব অ্যাপে, ল্যাব নেটওয়ার্কে চালান। অন্য কারো সাইটে লিখিত অনুমতি ছাড়া কখনো active scan চালাবেন না।

## ৬. সেটআপ

```
docker pull ghcr.io/zaproxy/zaproxy:stable
# ল্যাব অ্যাপ চালু আছে কি না দেখুন (Juice Shop বা নিজের অ্যাপ)
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/ || echo "অ্যাপ চালু করুন"
mkdir -p ~/technova-lab/lab07-dast/reports
```

## ৭. কমান্ড ও কনফিগারেশন

```
# Baseline স্ক্যান (নিরাপদ, passive-ধাঁচের) — শুধু ল্যাব অ্যাপের URL-এ
docker run --rm --network host \
  -v ~/technova-lab/lab07-dast/reports:/zap/wrk:rw \
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://localhost:3000/ \
  -r zap-baseline-report.html -J zap-baseline-report.json

ls -lh ~/technova-lab/lab07-dast/reports/

# নিরাপত্তা হেডার পরীক্ষা
curl -s -I http://localhost:3000/ | grep -i -E "content-security-policy|x-frame-options|
strict-transport| x-content-type"
```

## ৮. প্রত্যাশিত আউটপুট

HTML ও JSON রিপোর্ট তৈরি হবে; অনুপস্থিত CSP, X-Frame-Options, HSTS-ধাঁচের সতর্কতা (Medium/Low) দেখা যাবে।

## ৯. যাচাইকরণ (Verification)

- রিপোর্টে High সতর্কতা আছে কি? থাকলে প্রতিটির প্রমাণ যাচাই করুন।
- Express-এ Helmet যোগ করার পর পুনঃস্ক্যানে হেডার-সতর্কতা কমে কি?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **ZAP টার্গেটে পৌঁছাতে পারছে না:** `--network host` ব্যবহার করেছেন কি না দেখুন; localhost-এর বদলে 127.0.0.1 চেষ্টা করুন।
- **স্ক্যান খুব ধীর:** baseline স্ক্যানেই সীমাবদ্ধ থাকুন; full active scan ল্যাবে সময় নেয়।

## ১১. নিরাপত্তা সতর্কতা

Active scan শুধু নিজের অ্যাপে। স্ক্যানের আগে ডেটা ব্যাকআপ নিন — DAST ফর্ম জমা দিয়ে পরীক্ষামূলক ডেটা তৈরি করতে পারে।

## ১২. ক্লিনআপ (Cleanup)

```
docker stop juice-shop 2>/dev/null || true
# রিপোর্ট সংরক্ষণ করুন; স্ক্যান করা অ্যাপ বন্ধ করুন
ls ~/technova-lab/lab07-dast/reports/
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. SAST বনাম DAST — কোনটি কখন বেশি কার্যকর?
2. কেন DAST-কে CI/CD-তে প্রতিটি কমিটে না চালিয়ে রাতে (nightly) চালানো হয়?
3. একটি High সতর্কতা false positive প্রমাণিত হলে কী করবেন?

ল্যাব ৮

# Lab 8 — Dependency and SCA Scanning

SCA (Software Composition Analysis) মানে আপনার অ্যাপ যে তৃতীয় পক্ষের লাইব্রেরি/প্যাকেজ ব্যবহার করে, সেগুলোতে জানা দুর্বলতা (known vulnerability) আছে কি না খোঁজা। আধুনিক অ্যাপের বেশিরভাগ কোডই অন্যের লেখা — তাই নির্ভরতা (dependency) পরীক্ষা করা অপরিহার্য।

## ১. উদ্দেশ্য (Objectives)

- npm audit ও Snyk/Trivy দিয়ে dependency স্ক্যান করা।
- CVSS স্কোর (দুর্বলতার তীব্রতার মান) পড়ে অগ্রাধিকার দেওয়া।
- দুর্বল প্যাকেজ আপডেট করে পুনঃস্ক্যান করা।
- Dependabot-ধাঁচের স্বয়ংক্রিয় আপডেট নীতি লেখা।

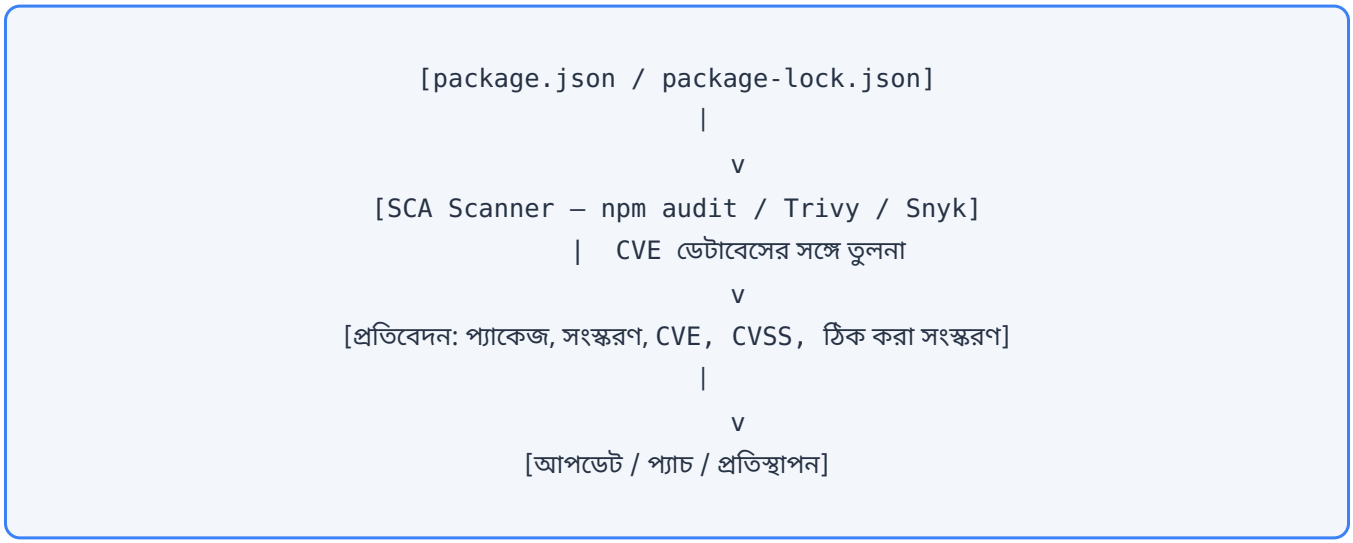
## ২. পূর্বশর্ত (Prerequisites)

- Node.js ও npm; একটি নমুনা package.json সহ প্রজেক্ট।

## ৩. আনুমানিক সময়

২ ঘণ্টা ৩০ মিনিট।

## ৪. আর্কিটেকচার




## ৫. নিরাপদ ল্যাব পরিবেশ

শুধু নিজের প্রজেক্ট স্ক্যান করুন। স্ক্যানে কোনো সিক্রেট পাঠানো হয় না তা নিশ্চিত করুন। আপডেটের আগে `package-lock.json`-এর ব্যাকআপ নিন।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab08-sca/demo
cd ~/technova-lab/lab08-sca/demo
npm init -y
# ইচ্ছাকৃতভাবে পুরোনো সংস্করণ (শুধু ল্যাবের জন্য — দুর্বলতা দেখতে)
npm install lodash@4.17.15 --save 2>&1 | tail -5
cat package.json
```

## ৭. কমান্ড ও কনফিগারেশন

```
cd ~/technova-lab/lab08-sca/demo
npm audit --json > npm-audit.json 2>&1 || true
npm audit 2>&1 | head -60

# Trivy দিয়ে ফাইলসিস্টেম স্ক্যান
docker run --rm -v "$PWD":/app aquasec/trivy:latest fs --format table /app

# ঠিক করা: নিরাপদ সংস্করণে আপডেট
npm audit fix 2>&1 | tail -10
npm audit 2>&1 | tail -20
```

> **CONCEPT:** CVE হলো জানা দুর্বলতার পরিচয়-নম্বর (যেমন CVE-2021-23337) আর CVSS হলো তার তীব্রতার স্কোর ০–১০। CVSS ৯-এর উপরে মানে Critical — তাৎক্ষণিক ব্যবস্থা নিন।

## ৮. প্রত্যাশিত আউটপুট

`npm audit`-এ দুর্বলতার সংখ্যা ও তীব্রতা; Trivy টেবিলে প্যাকেজ-ভিত্তিক ফলাফল; `npm audit fix`-এর পর সংখ্যা কমে যাওয়া।

## ৯. যাচাইকরণ (Verification)

- আপডেটের পর অ্যাপের টেস্ট কি এখনো পাস করে? (`npm test` চালান)
- কোনো Critical দুর্বলতা অবশিষ্ট থাকলে তার জন্য ব্যতিক্রম (exception) ও সময়সীমা লেখা আছে কি?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **npm audit fix ভেঙে ফেলেছে:** ব্যাকআপ থেকে `package-lock.json` ফিরিয়ে আনুন; major সংস্করণ আপডেট ম্যানুয়ালি পরীক্ষা করুন।
- **Trivy ডেটাবেস ডাউনলোড ব্যর্থ:** নেটওয়ার্ক যাচাই করে আবার চালান।

## ১১. নিরাপত্তা সতর্কতা

অপরিচিত প্যাকেজ ইনস্টল করবেন না। `npm install`-এর আগে প্যাকেজের নাম, ডাউনলোড সংখ্যা ও রক্ষণাবেক্ষণ যাচাই করুন — typosquatting (নামের বানান ভুলিয়ে নকল প্যাকেজ) একটি বাস্তব আক্রমণ।

## ১২. ক্লিনআপ (Cleanup)

```
cd ~/technova-lab/lab08-sca/demo
rm -rf node_modules
ls -l
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. একটি Critical দুর্বলতা, যার কোনো প্যাচ নেই — তখন কী করবেন?
2. কেন lock ফাইল (package-lock.json) কমিট করা গুরুত্বপূর্ণ?
3. SCA স্ক্যান কত ঘন ঘন চালানো উচিত?

ল্যাব ৯

# Lab 9 — Docker Image Vulnerability Scanning

কনটেইনার ইমেজে অ্যাপ কোড ছাড়াও অপারেটিং সিস্টেম প্যাকেজ থাকে — সেগুলোতেও দুর্বলতা থাকতে পারে। এই ল্যাবে আপনি Trivy দিয়ে Docker ইমেজ স্ক্যান করবেন, ছোট ও নিরাপদ বেস ইমেজ বেছে নেবেন এবং মাল্টি-স্টেজ বিল্ড (multi-stage build) লিখবেন।

## ১. উদ্দেশ্য (Objectives)

- Trivy দিয়ে ইমেজ স্ক্যান করা ও ফলাফল ব্যাখ্যা করা।
- Alpine/slim বেস ইমেজ ও মাল্টি-স্টেজ বিল্ড ব্যবহার করা।
- non-root ব্যবহারকারী হিসেবে কনটেইনার চালানো।
- CI-তে ইমেজ স্ক্যান গেট যোগ করার পরিকল্পনা লেখা।

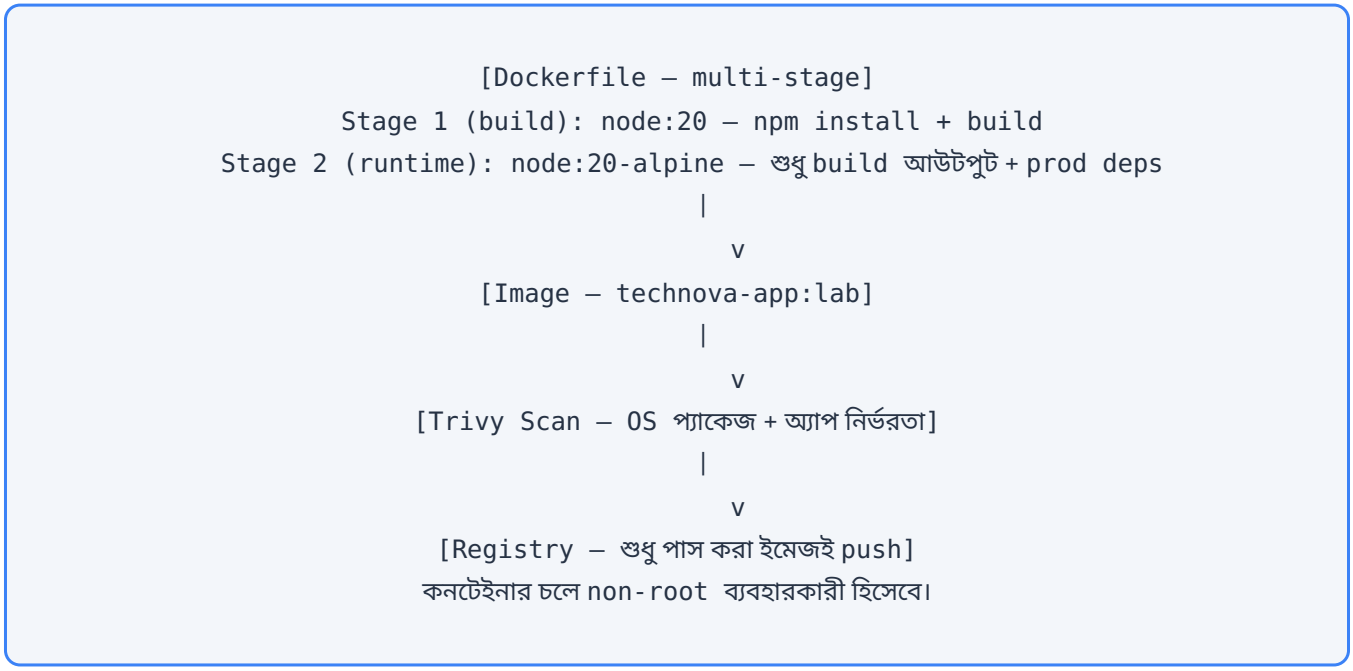
## ২. পূর্বশর্ত (Prerequisites)

- Docker; Lab 8 সম্পন্ন।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

শুধু ল্যাব রেজিস্ট্রি/লোকাল ইমেজ ব্যবহার করুন। অপরিচিত Docker Hub ইমেজ চালাবেন না — ইমেজেও ম্যালওয়্যার থাকতে পারে।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab09-docker/app
cd ~/technova-lab/lab09-docker/app
cat > package.json <<'EOF'
{ "name": "technova-app", "version": "1.0.0",
  "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.19.2" } }
EOF
cat > server.js <<'EOF'
const express = require('express');
const app = express();
app.get('/health', (req, res) => res.json({ status: 'ok' }));
app.listen(3000);
EOF
ls -l
```

## ৭. কমান্ড ও কনফিগারেশন

```
cd ~/technova-lab/lab09-docker/app
cat > Dockerfile <<'EOF'
# Stage 1 — build
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
# Stage 2 — runtime (ছোট, নিরাপদ)
FROM node:20-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/package.json /app/server.js ./
USER appuser
EXPOSE 3000
CMD ["node", "server.js"]
EOF

docker build -t technova-app:lab .
docker run -d --name technova-lab-app -p 3000:3000 technova-app:lab
curl -s http://localhost:3000/health

# Trivy স্ক্যান
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:latest image --severity HIGH,CRITICAL technova-app:lab
```

## ৮. প্রত্যাশিত আউটপুট

ইমেজ তৈরি হবে; `/health` থেকে `{"status":"ok"}` ফিরবে; Trivy রিপোর্টে HIGH/CRITICAL দুর্বলতার সংখ্যা দেখা যাবে (Alpine বেসে সাধারণত কম)।

## ৯. যাচাইকরণ (Verification)

```
docker ps --filter name=technova-lab-app
docker exec technova-lab-app whoami
# প্রত্যাশা: appuser (root নয়)
docker images technova-app:lab --format "{{.Size}}"
```

## ১০. সমস্যা সমাধান (Troubleshooting)

- **বিল্ড ব্যর্থ:** Dockerfile-এর প্রতিটি ধাপ আলাদা করে চালান; `npm ci` -এর জন্য package-lock.json আছে কি না দেখুন।
- **কনটেইনার সাথে সাথে বন্ধ:** `docker logs technova-lab-app` দেখুন; পোর্ট সংঘাত যাচাই করুন।

**TROUBLESHOOTING:** `USER appuser` -এর পর ফাইল পড়তে না পারলে COPY-র সময় মালিকানা ঠিক করুন ( `--chown=appuser:appgroup` )।

## ১১. নিরাপত্তা সতর্কতা

কনটেইনারকে কখনো root হিসেবে চালাবেন না, যদি না একান্ত প্রয়োজন হয়। `--privileged` ফ্ল্যাগ ল্যাবে এড়িয়ে চলুন। ইমেজে সিক্রেট (API কী, .env) COPY করবেন না।

## ১২. ক্লিনআপ (Cleanup)

```
docker stop technova-lab-app
docker rm technova-lab-app
docker rmi technova-app:lab
docker ps -a
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. মাল্টি-স্টেজ বিল্ড কেন নিরাপত্তা বাড়ায়?
2. একটি Critical দুর্বলতা থাকা ইমেজ কি কখনো deploy করা উচিত? কখন ব্যতিক্রম দেওয়া যায়?
3. বেস ইমেজ আপডেটের দায়িত্ব কার?

ল্যাব ১০

# Lab 10 — Kubernetes Security Audit

Kubernetes (K8s) শক্তিশালী, কিন্তু ভুল কনফিগারেশনে তা বড় ঝুঁকি হয়। এই ল্যাবে আপনি kind বা Minikube-এ একটি স্থানীয় ক্লাস্টার তৈরি করে kube-bench ও kubeaudit-ধাঁচের চেক চালাবেন, RBAC ও Pod Security Standards পরীক্ষা করবেন।

## ১. উদ্দেশ্য (Objectives)

- kind/Minikube-এ স্থানীয় ক্লাস্টার চালানো।
- RBAC (Role-Based Access Control) ভূমিকা ও বাইন্ডিং পরীক্ষা করা।
- Pod Security Standards (restricted) প্রয়োগ করা।
- অনিরাপদ পড (privileged, hostPath) চিহ্নিত করা।

## ২. পূর্বশর্ত (Prerequisites)

- Docker; kubectl ইনস্টল করা; Lab 9 সম্পন্ন।

## ৩. আনুমানিক সময়

৪ ঘণ্টা।

## ৪. আর্কিটেকচার

```
[Control Plane — kind / Minikube, স্থানীয়]
                    |
                    v
              [Worker Node]
        - Namespace: technova-lab
- Pod: technova-app (non-root, read-only FS)
        - ServiceAccount: সীমিত অনুমতি
      - NetworkPolicy: শুধু প্রয়োজনীয় ট্রাফিক
[RBAC: Role + RoleBinding — কে কী করতে পারে]
```

## ৫. নিরাপদ ল্যাব পরিবেশ

শুধু স্থানীয় kind/Minikube ক্লাস্টার ব্যবহার করুন। কোনো বাস্তব ক্লাস্টারে পরীক্ষা করবেন না। ক্লাস্টারে আসল ডেটা বা সিক্রেট রাখবেন না।

## ৬. সেটআপ

```
# kind ইনস্টল (প্রয়োজনে) ও ক্লাস্টার তৈরি
kind create cluster --name technova-lab --wait 3m
kubectl cluster-info
kubectl get nodes
kubectl create namespace technova-lab
kubectl config set-context --current --namespace=technova-lab
```

## ৭. কমান্ড ও কনফিগারেশন

```
# ১. RBAC পরীক্ষা — কে cluster-admin?
kubectl get clusterrolebindings -o wide | grep -i admin
kubectl auth can-i --list --namespace=technova-lab

# ২. অনিরাপদ পড খুঁজুন
kubectl get pods -A -o json | grep -i privileged || echo "কোনো privileged পড পাওয়া যায়নি (ভালো)"

# ৩. নিরাপদ পড ডেপ্লয় করুন
cat > secure-pod.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: technova-app
  namespace: technova-lab
  labels:
    app: technova
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: app
    image: nginx:1.27-alpine
    ports:
    - containerPort: 80
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
EOF
kubectl apply -f secure-pod.yaml
kubectl get pod technova-app -n technova-lab

# ৪. NetworkPolicy উদাহরণ
cat > netpol.yaml <<'EOF'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: technova-deny-all
  namespace: technova-lab
spec:
  podSelector: {}
```

```
  policyTypes: ["Ingress", "Egress"]
EOF
kubectl apply -f netpol.yaml
```

> **CONCEPT:** Pod Security Standards-এর restricted প্রোফাইল মানে পড root হতে পারবে না, privilege বাড়াতে পারবে না, সব capability বাদ দেবে — আক্রমণকারী পডে ঢুকলেও হোস্টে পৌঁছাতে পারবে না।

## ৮. প্রত্যাশিত আউটপুট

ক্লাস্টার Ready অবস্থায়; নিরাপদ পড Running; `auth can-i` তালিকায় অপ্রয়োজনীয় অনুমতি নেই; NetworkPolicy প্রয়োগ হয়েছে।

## ৯. যাচাইকরণ (Verification)

```
kubectl get pods -n technova-lab
kubectl describe pod technova-app -n technova-lab | grep -i -E "runAsNonRoot|allowPrivilege"
kubectl get networkpolicy -n technova-lab
kubectl get roles,rolebindings -n technova-lab
```

## ১০. সমস্যা সমাধান (Troubleshooting)

- **Pod Pending:** `kubectl describe pod` দেখুন — ইমেজ টানতে না পারা বা resource সীমা সাধারণ কারণ।
- **NetworkPolicy-র পর সংযোগ বন্ধ:** deny-all নীতি প্রয়োগের পর প্রয়োজনীয় allow নিয়ম যোগ করুন; ল্যাবে প্রথমে নীতি ছাড়া পরীক্ষা করুন।

## ১১. নিরাপত্তা সতর্কতা

কখনো cluster-admin ভূমিকা অ্যাপের ServiceAccount-কে দেবেন না। Secrets-কে environment variable-এর বদলে ভলিউম মাউন্ট বা Vault দিয়ে দিন (Lab 5 দেখুন)।

## ১২. ক্লিনআপ (Cleanup)

```
kubectl delete pod technova-app -n technova-lab
kubectl delete -f netpol.yaml
kind delete cluster --name technova-lab
docker ps
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. একটি পড আপস (compromise) হলে RBAC কীভাবে ক্ষতি সীমিত করে?
2. NetworkPolicy ছাড়া ক্লাস্টারের ভিতরে কী ঝুঁকি থাকে?
3. কোন চেকগুলো CI-তে স্বয়ংক্রিয় করা যায়?

ল্যাব ১১

# Lab 11 — Security Gates in CI/CD

Security gate (নিরাপত্তা ফটক) মানে CI/CD পাইপলাইনে এমন একটি ধাপ, যা ব্যর্থ হলে কোড সামনে এগোতে পারে না — যেমন SAST-এ Critical পাওয়া গেলে deploy বন্ধ। এই ল্যাবে আপনি GitHub Actions-এ SAST, SCA ও ইমেজ স্ক্যান গেট যোগ করবেন।

## ১. উদ্দেশ্য (Objectives)

- GitHub Actions ওয়ার্কফ্লোতে নিরাপত্তা ধাপ যোগ করা।
- গেটের ব্যর্থতার নীতি (fail-closed বনাম fail-open) বোঝা।
- Pull request-এ নিরাপত্তা ফলাফল দেখানো।

## ২. পূর্বশর্ত (Prerequisites)

- GitHub অ্যাকাউন্ট ও একটি ল্যাব রেপো; Lab 6, 8, 9 সম্পন্ন।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার

```
[Push / Pull Request]
        |
        v
   [CI Pipeline]
  1. Lint + Unit Test
  2. SAST (Semgrep) — Critical পেলে FAIL
  3. SCA (npm audit) — High+ পেলে FAIL
  4. Build Docker Image
  5. Image Scan (Trivy) — Critical পেলে FAIL
  6. SBOM (পরের ল্যাব)
        |
        v
[Deploy — শুধু সব গেট পাস করলে]
```

## ৫. নিরাপদ ল্যাব পরিবেশ

শুধু নিজের ল্যাব রেপোতে কাজ করুন। GitHub Secrets-এ আসল সিক্রেট রাখবেন না — CHANGE-ME-... প্লেসহোল্ডার ব্যবহার করুন।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab11-cicd/.github/workflows
cd ~/technova-lab/lab11-cicd
git init 2>/dev/null || true
ls .github/workflows/
```

## ৭. কমান্ড ও কনফিগারেশন

```
cat > .github/workflows/security.yml <<'EOF'
name: Security Gates
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
      - name: Install dependencies
        run: npm ci
      - name: SAST with Semgrep
        uses: returntocorp/semgrep-action@v1
        with:
          config: p/security-audit
      - name: SCA with npm audit
        run: npm audit --audit-level=high
      - name: Build image
        run: docker build -t technova-app:${{ github.sha }} .
      - name: Image scan with Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: technova-app:${{ github.sha }}
          format: table
          exit-code: '1'
          severity: CRITICAL,HIGH
EOF
cat .github/workflows/security.yml
```

**CONCEPT:** Fail-closed মানে সন্দেহ হলে বন্ধ করে দেওয়া (নিরাপদ); fail-open মানে স্ক্যানার ব্যর্থ হলে তবুও এগিয়ে যাওয়া (ঝুঁকিপূর্ণ)। নিরাপত্তা গেট সবসময় fail-closed হওয়া উচিত।

## ৮. প্রত্যাশিত আউটপুট

ইচ্ছাকৃত দুর্বল কোড push করলে পাইপলাইন লাল (failed) হবে; ঠিক করা কোডে সবুজ (passed) হবে। প্রতিটি ধাপের লগ GitHub Actions-এ দেখা যাবে।

## ৯. যাচাইকরণ (Verification)

- GitHub Actions ট্যাবে প্রতিটি ধাপের অবস্থা দেখুন।
- Trivy ধাপ কি Critical পেলে সত্যিই থামে?
- PR-এ নিরাপত্তা মন্তব্য/চেক দেখা যাচ্ছে কি?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **npm audit সবসময় ব্যর্থ:** `--audit-level=high` ঠিক আছে কি না দেখুন; সাময়িকভাবে `|| true` দিয়ে নয়, ব্যতিক্রম নীতি লিখে সমাধান করুন।
- **Trivy অ্যাকশন পুরোনো সংস্করণ:** অ্যাকশনের সংস্করণ পিন করুন (যেমন @0.20.0) — latest ব্যবহার এড়ান।

## ১১. নিরাপত্তা সতর্কতা

CI লগে সিক্রেট ছাপা হতে পারে — `set -x` বা verbose লগে টোকেন দেখাবেন না। তৃতীয় পক্ষের অ্যাকশন ব্যবহারের আগে তার সোর্স যাচাই করুন।

## ১২. ক্লিনআপ (Cleanup)

```
# ল্যাব রেপোটি GitHub-এ private রাখুন; পরীক্ষামূলক branch মুছে ফেলুন
git branch -d test-vuln 2>/dev/null || true
git status
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. কোন গেটটি সবচেয়ে বেশি ভুল সতর্কতা (false positive) দেয়?
2. জরুরি hotfix-এর জন্য গেট বাইপাস করার নিরাপদ পদ্ধতি কী?
3. গেট ব্যর্থ হলে ডেভেলপারকে কত দ্রুত জানানো উচিত?

ল্যাব ১২

# Lab 12 — SBOM Generation and Artifact Signing

SBOM (Software Bill of Materials) হলো আপনার সফটওয়্যারের উপাদানের তালিকা — কোন লাইব্রেরি, কোন সংস্করণ, কোথা থেকে এসেছে। Artifact signing (স্বাক্ষর) প্রমাণ করে ইমেজটি আসলেই আপনার CI থেকে এসেছে, মাঝপথে বদলায়নি। একসঙ্গে এগুলো সরবরাহ-শৃঙ্খল (supply chain) নিরাপত্তার ভিত্তি।

## ১. উদ্দেশ্য (Objectives)

- Syft দিয়ে SBOM (SPDX/CycloneDX) তৈরি করা।
- Cosign দিয়ে ইমেজে স্বাক্ষর (sign) ও যাচাই (verify) করা।
- SBOM থেকে দুর্বলতা খুঁজে বের করা।

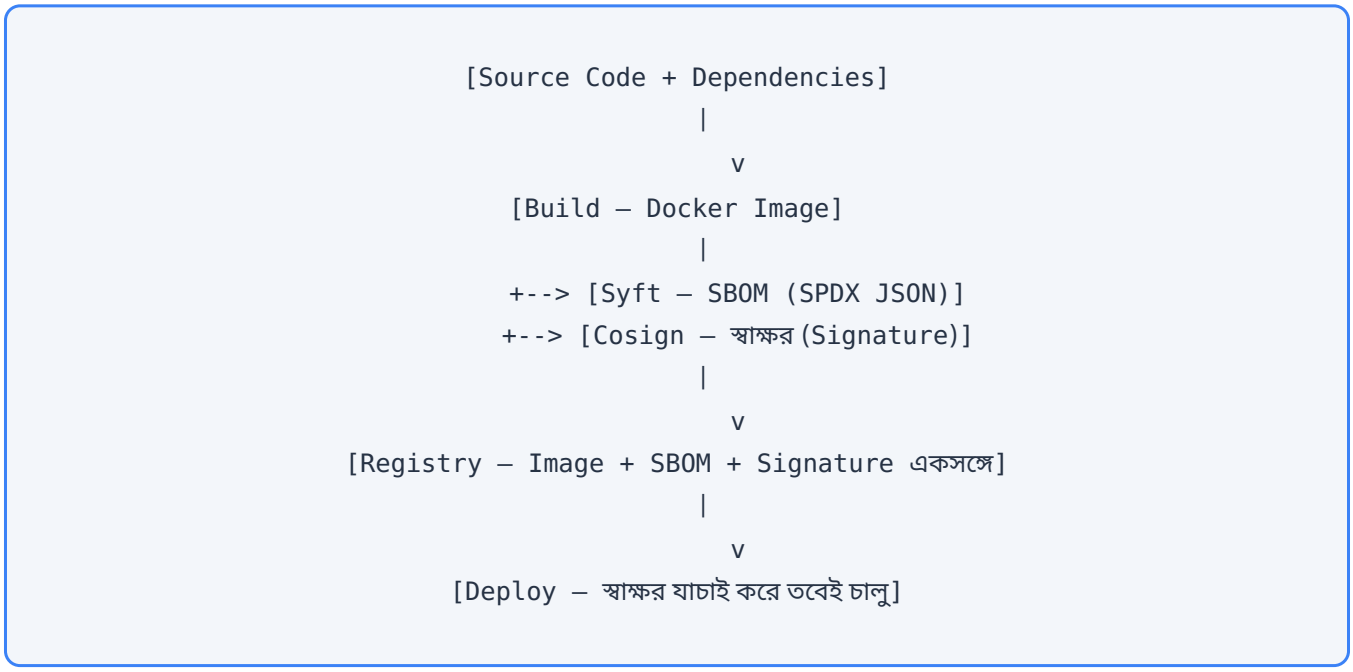
## ২. পূর্বশর্ত (Prerequisites)

- Docker; Lab 9 সম্পন্ন; একটি ল্যাব ইমেজ।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার




## ৫. নিরাপদ ল্যাব পরিবেশ

স্বাক্ষরের কী (signing key) শুধু ল্যাবে তৈরি করুন, কখনো শেয়ার করবেন না। প্রোডাকশনে keyless স্বাক্ষর (OIDC) ব্যবহারের কথা ভাবুন।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab12-sbom
cd ~/technova-lab/lab12-sbom
# Syft ও Cosign (ল্যাব VM-এ বাইনারি ডাউনলোড)
curl -sSfL https://raw.githubusercontent.com/anchore/syft/main/install.sh | sh -s -- -b /usr/
local/bin
cosign version 2>/dev/null || echo "cosign ইনস্টল করুন — docs.sigstore.dev দেখুন"
docker images technova-app:lab 2>/dev/null || docker pull nginx:1.27-alpine
```

## ৭. কমান্ড ও কনফিগারেশন

```
cd ~/technova-lab/lab12-sbom
# ১. SBOM তৈরি (SPDX JSON)
syft packages nginx:1.27-alpine -o spdx-json > sbom.spdx.json
head -40 sbom.spdx.json
syft packages nginx:1.27-alpine -o cyclonedx-json > sbom.cdx.json

# ২. SBOM থেকে দুর্বলতা পরীক্ষা (Grype)
docker run --rm -v "$PWD":/work anchore/grype:latest sbom:/work/sbom.spdx.json

# ৩. Cosign কী তৈরি (ল্যাবের জন্য; পাসওয়ার্ড: CHANGE-ME-COSIGN-PASSWORD)
cosign generate-key-pair
ls -l cosign.*

# ৪. ইমেজে স্বাক্ষর ও যাচাই (লোকাল রেজিস্ট্রি/ল্যাব রেজিস্ট্রিতে push-এর পর)
# cosign sign --key cosign.key technova.example/technova-app:lab
# cosign verify --key cosign.pub technova.example/technova-app:lab
echo "স্বাক্ষর ধাপটি ল্যাব রেজিস্ট্রি থাকলে চালান; না থাকলে কী তৈরিই ডেলিভারেবল"
```

## ৮. প্রত্যাশিত আউটপুট

SBOM JSON ফাইলে প্যাকেজের তালিকা (নাম, সংস্করণ, লাইসেন্স); Grype-তে দুর্বলতার তালিকা; cosign কী-জোড়া তৈরি হবে।

## ৯. যাচাইকরণ (Verification)

```
jq '.packages | length' sbom.spdx.json
jq -r '.packages[].name' sbom.spdx.json | head -20
ls -lh cosign.* sbom.*
```

## ১০. সমস্যা সমাধান (Troubleshooting)

- **Syft ইমেজ খুঁজে পাচ্ছে না:** ইমেজ লোকালে pull করা আছে কি না `docker images` দিয়ে দেখুন।
- **Cosign পাসওয়ার্ড চাইছে:** কী তৈরির সময় দেওয়া পাসওয়ার্ড ব্যবহার করুন; ল্যাবে সহজ পাসওয়ার্ড রাখলেও প্রোডাকশনে শক্তিশালী পাসওয়ার্ড আবশ্যক।

## ১১. নিরাপত্তা সতর্কতা

স্বাক্ষরের প্রাইভেট কী ফাঁস হলে আক্রমণকারী আপনার নামে ইমেজে স্বাক্ষর করতে পারবে — কী হারালে বা ফাঁস হলে তাৎক্ষণিকভাবে প্রত্যাহার (revoke) করুন।

## ১২. ক্লিনআপ (Cleanup)

```
# ল্যাব কী নিরাপদে সংরক্ষণ করুন বা মুছে ফেলুন (পরীক্ষার কী প্রোডাকশনে ব্যবহার করবেন না)
rm -f cosign.key cosign.pub
ls ~/technova-lab/lab12-sbom/
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. SBOM না থাকলে Log4Shell-ধাঁচের ঘটনায় কী সমস্যা হয়?
2. স্বাক্ষর যাচাই ছাড়া deploy করলে কী ঝুঁকি থাকে?
3. SBOM কত ঘন ঘন হালনাগাদ করা উচিত?

ল্যাব ১৩

# Lab 13 — Terraform and Kubernetes Policy as Code

Policy as Code মানে নিরাপত্তা নিয়মকে কোড হিসেবে লেখা, যাতে তা স্বয়ংক্রিয়ভাবে পরীক্ষা করা যায়। এই ল্যাবে আপনি Terraform দিয়ে অবকাঠামো (infrastructure) তৈরি করবেন এবং OPA/Conftest দিয়ে Kubernetes ম্যানিফেস্টে নীতি প্রয়োগ করবেন — যেমন “privileged পড নিষিদ্ধ”।

## ১. উদ্দেশ্য (Objectives)

- Terraform দিয়ে ল্যাব অবকাঠামো (নেটওয়ার্ক, VM) সংজ্ঞায়িত করা।
- OPA Rego নীতি লিখে ম্যানিফেস্ট যাচাই করা।
- Conftest দিয়ে CI-তে নীতি পরীক্ষা চালানো।

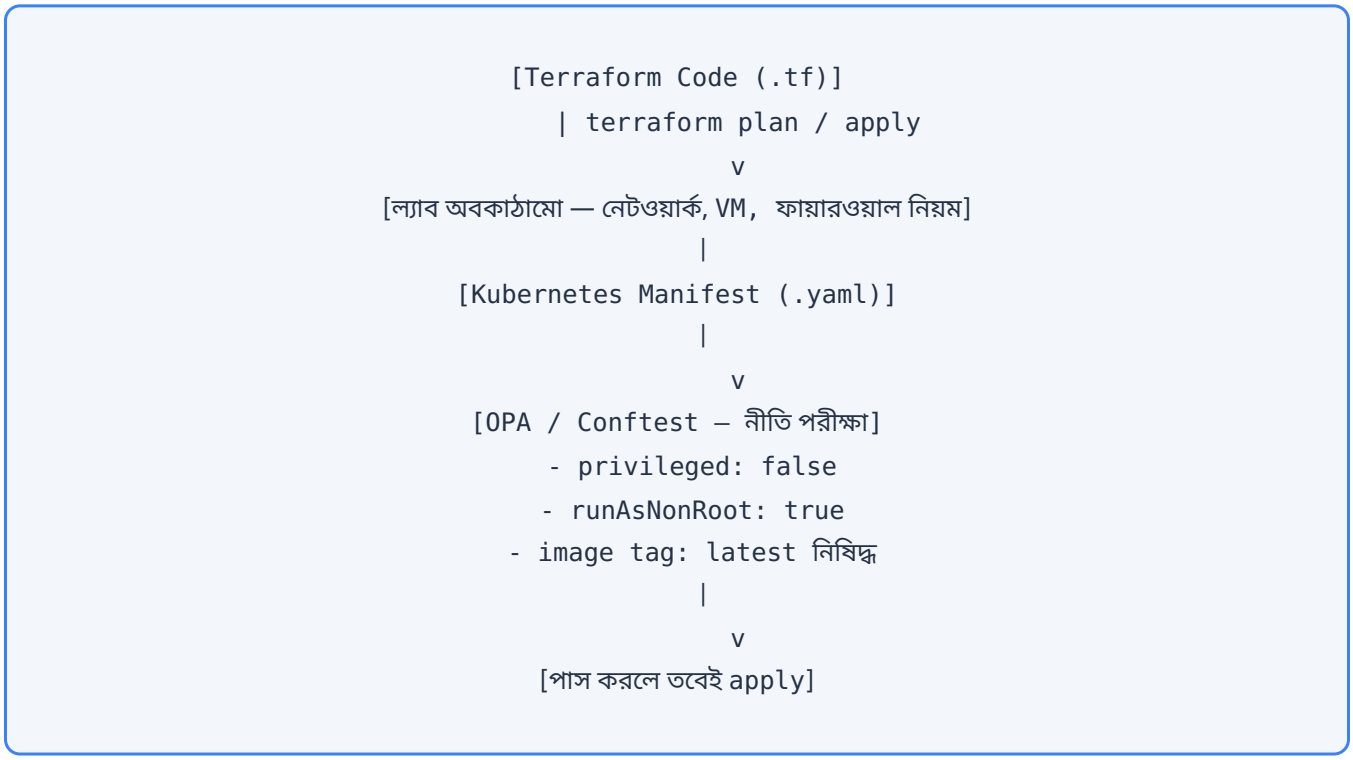
## ২. পূর্বশর্ত (Prerequisites)

- Terraform ইনস্টল করা; Lab 10 সম্পন্ন; YAML ও HCL-এর ধারণা।

## ৩. আনুমানিক সময়

৪ ঘণ্টা।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

Terraform শুধু ল্যাব ক্লাউড অ্যাকাউন্ট বা local provider-এ চালান। `terraform apply`-এর আগে `plan` পড়ুন। State ফাইলে সিক্রেট থাকতে পারে — তা Git-এ কমিট করবেন না।

## ৬. সেটআপ

```
terraform version
mkdir -p ~/technova-lab/lab13-policy
cd ~/technova-lab/lab13-policy
cat > main.tf <<'EOF'
terraform {
  required_version = ">= 1.5.0"
}
# ল্যাবের জন্য local/null রিসোর্স — আসল ক্লাউড নয়
resource "null_resource" "technova_lab" {
  provisioner "local-exec" {
    command = "echo 'TechNova lab infra — CHANGE-ME-PLACEHOLDER'"
  }
}
EOF
terraform init
terraform plan
```

## ৭. কমান্ড ও কনফিগারেশন

```
cd ~/technova-lab/lab13-policy
# ১. Terraform পরীক্ষা
terraform fmt
terraform validate
terraform plan -out=tfplan

# ২. OPA নীতি লিখুন (Rego)
mkdir -p policy
cat > policy/deny_privileged.rego <<'EOF'
package main
deny[msg] {
  input.kind == "Pod"
  input.spec.containers[_].securityContext.privileged == true
  msg := "Privileged পড নিষিদ্ধ"
}
deny[msg] {
  input.kind == "Pod"
  not input.spec.securityContext.runAsNonRoot
  msg := "runAsNonRoot বাধ্যতামূলক"
}
EOF

# ৩. Conftest দিয়ে যাচাই
# conftest ইনস্টল থাকলে:
# conftest test ../lab10/secure-pod.yaml -p policy/
# না থাকলে OPA দিয়ে:
# opa eval --data policy/ --input secure-pod.yaml "data.main.deny"
cat > secure-pod.yaml <<'EOF'
```

```
apiVersion: v1
kind: Pod
metadata:
  name: technova-app
spec:
  securityContext:
    runAsNonRoot: true
  containers:
  - name: app
    image: nginx:1.27-alpine
    securityContext:
      privileged: false
EOF
cat secure-pod.yaml
```

**CONCEPT:** OPA (Open Policy Agent) হলো নীতি পরীক্ষার ইঞ্জিন, Rego তার ভাষা। Conftest সেই ইঞ্জিনকে YAML/JSON ফাইলে চালানোর সহজ মোড়ক। একবার নীতি লিখলে তা CI, Kubernetes (Gatekeeper) ও Terraform — সবখানে পুনঃব্যবহার করা যায়।

## ৮. প্রত্যাশিত আউটপুট

`terraform plan`-এ পরিবর্তনের তালিকা; নিরাপদ পড নীতি পরীক্ষায় পাস; privileged পড নীতি পরীক্ষায় ব্যর্থ (deny বার্তাসহ)।

## ৯. যাচাইকরণ (Verification)

- privileged: true করা একটি পড কি নীতিতে আটকায়?
- `terraform validate` কি সফল?
- নীতি ফাইল কি সংস্করণ নিয়ন্ত্রণে (Git) আছে?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **Rego সিনট্যাক্স ভুল:** `opa fmt` ও `opa check` চালান।
- **Terraform provider ডাউনলোড ব্যর্থ:** নেটওয়ার্ক যাচাই করুন; ল্যাবে local/null provider ব্যবহার করুন।

## ১১. নিরাপত্তা সতর্কতা

Terraform state ফাইলে সিক্রেট থাকতে পারে — রিমোট backend-এ এনক্রিপশন ও অ্যাক্সেস নিয়ন্ত্রণ বাধ্যতামূলক। `terraform.tfstate` কখনো পাবলিক রেপোতে দেবেন না।

## ১২. ক্লিনআপ (Cleanup)

```
terraform destroy -auto-approve 2>/dev/null || true
rm -rf .terraform tfplan terraform.tfstate* 2>/dev/null; echo "cleaned"
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. নীতি কোডে লেখার সুবিধা কী — ম্যানুয়াল পর্যালোচনার তুলনায়?
2. কোন নীতিগুলো কঠোর (deny) আর কোনগুলো সতর্কতা (warn) হওয়া উচিত?
3. নীতি বদলালে পুরোনো চলমান workload-এর কী হবে?

ল্যাব ১৪

# Lab 14 — Cloud Security Posture Assessment

Cloud Security Posture (ক্লাউড নিরাপত্তা অবস্থা) মানে আপনার ক্লাউড অ্যাকাউন্টের কনফিগারেশন কতটা নিরাপদ — খোলা S3 বাকেট, অনুপস্থিত লগিং, অতিরিক্ত অনুমতি আছে কি না। এই ল্যাবে আপনি Prowler বা Scout Suite-ধাঁচের চেক দিয়ে ল্যাব অ্যাকাউন্ট মূল্যায়ন করবেন।

## ১. উদ্দেশ্য (Objectives)

- CSPM (Cloud Security Posture Management) চেক চালানো।
- খোলা স্টোরেজ, অনুপস্থিত এনক্রিপশন ও লগিং চিহ্নিত করা।
- ফলাফলকে CIS Benchmark-এর সঙ্গে মেলানো।
- প্রতিকার-পরিকল্পনা (remediation plan) লেখা।

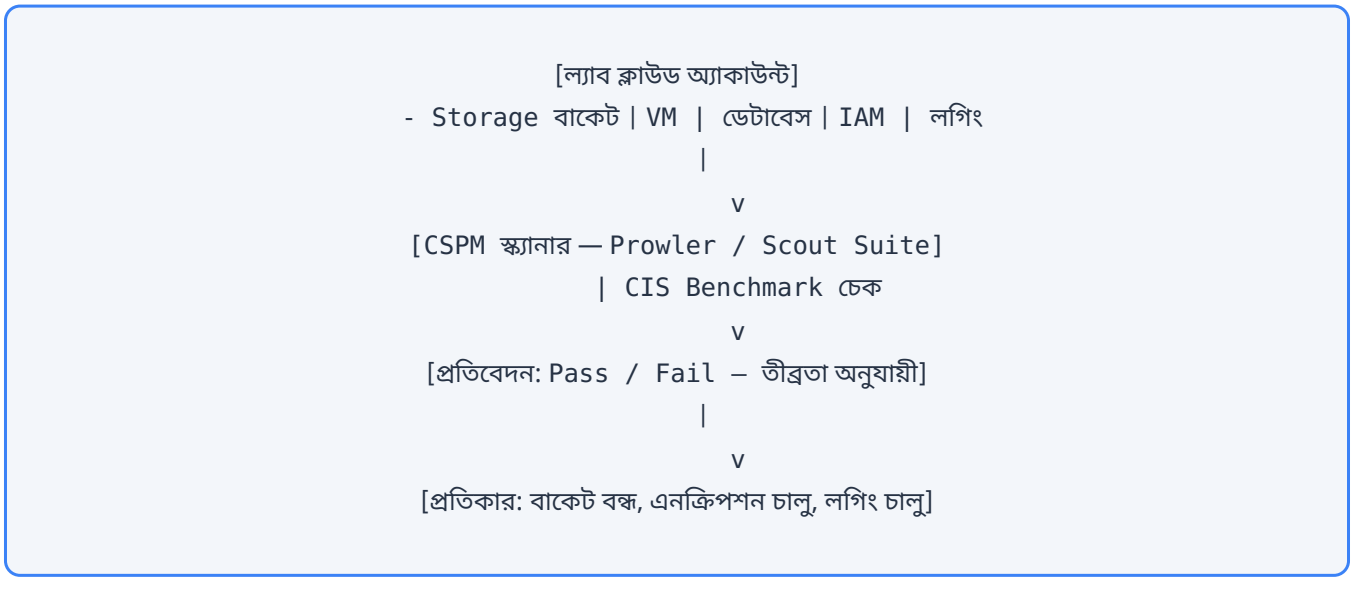
## ২. পূর্বশর্ত (Prerequisites)

- Lab 4 সম্পন্ন; ক্লাউড কনসোলের ধারণা; Python 3।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার




## ৫. নিরাপদ ল্যাব পরিবেশ

শুধু ল্যাব ক্লাউড অ্যাকাউন্ট স্ক্যান করুন। অন্যের অ্যাকাউন্ট বা প্রোডাকশন অ্যাকাউন্টে স্ক্যান চালাবেন না। স্ক্যানারকে শুধু read-only অনুমতি দিন।

## ৬. সেটআপ

```
pip install prowler 2>&1 | tail -3
mkdir -p ~/technova-lab/lab14-cspm/reports
# ল্যাব অ্যাকাউন্টের read-only ক্রেডেনশিয়াল পরিবেশ ভেরিয়েবলে দিন (আসল কী নয়):
export AWS_ACCESS_KEY_ID='CHANGE-ME-LAB-ACCESS-KEY'
export AWS_SECRET_ACCESS_KEY='CHANGE-ME-LAB-SECRET-KEY'
export AWS_DEFAULT_REGION='ap-south-1'
echo "ক্রেডেনশিয়াল শুধু ল্যাব অ্যাকাউন্টের — যাচাই করে নিন"
```

## ৭. কমান্ড ও কনফিগারেশন

```
# Prowler স্ক্যান (ল্যাব অ্যাকাউন্টে; ক্লাউড না থাকলে চেকলিস্ট দিয়ে ম্যানুয়াল মূল্যায়ন করুন)
prowler aws --output-directory ~/technova-lab/lab14-cspm/reports 2>&1 | tail -20
ls -lh ~/technova-lab/lab14-cspm/reports/

# ম্যানুয়াল চেকলিস্ট (ক্লাউড অ্যাকাউন্ট না থাকলে এটাই ডেলিভারেবল):
cat > ~/technova-lab/lab14-cspm/checklist.md <<'EOF'
| চেক | প্রত্যাশা | ফলাফল |
| S3 বাকেট পাবলিক নয় | Block Public Access চালু | |
| ডেটাবেস এনক্রিপশন | at-rest এনক্রিপশন চালু | |
| CloudTrail / লগিং | সব অঞ্চলে চালু | |
| Root অ্যাকাউন্টে MFA | চালু | |
| পুরোনো অ্যাক্সেস কী | 90 দিনের বেশি নয় | |
| খোলা সিকিউরিটি গ্রুপ | 0.0.0.0/0-তে 22 পোর্ট খোলা নয় | |
EOF
cat ~/technova-lab/lab14-cspm/checklist.md
```

## ৮. প্রত্যাশিত আউটপুট

HTML/JSON প্রতিবেদনে Pass/Fail চেকের তালিকা; অন্তত ৫টি ব্যর্থ চেকের জন্য প্রতিকার-পরিকল্পনা।

## ৯. যাচাইকরণ (Verification)

- সবচেয়ে গুরুতর (Critical) ব্যর্থতাগুলো কি চিহ্নিত হয়েছে?
- প্রতিটি ব্যর্থতার জন্য কি দায়িত্বপ্রাপ্ত ব্যক্তি ও সময়সীমা আছে?
- পুনঃস্ক্যানে ঠিক করা চেকগুলো কি Pass হয়?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **স্ক্যানার অনুমতি পাচ্ছে না:** read-only নীতি (SecurityAudit) সংযুক্ত আছে কি না দেখুন।
- **অনেক চেক প্রযোজ্য নয়:** যে সার্ভিস ব্যবহার করেন না, সেগুলো নোট করে বাদ দিন — সব Fail সমান গুরুত্বপূর্ণ নয়।

## ১১. নিরাপত্তা সতর্কতা

স্ক্যান রিপোর্টে ক্লাউডের দুর্বলতার পূর্ণ চিত্র থাকে — তা সংবেদনশীল। রিপোর্ট শেয়ার করবেন না, অ্যাক্সেস সীমিত রাখুন।

## ১২. ক্লিনআপ (Cleanup)

```
unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY
# ল্যাব ক্রেডেনশিয়াল ঘোরান (rotate) বা মুছে ফেলুন
ls ~/technova-lab/lab14-cspm/reports/
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. কোন ভুল কনফিগারেশন সবচেয়ে বেশি ডেটা ফাঁসের কারণ হয়?
2. CSPM স্ক্যান কত ঘন ঘন চালানো উচিত?
3. একটি চেক বারবার Fail হলে — নীতি বদলাবেন নাকি ব্যতিক্রম দেবেন?

ল্যাব ১৫

# Lab 15 — Vulnerability Management Workflow

দুর্বলতা খুঁজে পাওয়াই শেষ নয় — তা ঠিক করা, যাচাই করা ও নথিভুক্ত করার একটি ধারাবাহিক প্রক্রিয়া (workflow) দরকার। এই ল্যাবে আপনি একটি দুর্বলতা-ব্যবস্থাপনা (vulnerability management) জীবনচক্র তৈরি করবেন: আবিষ্কার → ত্রিয়াজ → প্রতিকার → যাচাই → বন্ধ।

## ১. উদ্দেশ্য (Objectives)

- দুর্বলতার জীবনচক্র (lifecycle) ডিজাইন করা।
- CVSS ও ব্যবসায়িক প্রভাব দিয়ে অগ্রাধিকার (SLA) নির্ধারণ করা।
- একটি ট্র্যাকিং বোর্ড (টেবিল/ইস্যু) তৈরি করা।
- প্যাচ যাচাই ও অবশিষ্ট ঝুঁকি নথিভুক্ত করা।

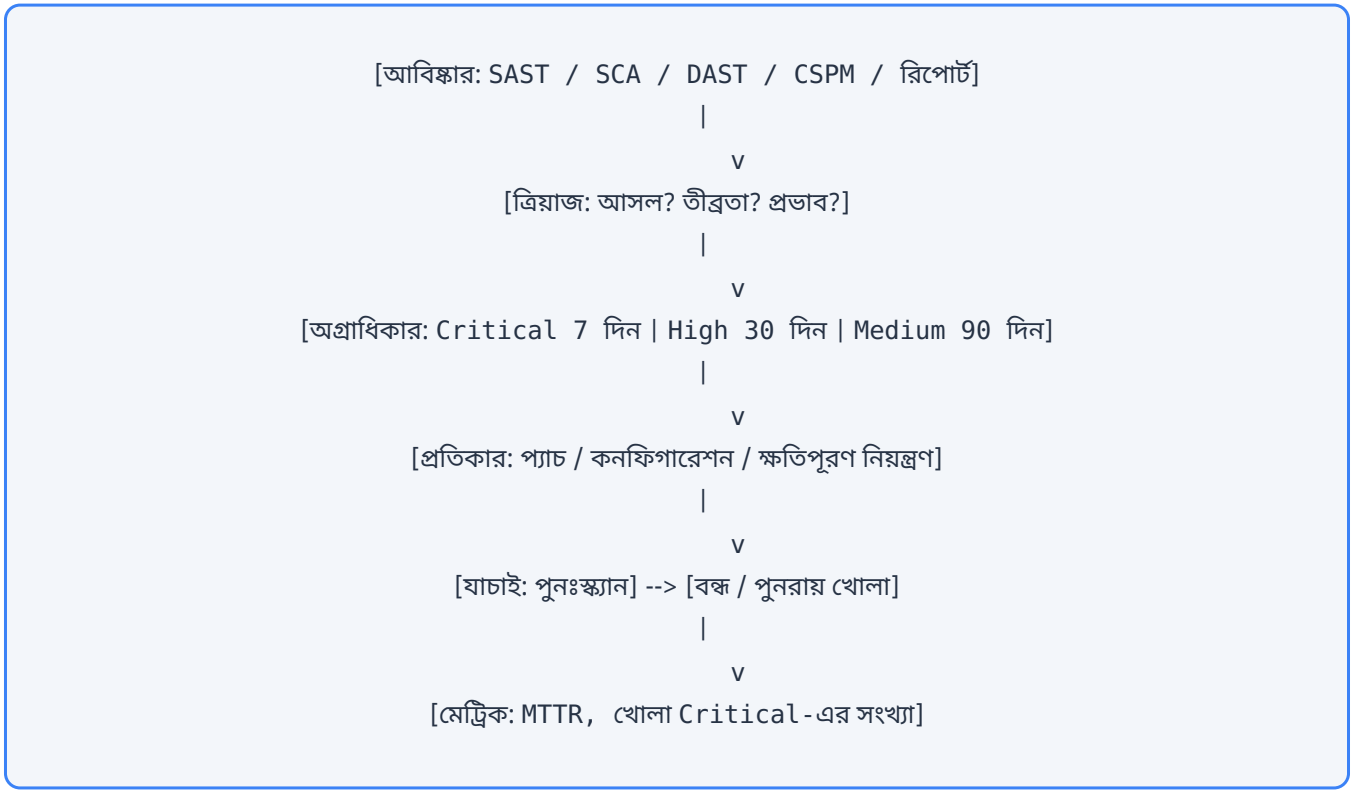
## ২. পূর্বশর্ত (Prerequisites)

- Lab 6, 8, 9, 12 সম্পন্ন; আগের ল্যাবের স্ক্যান রিপোর্ট।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

এই ল্যাব প্রক্রিয়া-কেন্দ্রিক; কোনো বাস্তব সিস্টেমে প্যাচ পরীক্ষা করবেন না। ল্যাবের নমুনা দুর্বলতা দিয়েই workflow চালান।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab15-vuln
cd ~/technova-lab/lab15-vuln
cat > vuln-register.md <<'EOF'
# দুর্বলতা রেজিস্টার (Vulnerability Register)
| ID | উৎস | CVE/নিয়ম | CVSS | সম্পদ | দায়িত্ব | SLA | অবস্থা |
| VULN-001 | SCA | lodash পুরোনো সংস্করণ | 7.4 | technova-app | Nahid | 30 দিন | খোলা |
| VULN-002 | SAST | SQL injection ধাঁচ | 8.1 | API /user | Nahid | 7 দিন | খোলা |
| VULN-003 | Image | বেস ইমেজ দুর্বলতা | 6.5 | technova-app:lab | Nahid | 30 দিন | খোলা |
EOF
cat vuln-register.md
```

## ৭. কমান্ড ও কনফিগারেশন

```
cat > sla-policy.md <<'EOF'
# প্রতিকার SLA (ল্যাব নীতি)
| তীব্রতা | CVSS | প্রতিকারের সময়সীমা |
| Critical | 9.0–10 | 7 দিন |
| High | 7.0–8.9 | 30 দিন |
| Medium | 4.0–6.9 | 90 দিন |
| Low | 0.1–3.9 | পরবর্তী রিলিজে |
# ব্যতিক্রম: লিখিত অনুমোদন + ক্ষতিপূরণ নিয়ন্ত্রণ (compensating control) + মেয়াদ
EOF
cat sla-policy.md

# একটি দুর্বলতা বন্ধ করার পরীক্ষা — পুনঃস্ক্যানের প্রমাণ সংযুক্ত করুন
# উদাহরণ: npm audit fix-এর পর npm audit আউটপুট সংরক্ষণ
echo "প্রমাণ: পুনঃস্ক্যানের আউটপুট VULN-001-এর পাশে সংযুক্ত করুন"
```

| তীব্রতা | CVSS | SLA | উদাহরণ ব্যবস্থা |
|---|---|---|---|
| Critical | ৯.০–১০ | ৭ দিন | তাৎক্ষণিক প্যাচ, প্রয়োজনে সার্ভিস বন্ধ |
| High | ৭.০–৮.৯ | ৩০ দিন | পরিকল্পিত প্যাচ, ক্ষতিপূরণ নিয়ন্ত্রণ |
| Medium | ৪.০–৬.৯ | ৯০ দিন | পরবর্তী রিলিজে প্যাচ |
| Low | ০.১–৩.৯ | পরবর্তী রিলিজ | নজরদারি, নথিভুক্তকরণ |

## ৮. প্রত্যাশিত আউটপুট

একটি সম্পূর্ণ রেজিস্টার, SLA নীতি, এবং অন্তত একটি দুর্বলতার “আবিষ্কার থেকে বন্ধ” পর্যন্ত পূর্ণ চক্রের প্রমাণ।

## ৯. যাচাইকরণ (Verification)

- প্রতিটি খোলা Critical দুর্বলতার কি SLA ও দায়িত্বপ্রাপ্ত ব্যক্তি আছে?
- বন্ধ করা দুর্বলতার কি পুনঃস্ক্যানের প্রমাণ আছে?
- MTTR (গড় প্রতিকার সময়) কি গণনা করা যায়?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **দুর্বলতা বারবার ফিরে আসছে:** মূল কারণ (root cause) খুঁজুন — বেস ইমেজ পুরোনো, নাকি dependency pin করা হয়নি?
- **SLA মিস হচ্ছে:** অগ্রাধিকার পুনর্বিবেচনা করুন; ক্ষতিপূরণ নিয়ন্ত্রণ (যেমন WAF নিয়ম) দিয়ে সময় কিনুন।

> **REAL WORLD:** বহু প্রতিষ্ঠানে দুর্বলতা "খুঁজে পাওয়া" হয়, কিন্তু "বন্ধ করা" হয় না — কারণ দায়িত্ব ও সময়সীমা লেখা থাকে না। রেজিস্টার + SLA + পুনঃস্ক্যান — এই তিনটি থাকলেই workflow সম্পূর্ণ হয়।

## ১১. নিরাপত্তা সতর্কতা

দুর্বলতার রেজিস্টার নিজেই সংবেদনশীল — এতে আক্রমণকারীর জন্য কাজের তালিকা থাকে। অ্যাক্সেস সীমিত রাখুন, শেয়ার করার আগে পর্যালোচনা করুন।

## ১২. ক্লিনআপ (Cleanup)

```
ls -l ~/technova-lab/lab15-vuln/
# রেজিস্টার ও নীতি সংরক্ষণ করুন — এগুলোই ডেলিভারেবল; কোনো সার্ভিস চালু করা হয়নি
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. কোন দুর্বলতা "স্বীকার" (accept) করা যায়, আর কোনটি নয়?
2. MTTR কমাতে সবচেয়ে কার্যকর একটি পরিবর্তন কী?
3. দুর্বলতা ব্যবস্থাপনাকে কীভাবে DevSecOps পাইপলাইনের সঙ্গে যুক্ত করবেন?

ল্যাব ১৬

# Lab 16 — MERN Application SLI and SLO Design

SRE (Site Reliability Engineering) শুরু হয় পরিমাপ দিয়ে। SLI (Service Level Indicator) হলো সেবার মানের পরিমাপ — যেমন সফল অনুরোধের হার; SLO (Service Level Objective) হলো সেই পরিমাপের লক্ষ্য — যেমন ৯৯.৯% সফল অনুরোধ। এই ল্যাবে আপনি TechNova MERN অ্যাপের জন্য SLI/SLO ও error budget (ত্রুটি বাজেট) ডিজাইন করবেন।

## ১. উদ্দেশ্য (Objectives)

- ব্যবহারকারীর যাত্রা (user journey) থেকে SLI বেছে নেওয়া।
- SLO লক্ষ্য নির্ধারণ ও error budget গণনা করা।
- ৯৯.৯% ও ৯৯.৯৯% SLO-র অনুমোদিত downtime (বন্ধ থাকার সময়) গণনা করা।
- Burn rate (বাজেট খরচের হার) ও সতর্কতার নীতি লেখা।

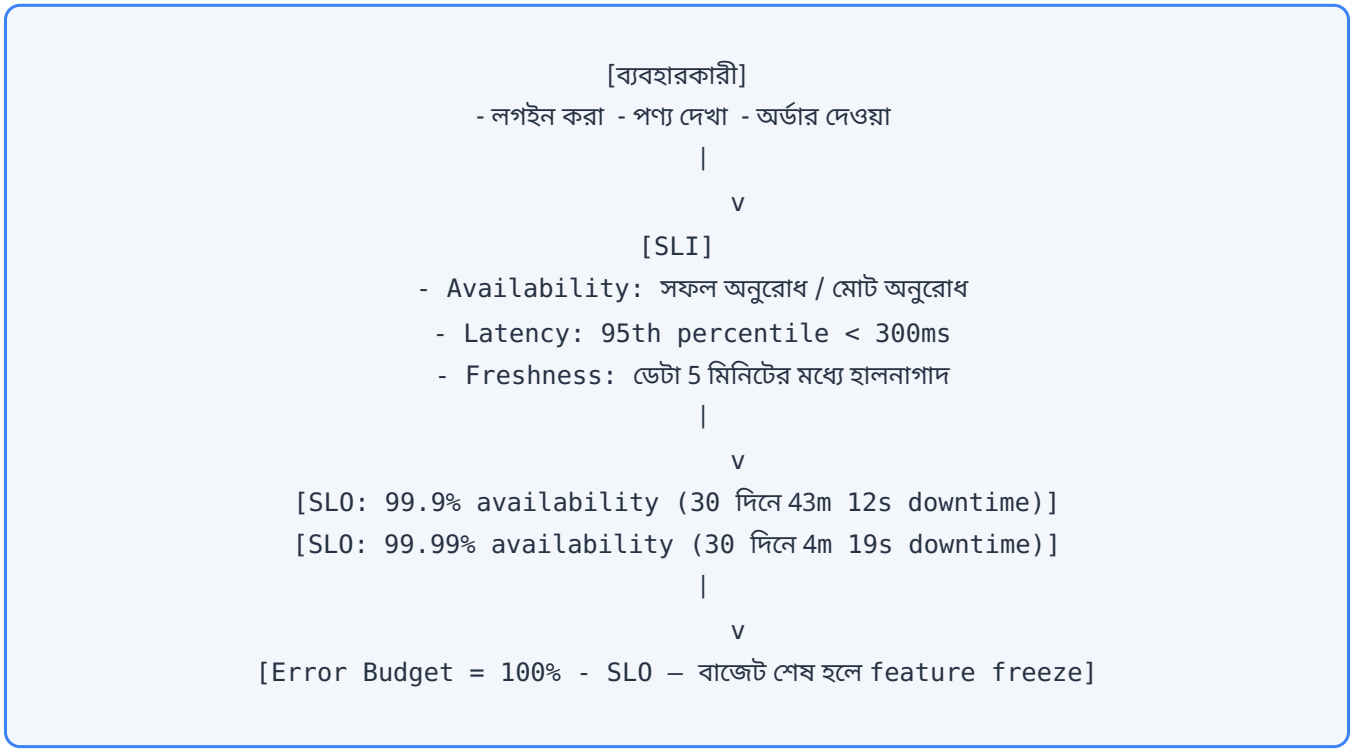
## ২. পূর্বশর্ত (Prerequisites)

- SRE ধারণার প্রাথমিক পরিচয়; Lab 1-এর আর্কিটেকচার।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

এই ল্যাব নকশা-কেন্দ্রিক; কোনো প্রোডাকশন সিস্টেমে পরিবর্তন নেই। গণনার জন্য ক্যালকুলেটর ও স্প্রেডশিট ব্যবহার করুন।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab16-slo
cd ~/technova-lab/lab16-slo
cat > slo-design.md <<'EOF'
# TechNova SLO নকশা
## ব্যবহারকারীর যাত্রা
1. হোমপেজ লোড
2. লগইন
3. পণ্য ব্রাউজ
4. চেকআউট / অর্ডার
## SLI
- Availability: HTTP 2xx / মোট অনুরোধ (5xx বাদে)
- Latency: p95 < 300ms (API)
- Durability: ডেটা হারানো 0
EOF
cat slo-design.md
```

## ৭. কমান্ড ও কনফিগারেশন

```
# SLO গণনা — 30 দিনের মাস = 30 * 24 * 60 = 43200 মিনিট
python3 -c "
total = 30*24*60
for slo in [99.9, 99.95, 99.99]:
    allowed = total * (100 - slo) / 100
    print(f'SLO {slo}% -> অনুমোদিত downtime: {allowed:.2f} মিনিট')
"
# প্রত্যাশিত আউটপুট:
# SLO 99.9%  -> 43.20 মিনিট (43m 12s)
# SLO 99.95% -> 21.60 মিনিট
# SLO 99.99% -> 4.32 মিনিট (4m 19s)
```

**CONCEPT:** ৯৯.৯৯% SLO-তে ৩০ দিনের মাসে অনুমোদিত downtime ৪ মিনিট ১৯ সেকেন্ড (৪.৩২ মিনিট)। অনেক জায়গায় ভুল করে ৪ মিনিট ২৩ সেকেন্ড লেখা হয় — সঠিক গণনা: ৪৩২০০ × ০.০০০১ = ৪.৩২ মিনিট = ৪ মিনিট ১৯ সেকেন্ড।

| SLO | ৩০ দিনে অনুমোদিত downtime | Error budget |
|---|---|---|
| ৯৯.৯% | ৪৩ মিনিট ১২ সেকেন্ড | ০.১% |
| ৯৯.৯৫% | ২১ মিনিট ৩৬ সেকেন্ড | ০.০৫% |
| ৯৯.৯৯% | ৪ মিনিট ১৯ সেকেন্ড | ০.০১% |

## ৮. প্রত্যাশিত আউটপুট

৩–৪টি SLI-র সংজ্ঞা, প্রতিটির SLO লক্ষ্য, error budget গণনা, এবং burn-rate সতর্কতার নীতি (যেমন ১৪.৪× burn হলে পেজ, ৬× হলে টিকিট)।

## ৯. যাচাইকরণ (Verification)

- প্রতিটি SLI কি ব্যবহারকারীর অনুভূত মানের সঙ্গে সরাসরি যুক্ত?
- গণনা কি উপরের টেবিলের সঙ্গে মেলে? ৯৯.৯৯% = ৪ মিনিট ১৯ সেকেন্ড কি না যাচাই করুন।
- Error budget শেষ হলে কী হবে — তা কি লেখা আছে?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **SLO খুব কঠিন মনে হচ্ছে:** বর্তমান পরিমাপ দিয়ে শুরু করুন; প্রথমে ৯৯.৯% লক্ষ্য নিন, পরে বাড়ান।
- **কোন SLI বেছে নেব বুঝতে পারছি না:** ব্যবহারকারী সবচেয়ে বেশি অভিযোগ করে এমন যাত্রা দিয়ে শুরু করুন।

## ১১. নিরাপত্তা সতর্কতা

SLO ড্যাশবোর্ডে ব্যবহারকারীর ব্যক্তিগত ডেটা দেখাবেন না — শুধু সমষ্টিগত (aggregated) মেট্রিক ব্যবহার করুন।

## ১২. ক্লিনআপ (Cleanup)

```
ls -l ~/technova-lab/lab16-slo/
# নকশা ফাইল সংরক্ষণ করুন; কোনো সার্ভিস চালু করা হয়নি
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. ১০০% SLO কেন অবাস্তব লক্ষ্য?
2. Error budget শেষ হলে feature কাজ বন্ধ করা কি ন্যায্য? কেন?
3. Availability বনাম latency — কোন SLI ব্যবহারকারী বেশি অনুভব করে?

ল্যাব ১৭

# Lab 17 — Prometheus and Grafana SLO Dashboard

Prometheus মেট্রিক সংগ্রহ করে, Grafana তা সুন্দর ড্যাশবোর্ডে দেখায়। এই ল্যাবে আপনি Docker Compose দিয়ে Prometheus + Grafana + একটি নমুনা Node.js অ্যাপ চালিয়ে SLO ড্যাশবোর্ড তৈরি করবেন — availability, latency ও error budget দৃশ্যমান করবেন।

## ১. উদ্দেশ্য (Objectives)

- Prometheus + Grafana Docker Compose-এ চালানো।
- Node.js অ্যাপে /metrics endpoint যোগ করা (prom-client)।
- PromQL দিয়ে SLI কুয়েরি লেখা।
- Grafana-তে SLO ও error budget প্যানেল তৈরি করা।

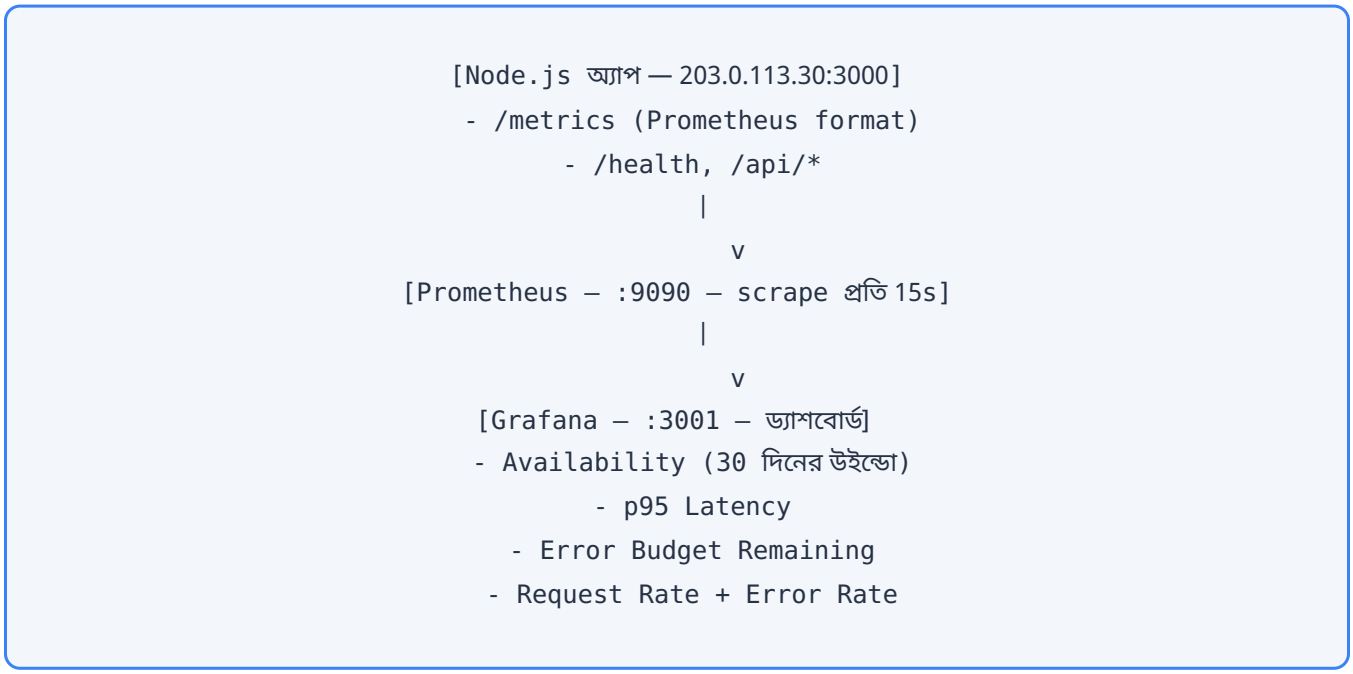
## ২. পূর্বশর্ত (Prerequisites)

- Docker Compose; Lab 16 সম্পন্ন; Node.js।

## ৩. আনুমানিক সময়

৪ ঘণ্টা।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

শুধু লোকাল Docker নেটওয়ার্কে চালান। Grafana-র ডিফল্ট পাসওয়ার্ড (admin/admin) বদলে CHANGE-ME-GRAFANA-PASSWORD ব্যবহার করুন। /metrics endpoint-এ সংবেদনশীল ডেটা রাখবেন না।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab17-observability
cd ~/technova-lab/lab17-observability
cat > docker-compose.yml <<'EOF'
services:
  app:
    build: ./app
    ports:
      - "3000:3000"
  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
  grafana:
    image: grafana/grafana:latest
    ports:
      - "3001:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=CHANGE-ME-GRAFANA-PASSWORD
EOF
cat > prometheus.yml <<'EOF'
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'technova-app'
    static_configs:
      - targets: ['app:3000']
EOF
cat docker-compose.yml
```

## ৭. কমান্ড ও কনফিগারেশন

```
mkdir -p app
cat > app/package.json <<'EOF'
{ "name": "technova-metrics-app", "version": "1.0.0",
  "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.19.2", "prom-client": "^15.1.0" } }
EOF
cat > app/server.js <<'EOF'
const express = require('express');
const client = require('prom-client');
const app = express();
client.collectDefaultMetrics();
const httpRequests = new client.Counter({
  name: 'http_requests_total',
  help: 'Total HTTP requests',
```

```
  labelNames: ['method', 'route', 'status']
});
app.use((req, res, next) => {
  res.on('finish', () => httpRequests.inc({ method: req.method, route: req.path, status:
res.statusCode }));
  next();
});
app.get('/health', (req, res) => res.json({ status: 'ok' }));
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', client.register.contentType);
  res.end(await client.register.metrics());
});
app.listen(3000);
EOF
cat > app/Dockerfile <<'EOF'
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
EOF
docker compose up -d --build
sleep 15
curl -s http://localhost:3000/health
curl -s http://localhost:9090/-/healthy
```

## ৮. প্রত্যাশিত আউটপুট

অ্যাপ `/health`-এ ok দেবে; Prometheus-এ `technova-app` টার্গেট Up দেখাবে; Grafana http://localhost:3001 -এ খুলবে। PromQL উদাহরণ:

```
# সফল অনুরোধের হার (availability SLI)
sum(rate(http_requests_total{status=~"2.."}[5m]))
  / sum(rate(http_requests_total[5m]))

# p95 latency (histogram থাকলে)
# histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

## ৯. যাচাইকরণ (Verification)

```
curl -s http://localhost:3000/metrics | head -20
curl -s "http://localhost:9090/api/v1/query?query=up" | head -c 500
docker compose ps
```

- Grafana-তে Prometheus ডেটা সোর্স যোগ করে একটি প্যানেলে উপরের কুয়েরি চালান — গ্রাফ দেখা যাচ্ছে কি?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **Prometheus টার্গেট Down:** `prometheus.yml`-এ হোস্টনাম `app:3000` ঠিক আছে কি না দেখুন; `docker compose logs app` দেখুন।

- **Grafana-তে ডেটা নেই:** ডেটা সোর্স URL `http://prometheus:9090` (কনটেইনার নাম) ব্যবহার করুন, localhost নয়।

## ১১. নিরাপত্তা সতর্কতা

Grafana ও Prometheus ল্যাব নেটওয়ার্কের বাইরে খুলবেন না। /metrics-এ ব্যক্তিগত ডেটা বা লেবেলে ব্যবহারকারী-আইডি দেবেন না (cardinality বিস্ফোরণ ও গোপনীয়তা ঝুঁকি)।

## ১২. ক্লিনআপ (Cleanup)

```
docker compose down -v
docker compose ps
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. কোন মেট্রিকটি SLO-র জন্য সবচেয়ে উপযোগী — counter, gauge, নাকি histogram?
2. ড্যাশবোর্ডে অনেক প্যানেল থাকলে কী সমস্যা হয়?
3. Prometheus ডেটা কতদিন রাখা উচিত? কেন?

ল্যাব ১৮

# Lab 18 — Alerting and On-Call Simulation

অ্যালার্ট (Alert) মানে সমস্যা হলে সঠিক মানুষকে সঠিক সময়ে জানানো। On-call (অন-কল) মানে নির্দিষ্ট সময়ে একজন প্রকৌশলী সমস্যার প্রথম প্রতিক্রিয়াকারী। এই ল্যাবে আপনি Alertmanager দিয়ে সতর্কতা নিয়ম লিখবেন, নীরব সময় (silence) ও রাউটিং শিখবেন এবং একটি অন-কল অনুশীলন (simulation) চালাবেন।

## ১. উদ্দেশ্য (Objectives)

- Prometheus alerting নিয়ম (alerting rule) লেখা।
- Alertmanager-এ রাউটিং, গ্রুপিং ও নীরবতা (silence) কনফিগার করা।
- Burn-rate ভিত্তিক অ্যালার্ট (multi-window) বোঝা।
- একটি অন-কল দৃশ্য অনুশীলন করে প্রতিক্রিয়া সময় মাপা।

## ২. পূর্বশর্ত (Prerequisites)

- Lab 16 ও Lab 17 সম্পন্ন; Prometheus/Alertmanager ধারণা।

## ৩. আনুমানিক সময়

৩ ঘণ্টা ৩০ মিনিট।

## ৪. আর্কিটেকচার

```
[Prometheus — নিয়ম মূল্যায়ন প্রতি 15s]
                  |
                  v
        [Alertmanager — :9093]
   - গ্রুপিং (group_by: alertname)
- রাউটিং: critical -> PagerDuty/ফোন, warning -> Slack/ইমেইল
   - নীরবতা: রক্ষণাবেক্ষণের সময়
                  |
                  v
[On-call প্রকৌশলী — স্বীকৃতি (acknowledge) + প্রতিক্রিয়া]
   - MTTA: স্বীকৃতির সময়
   - MTTR: সমাধানের সময়
```

## ৫. নিরাপদ ল্যাব পরিবেশ

এই ল্যাবে কোনো বাস্তব পেজার বা ফোন নম্বর ব্যবহার করবেন না — webhook-এর বদলে লোকাল লগ/ইমেইল অনুকরণ ব্যবহার করুন। অনুশীলনের অ্যালার্ট যেন বাস্তব দলকে বিরক্ত না করে।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab18-alerting
cd ~/technova-lab/lab18-alerting
cat > alerts.yml <<'EOF'
groups:
  - name: technova-slo
    rules:
      - alert: HighErrorRate
        expr: |
          sum(rate(http_requests_total{status=~"5.."}[5m]))
          / sum(rate(http_requests_total[5m])) > 0.05
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "TechNova-তে 5xx হার 5%-এর বেশি"
          runbook: "https://technova.example/runbooks/high-error-rate"
      - alert: HighLatency
        expr: |
          histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by
(le)) > 0.5
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "p95 latency 500ms-এর বেশি"
EOF
cat alerts.yml
```

## ৭. কমান্ড ও কনফিগারেশন

```
cat > alertmanager.yml <<'EOF'
global:
  resolve_timeout: 5m
route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: 'lab-log'
  routes:
    - match:
        severity: critical
      receiver: 'lab-log-critical'
receivers:
  - name: 'lab-log'
    webhook_configs:
      - url: 'http://127.0.0.1:5001/webhook'
  - name: 'lab-log-critical'
    webhook_configs:
      - url: 'http://127.0.0.1:5001/webhook-critical'
EOF
cat alertmanager.yml

# Prometheus কনফিগে নিয়ম ও Alertmanager যোগ করার নমুনা:
```

```
cat > prometheus-alerting-snippet.yml <<'EOF'
rule_files:
  - alerts.yml
alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']
EOF
cat prometheus-alerting-snippet.yml
# নীতি: critical অ্যালার্টে 15 মিনিটের মধ্যে স্বীকৃতি, warning-এ 1 ঘণ্টা
```

> **CONCEPT:** Alert fatigue (সতর্কতা-ক্লান্তি) মানে এত বেশি অ্যালার্ট আসা যে দল সেগুলো উপেক্ষা করতে শুরু করে। প্রতিটি অ্যালার্ট অবশ্যই actionable (করণীয়) হতে হবে — "কী করব" না জানলে সেটি অ্যালার্ট নয়, টিকিট।

## ৮. প্রত্যাশিত আউটপুট

ইচ্ছাকৃতভাবে 5xx হার বাড়ালে HighErrorRate অ্যালার্ট Firing অবস্থায় দেখা যাবে; Alertmanager-এ গ্রুপিং ও রাউটিং দেখা যাবে; অন-কল অনুশীলনের লগে MTTA/MTTR লেখা হবে।

## ৯. যাচাইকরণ (Verification)

- Prometheus UI-তে Alerts পেজে নিয়মটি দেখা যাচ্ছে কি?
- Alertmanager UI-তে (http://localhost:9093) অ্যালার্টটি রাউট হচ্ছে কি?
- অন-কল অনুশীলনে স্বীকৃতির সময় কি ১৫ মিনিটের মধ্যে ছিল?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **অ্যালার্ট firing হচ্ছে না:** `for: 5m` সময় পার হয়েছে কি না দেখুন; PromQL কুয়েরি Prometheus-এ সরাসরি চালিয়ে মান যাচাই করুন।
- **বারবার একই অ্যালার্ট:** `repeat_interval` ও গ্রুপিং যাচাই করুন; সাময়িক silence ব্যবহার করুন।

## ১১. নিরাপত্তা সতর্কতা

অ্যালার্ট বার্তায় সিক্রেট, টোকেন বা ব্যক্তিগত ডেটা পাঠাবেন না। Webhook URL-এ টোকেন থাকলে তা Vault-এ রাখুন (Lab 5)।

## ১২. ক্লিনআপ (Cleanup)

```
# অনুশীলনের silence মুছে ফেলুন; পরীক্ষামূলক অ্যালার্ট বন্ধ করুন
# Alertmanager বন্ধ করলে:
docker stop alertmanager 2>/dev/null || true
ls ~/technova-lab/lab18-alerting/
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. কোন অ্যালার্টটি পেজ (জাগানো) করার যোগ্য, আর কোনটি শুধু টিকিট?
2. On-call-এ বার্নআউট এড়াতে রোটেশন কেমন হওয়া উচিত?
3. একটি অ্যালার্ট বারবার false positive হলে কী করবেন?

ল্যাব ১৯

# Lab 19 — Incident Response and RCA

Incident (ঘটনা) মানে সেবা ব্যাহত হওয়া — যেমন সাইট ডাউন বা চেকআউট ব্যর্থ। এই ল্যাবে আপনি একটি অনুকরণীয় ঘটনার প্রতিক্রিয়া (incident response) অনুশীলন করবেন: ভূমিকা নির্ধারণ, যোগাযোগ, প্রশমন (mitigation) এবং পরে RCA (Root Cause Analysis — মূল কারণ বিশ্লেষণ) ও blameless postmortem (দোষারোপহীন পর্যালোচনা) লিখবেন।

## ১. উদ্দেশ্য (Objectives)

- ঘটনার তীব্রতা (severity) নির্ধারণ ও ভূমিকা (Incident Commander, Comms, Ops) ভাগ করা।
- সময়রেখা (timeline) তৈরি করা।
- Five Whys দিয়ে মূল কারণ খুঁজে বের করা।
- Blameless postmortem লিখে প্রতিকারমূলক কাজ (action item) নির্ধারণ করা।

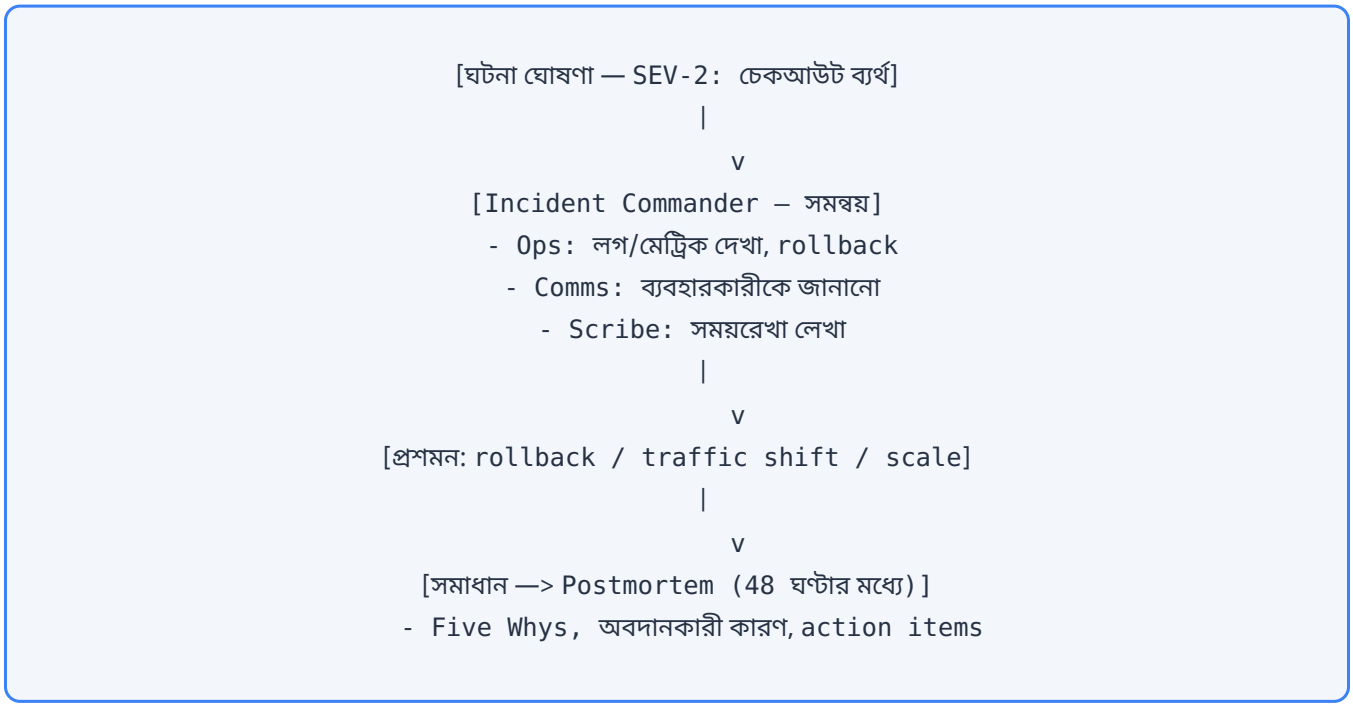
## ২. পূর্বশর্ত (Prerequisites)

- Lab 17 ও Lab 18 সম্পন্ন; একটি চলমান ল্যাব অ্যাপ।

## ৩. আনুমানিক সময়

৪ ঘণ্টা (অনুশীলন ২ ঘণ্টা + postmortem ২ ঘণ্টা)।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

ঘটনা অনুশীলন শুধু ল্যাব অ্যাপে — প্রোডাকশন সিস্টেমে ইচ্ছাকৃত বিভ্রাট তৈরি করবেন না। অনুশীলনের আগে সবাইকে জানান যে এটি একটি drill (মহড়া)।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab19-incident
cd ~/technova-lab/lab19-incident
cat > incident-template.md <<'EOF'
# Incident — INC-2026-11-09-001 (ল্যাব অনুশীলন)
- Severity: SEV-2
- শুরু: 10:00 +06
- Incident Commander:
- Comms:
- Ops:
- Scribe:
## সময়রেখা
| সময় | ঘটনা |
| 10:00 | অ্যালার্ট: HighErrorRate firing |
| 10:05 | ঘটনা ঘোষণা |
## প্রভাব
- ব্যবহারকারী: চেকআউট ব্যর্থ
- সময়কাল:
## প্রশমন
## মূল কারণ (Five Whys)
## Action Items
| কাজ | দায়িত্ব | সময়সীমা |
EOF
cat incident-template.md
```

## ৭. কমান্ড ও কনফিগারেশন

```
# ঘটনার সময় লগ ও মেট্রিক সংগ্রহ (ল্যাব অ্যাপে)
docker compose logs --tail 100 app 2>/dev/null || docker logs technova-lab-app --tail 100 2>/
dev/null || echo "ল্যাব অ্যাপের লগ দেখুন"
curl -s http://localhost:9090/api/v1/query?query=up 2>/dev/null | head -c 300
echo ""
# Five Whys উদাহরণ:
cat > five-whys.md <<'EOF'
সমস্যা: চেকআউট ব্যর্থ
কেন ১: পেমেন্ট API 500 দিচ্ছে
কেন ২: ডেটাবেস সংযোগ শেষ (connection pool exhausted)
কেন ৩: সংযোগ বন্ধ করা হচ্ছিল না (leak)
কেন ৪: কোড রিভিউতে ধরা পড়েনি
কেন ৫: সংযোগ-লিকের জন্য কোনো টেস্ট/অ্যালার্ট ছিল না
মূল কারণ: সংযোগ ব্যবস্থাপনায় টেস্ট ও মনিটরিংয়ের ঘাটতি
EOF
cat five-whys.md
```

> **REAL WORLD:** Blameless postmortem মানে "কে ভুল করেছে" নয়, "সিস্টেম কীভাবে ভুল হতে দিল" তা খোঁজা। দোষারোপ করলে মানুষ ভুল লুকায়; দোষারোপ না করলে মানুষ সত্য বলে — আর সত্য না জানলে সিস্টেম ঠিক হয় না।

## ৮. প্রত্যাশিত আউটপুট

একটি সম্পূর্ণ incident টাইমলাইন, Five Whys বিশ্লেষণ, এবং অন্তত ৩টি action item সহ blameless postmortem।

## ৯. যাচাইকরণ (Verification)

- Postmortem কি ৪৮ ঘণ্টার মধ্যে লেখা হয়েছে?
- প্রতিটি action item-এর কি দায়িত্বপ্রাপ্ত ব্যক্তি ও সময়সীমা আছে?
- একই ঘটনা আবার ঘটলে কি অ্যালার্ট আগেই ধরবে?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **টাইমলাইন অসম্পূর্ণ:** লগের সময় (timestamp) ও অ্যালার্টের সময় মিলিয়ে পুনর্গঠন করুন।
- **মূল কারণে পৌঁছাতে পারছি না:** "কেন" জিজ্ঞাসা চালিয়ে যান যতক্ষণ না এমন একটি কারণ পান যা ঠিক করলে পুনরাবৃত্তি থামবে।

## ১১. নিরাপত্তা সতর্কতা

Incident-এর সময় সিক্রেট বা ব্যক্তিগত ডেটা চ্যাটে শেয়ার করবেন না। Postmortem-এ ব্যবহারকারীর ব্যক্তিগত তথ্য রাখবেন না।

## ১২. ক্লিনআপ (Cleanup)

```
# ল্যাব অ্যাপ স্বাভাবিক অবস্থায় ফিরিয়ে আনুন (rollback/restart)
docker compose up -d 2>/dev/null || docker start technova-lab-app 2>/dev/null || echo "অ্যাপ স্বাভাবিক আছে কি না যাচাই করুন"
ls ~/technova-lab/lab19-incident/
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. Blameless সংস্কৃতি ছাড়া postmortem কেন ব্যর্থ হয়?
2. কখন ঘটনাকে SEV-1 ঘোষণা করা উচিত?
3. একটি ভালো action item-এর বৈশিষ্ট্য কী?

ল্যাব ২০

# Lab 20 — Load Testing and Capacity Planning

Load testing (ভার পরীক্ষা) মানে অনেক ব্যবহারকারী একসঙ্গে এলে অ্যাপ কেমন আচরণ করে তা মাপা। Capacity planning (ধারণক্ষমতা পরিকল্পনা) মানে সেই পরিমাপ থেকে সিদ্ধান্ত নেওয়া — কত CPU, মেমরি, ইনস্ট্যান্স দরকার। এই ল্যাবে আপনি k6 দিয়ে ল্যাব অ্যাপে ভার দেবেন ও ফলাফল বিশ্লেষণ করবেন।

## ১. উদ্দেশ্য (Objectives)

- k6 দিয়ে লোড টেস্ট স্ক্রিপ্ট লেখা।
- Throughput, latency (p95) ও error rate মাপা।
- Breaking point (ভাঙনের সীমা) খুঁজে বের করা।
- Capacity পরিকল্পনা ও স্কেলিং সুপারিশ লেখা।

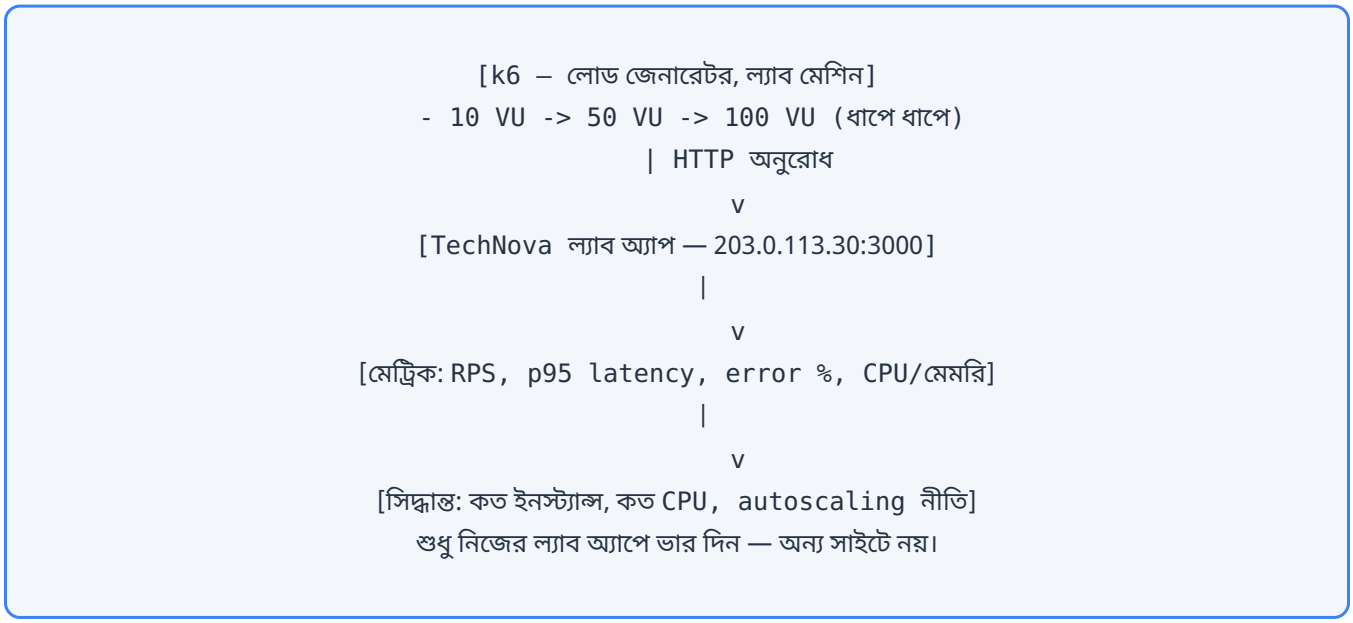
## ২. পূর্বশর্ত (Prerequisites)

- Docker; Lab 17 সম্পন্ন; একটি চলমান ল্যাব অ্যাপ।

## ৩. আনুমানিক সময়

৩ ঘণ্টা ৩০ মিনিট।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

> **WARNING:** লোড টেস্ট শুধু নিজের ল্যাব অ্যাপে চালান। অন্য কারো সাইটে লোড টেস্ট চালানো অননুমোদিত আক্রমণ (DDoS-এর মতো) হিসেবে গণ্য হতে পারে। ল্যাবেও ভার ধীরে ধীরে বাড়ান।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab20-load
cd ~/technova-lab/lab20-load
cat > load-test.js <<'EOF'
import http from 'k6/http';
import { check, sleep } from 'k6';
export const options = {
  stages: [
    { duration: '1m', target: 10 },
    { duration: '2m', target: 50 },
    { duration: '1m', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'],
    http_req_failed: ['rate<0.05'],
  },
};
export default function () {
  const res = http.get('http://localhost:3000/health');
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
EOF
cat load-test.js
```

## ৭. কমান্ড ও কনফিগারেশন

```
cd ~/technova-lab/lab20-load
# অ্যাপ চলছে কি না যাচাই
curl -s http://localhost:3000/health || echo "প্রথমে ল্যাব অ্যাপ চালু করুন (Lab 17)"

# k6 চালান (Docker দিয়ে)
docker run --rm --network host -v "$PWD":/scripts \
  grafana/k6:latest run /scripts/load-test.js 2>&1 | tee load-result.txt

tail -40 load-result.txt
```

## ৮. প্রত্যাশিত আউটপুট

k6 সারাংশে http_req_duration (p95), http_req_failed হার, ও মোট অনুরোধের সংখ্যা দেখা যাবে। Threshold পাস/ফেল স্পষ্ট থাকবে।

## ৯. যাচাইকরণ (Verification)

- p95 latency কি ৫০০ms-এর নিচে ছিল? না হলে কোন ধাপে ভেঙেছে?
- Error rate কি ৫%-এর নিচে ছিল?
- CPU/মেমরি গ্রাফ কি বোতল-ঘাড় (bottleneck) দেখায় — অ্যাপ, নাকি ডেটাবেস?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **k6 টার্গেটে পৌঁছাতে পারছে না:** `--network host` ব্যবহার করুন; অ্যাপের পোর্ট যাচাই করুন।
- **ফলাফল অস্থির:** ল্যাব মেশিনের নিজের CPU সীমা দেখুন — লোড জেনারেটর নিজেই বোতল-ঘাড় হতে পারে।

## ১১. নিরাপত্তা সতর্কতা

লোড টেস্টে আসল ব্যবহারকারীর ডেটা ব্যবহার করবেন না। টেস্ট ডেটা আলাদা রাখুন। টেস্টের পর অ্যাপের অবস্থা স্বাভাবিক আছে কি না যাচাই করুন।

## ১২. ক্লিনআপ (Cleanup)

```
# লোড টেস্ট শেষে অ্যাপ পুনরায় চালু করে স্বাভাবিক অবস্থা যাচাই করুন
curl -s http://localhost:3000/health
ls ~/technova-lab/lab20-load/
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. লোড টেস্টের ফলাফল থেকে কত ইনস্ট্যান্স দরকার — কীভাবে হিসাব করবেন?
2. ভার বাড়ানোর সময় প্রথমে কী ভাঙে — CPU, মেমরি, নাকি ডেটাবেস সংযোগ?
3. ক্যাশিং কীভাবে capacity পরিকল্পনা বদলে দেয়?

ল্যাব ২১

# Lab 21 — Backup, Restore and DR Exercise

Backup (ব্যাকআপ) মানে ডেটার অনুলিপি রাখা; restore (পুনরুদ্ধার) মানে তা ফিরিয়ে আনা; DR (Disaster Recovery — দুর্যোগ পুনরুদ্ধার) মানে বড় বিপর্যয়ের পর পুরো সিস্টেম ফিরিয়ে আনার পরিকল্পনা ও অনুশীলন। ব্যাকআপ নেওয়া হয়নি পরীক্ষা করা হলে তা ব্যাকআপ নয় — শুধু আশা। এই ল্যাবে আপনি MongoDB ব্যাকআপ নিয়ে আসলেই restore করে দেখাবেন।

## ১. উদ্দেশ্য (Objectives)

- MongoDB (mongodump/mongorestore) দিয়ে ব্যাকআপ ও restore করা।
- RPO (Recovery Point Objective — কত ডেটা হারানো চলবে) ও RTO (Recovery Time Objective — কত সময়ে ফিরতে হবে) নির্ধারণ করা।
- DR অনুশীলনের runbook লেখা ও সময় মাপা।
- ব্যাকআপের অখণ্ডতা (integrity) যাচাই করা।

## ২. পূর্বশর্ত (Prerequisites)

- Docker; MongoDB ল্যাব কনটেইনার; Lab 19 সম্পন্ন।

## ৩. আনুমানিক সময়

৩ ঘণ্টা ৩০ মিনিট।

## ৪. আর্কিটেকচার

```
[MongoDB Primary — 203.0.113.12:27017]
          | mongodump (দৈনিক, স্বয়ংক্রিয়)
                    v
[Backup Storage — আলাদা স্থান/বাকেট, এনক্রিপ্ট করা]
          - RPO: 1 ঘণ্টা (ঘণ্টায় ব্যাকআপ)
          - Retention: 30 দিন
                    |
                    v
[Restore অনুশীলন — আলাদা পরিবেশে mongorestore]
                    |
                    v
[যাচাই: রেকর্ড সংখ্যা, অ্যাপ সংযোগ, RTO পরিমাপ]
3-2-1 নিয়ম: ৩ কপি, ২ মাধ্যম, ১ অফসাইট।
```

## ৫. নিরাপদ ল্যাব পরিবেশ

শুধু ল্যাব ডেটাবেসে কাজ করুন — পরীক্ষামূলক ডেটা ব্যবহার করুন, আসল ব্যবহারকারীর ডেটা নয়। Restore অনুশীলন আলাদা পরিবেশে করুন, যাতে আসল ডেটা মুছে না যায়।

## ৬. সেটআপ

```
docker run -d --name technova-mongo -p 27017:27017 \
  -e MONGO_INITDB_ROOT_USERNAME=admin \
  -e MONGO_INITDB_ROOT_PASSWORD=CHANGE-ME-MONGO-PASSWORD \
  mongo:7
sleep 10
docker exec technova-mongo mongosh -u admin -p CHANGE-ME-MONGO-PASSWORD --eval
"db.runCommand({ping:1})"
mkdir -p ~/technova-lab/lab21-backup
```

## ৭. কমান্ড ও কনফিগারেশন

```
# ১. নমুনা ডেটা তৈরি (ল্যাব ডেটা)
docker exec technova-mongo mongosh -u admin -p CHANGE-ME-MONGO-PASSWORD --eval '
db = db.getSiblingDB("technova");
db.users.insertMany([{name:"Lab User 1"}, {name:"Lab User 2"}]);
print(db.users.countDocuments());
'

# ২. ব্যাকআপ নিন
docker exec technova-mongo mongodump -u admin -p CHANGE-ME-MONGO-PASSWORD \
  --out /tmp/backup-2026-11-09
docker cp technova-mongo:/tmp/backup-2026-11-09 ~/technova-lab/lab21-backup/
ls -R ~/technova-lab/lab21-backup/ | head -20

# ৩. ডেটা মুছে ফেলে restore পরীক্ষা (শুধু ল্যাব DB-তে!)
docker exec technova-mongo mongosh -u admin -p CHANGE-ME-MONGO-PASSWORD --eval '
db = db.getSiblingDB("technova");
db.users.deleteMany({});
print("after delete:", db.users.countDocuments());
'
# ৪. Restore করুন
docker exec technova-mongo mongorestore -u admin -p CHANGE-ME-MONGO-PASSWORD \
  /tmp/backup-2026-11-09
docker exec technova-mongo mongosh -u admin -p CHANGE-ME-MONGO-PASSWORD --eval '
db = db.getSiblingDB("technova");
print("after restore:", db.users.countDocuments());
'
```

**CONCEPT:** RPO বলে কতটুকু ডেটা হারানো চলবে (যেমন ১ ঘণ্টা), RTO বলে কত সময়ে সেবা ফিরতে হবে (যেমন ৪ ঘণ্টা)। RPO ছোট হলে ব্যাকআপ ঘন ঘন নিতে হয়; RTO ছোট হলে restore প্রক্রিয়া স্বয়ংক্রিয় ও অনুশীলিত হতে হয়।

## ৮. প্রত্যাশিত আউটপুট

ব্যাকআপ ফোল্ডারে BSON ফাইল; মুছে ফেলার পর রেকর্ড ০; restore-এর পর রেকর্ড আবার ২। DR runbook-এ RPO/RTO ও ধাপগুলো লেখা থাকবে।

## ৯. যাচাইকরণ (Verification)

```
docker exec technova-mongo mongosh -u admin -p CHANGE-ME-MONGO-PASSWORD --eval '
db = db.getSiblingDB("technova");
print(db.users.countDocuments());
'
ls -lh ~/technova-lab/lab21-backup/backup-2026-11-09/technova/
```

- Restore-এর সময় (RTO) কি লক্ষ্যের মধ্যে ছিল? সময় মেপে লিখুন।
- ব্যাকআপ ফাইল কি এনক্রিপ্ট করা বা সীমিত অ্যাক্সেসে রাখা আছে?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **mongodump অনুমতি পাচ্ছে না:** ব্যবহারকারী/পাসওয়ার্ড ও `--authenticationDatabase admin` যাচাই করুন।
- **Restore-এ ডুপ্লিকেট কী:** খালি ডেটাবেসে restore করুন; প্রয়োজনে `--drop` ব্যবহার করুন (শুধু ল্যাবে)।

## ১১. নিরাপত্তা সতর্কতা

ব্যাকআপে প্রোডাকশন ডেটার মতোই সংবেদনশীল তথ্য থাকে — ব্যাকআপ এনক্রিপ্ট করুন, অ্যাক্সেস সীমিত রাখুন, পুরোনো ব্যাকআপ নিরাপদে মুছুন।

## ১২. ক্লিনআপ (Cleanup)

```
docker stop technova-mongo
docker rm technova-mongo
# ব্যাকআপ ফাইল ল্যাব ফোল্ডারে সংরক্ষণ করুন; প্রয়োজনে এনক্রিপ্ট করুন
ls ~/technova-lab/lab21-backup/
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. ব্যাকআপ নেওয়া আছে কিন্তু restore পরীক্ষা করা হয়নি — কেন তা বিপজ্জনক?
2. RPO ১ ঘণ্টা বনাম ২৪ ঘণ্টা — খরচের পার্থক্য কী?
3. DR অনুশীলন কত ঘন ঘন করা উচিত?

ল্যাব ২২

# Lab 22 — Controlled Chaos Engineering Experiment

Chaos engineering (বিশৃঙ্খলা প্রকৌশল) মানে নিয়ন্ত্রিতভাবে সিস্টেমে ব্যর্থতা ঘটিয়ে দেখা — সিস্টেম কি নিজে থেকে সেরে ওঠে, অ্যালার্ট কি সময়মতো আসে, ব্যবহারকারী কি ক্ষতিগ্রস্ত হয়? এই ল্যাবে আপনি শুধু ল্যাব পরিবেশে, কঠোর সীমা (blast radius) মেনে, একটি পড বন্ধ করে resilience (সহনশীলতা) পরীক্ষা করবেন।

## ১. উদ্দেশ্য (Objectives)

- Chaos পরীক্ষার অনুমান (hypothesis) লেখা।
- Blast radius সীমিত করে একটি পড বন্ধ (pod kill) করা।
- পুনরুদ্ধারের সময় (recovery time) ও ব্যবহারকারীর প্রভাব মাপা।
- ফলাফল থেকে উন্নতির কাজ লেখা।

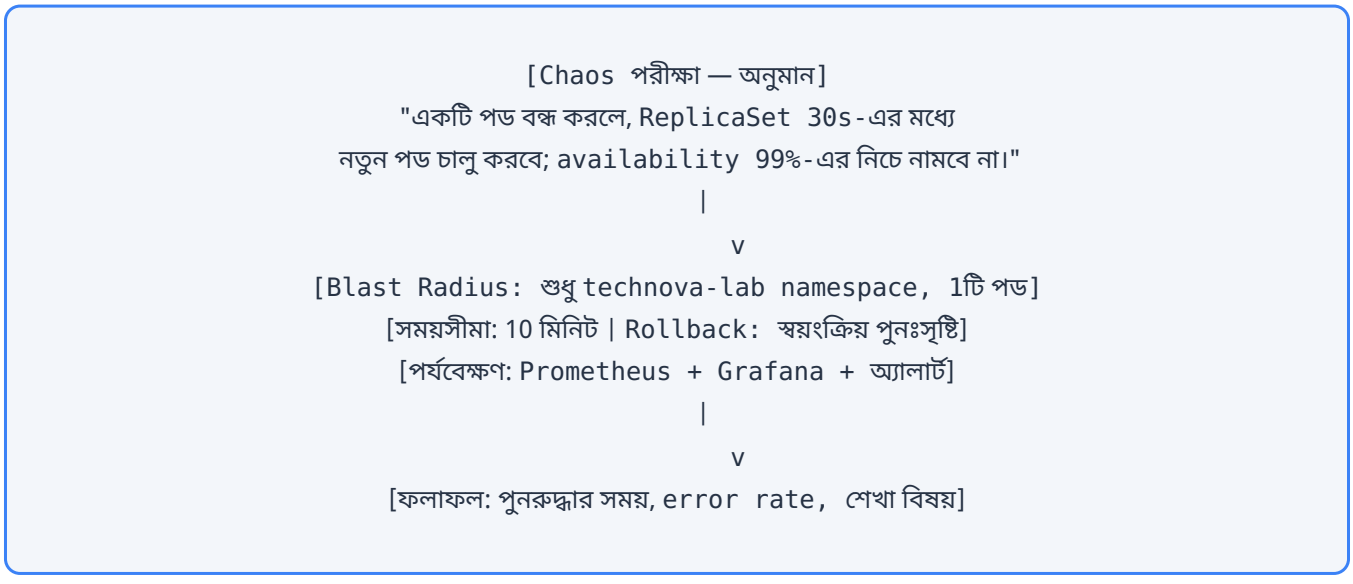
## ২. পূর্বশর্ত (Prerequisites)

- Lab 10, 17, 18 সম্পন্ন; kind/Minikube ক্লাস্টার; একটি চলমান ল্যাব অ্যাপ।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

> **WARNING:** Chaos পরীক্ষা শুধু ল্যাব ক্লাস্টারে, শুধু নিজের অ্যাপে, লিখিত অনুমোদন ও পর্যবেক্ষণসহ চালান। প্রোডাকশন, শেয়ার করা পরিবেশ বা অন্যের সিস্টেমে কখনো chaos চালাবেন না। প্রতিটি পরীক্ষায় abort (থামানো) ও rollback পরিকল্পনা থাকতে হবে।

## ৬. সেটআপ

```
kind create cluster --name chaos-lab --wait 3m 2>/dev/null || echo "ক্লাস্টার প্রস্তুত"
kubectl get nodes
kubectl create namespace technova-lab --dry-run=client -o yaml | kubectl apply -f -
cat > ~/technova-lab-chaos-hypothesis.md <<'EOF'
# Chaos অনুমান
- পরীক্ষা: একটি technova-app পড মুছে ফেলা
- অনুমান: 30s-এর মধ্যে নতুন পড Ready; error rate 1%-এর বেশি বাড়বে না
- Blast radius: technova-lab namespace, 1টি পড, 10 মিনিট
- Abort শর্ত: error rate 5%-এর বেশি হলে বা 2 মিনিটে পড ফিরে না এলে থামান
- Rollback: kubectl rollout restart / ReplicaSet স্বয়ংক্রিয়ভাবে পুনঃসৃষ্টি করে
EOF
cat ~/technova-lab-chaos-hypothesis.md
```

## ৭. কমান্ড ও কনফিগারেশন

```
# ১. 2টি replica সহ deployment তৈরি করুন
cat > chaos-app.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: technova-app
  namespace: technova-lab
spec:
  replicas: 2
  selector:
    matchLabels:
      app: technova
  template:
    metadata:
      labels:
        app: technova
    spec:
      containers:
      - name: app
        image: nginx:1.27-alpine
        ports:
        - containerPort: 80
EOF
kubectl apply -f chaos-app.yaml
kubectl get pods -n technova-lab -w --no-headers | head -5 &
sleep 20
kubectl get pods -n technova-lab
```

```
# ২. একটি পড বন্ধ করুন (chaos)
POD=$(kubectl get pods -n technova-lab -l app=technova -o
jsonpath='{.items[0].metadata.name}')
echo "বন্ধ করা হচ্ছে: $POD"
date +%T
kubectl delete pod "$POD" -n technova-lab
sleep 10
kubectl get pods -n technova-lab
date +%T
echo "পুনরুদ্ধারের সময় উপরের দুটি সময়ের পার্থক্য থেকে মাপুন"
```

## ৮. প্রত্যাশিত আউটপুট

একটি পড Terminating হবে, ReplicaSet নতুন পড তৈরি করবে; কিছুক্ষণের মধ্যে ২টি পড আবার Running হবে; error rate সামান্য বাড়তে পারে কিন্তু দ্রুত স্বাভাবিক হবে।

## ৯. যাচাইকরণ (Verification)

```
kubectl get pods -n technova-lab
kubectl describe deployment technova-app -n technova-lab | grep -i replicas
# অনুমান কি সত্য হলো? পুনরুদ্ধারের সময় ও error rate লিপিবদ্ধ করুন
```

## ১০. সমস্যা সমাধান (Troubleshooting)

- **নতুন পড আসছে না:** `kubectl describe pod` ও `kubectl get events -n technova-lab` দেখুন।
- **দুটি পডই বন্ধ হয়ে গেল:** পরীক্ষা থামান, deployment পুনরায় প্রয়োগ করুন — blast radius পরিকল্পনা পুনর্বিবেচনা করুন।

## ১১. নিরাপত্তা সতর্কতা

Chaos পরীক্ষা কখনো প্রোডাকশন ডেটা, পেমেন্ট বা ব্যবহারকারীর উপর চালাবেন না। পরীক্ষার সময় একজন পর্যবেক্ষক (observer) থাকতে হবে যিনি abort করতে পারেন।

## ১২. ক্লিনআপ (Cleanup)

```
kubectl delete -f chaos-app.yaml
kind delete cluster --name chaos-lab
docker ps
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. অনুমান ভুল প্রমাণিত হলে সেটি কি ব্যর্থতা, নাকি শেখা?
2. কোন chaos পরীক্ষা কখনোই করা উচিত নয়?
3. Chaos থেকে পাওয়া শেখা কীভাবে SLO ও অ্যালার্টে ফিরিয়ে আনবেন?

ল্যাব ২৩

# Lab 23 — Cloud Cost Audit and Optimization

ক্লাউড খরচ নিয়ন্ত্রণ ছাড়া বাড়তে থাকে — অব্যবহৃত VM, বড় ডিস্ক, ভুলে যাওয়া স্ন্যাপশট। এই ল্যাবে আপনি ল্যাব ক্লাউড অ্যাকাউন্টের খরচ অডিট করবেন, অপচয় চিহ্নিত করবেন এবং সাশ্রয়ের (optimization) পরিকল্পনা লিখবেন — নির্ভরযোগ্যতা না কমিয়ে।

## ১. উদ্দেশ্য (Objectives)

- খরচের ভাঙন (cost breakdown) পড়া — সার্ভিস, অঞ্চল, ট্যাগ অনুযায়ী।
- অব্যবহৃত/অতিরিক্ত সম্পদ (idle, over-provisioned) চিহ্নিত করা।
- Rightsizing (সঠিক আকার) ও স্বয়ংক্রিয় বন্ধের (auto-stop) সুপারিশ লেখা।
- বাজেট ও খরচের সতর্কতা (budget alert) নির্ধারণ করা।

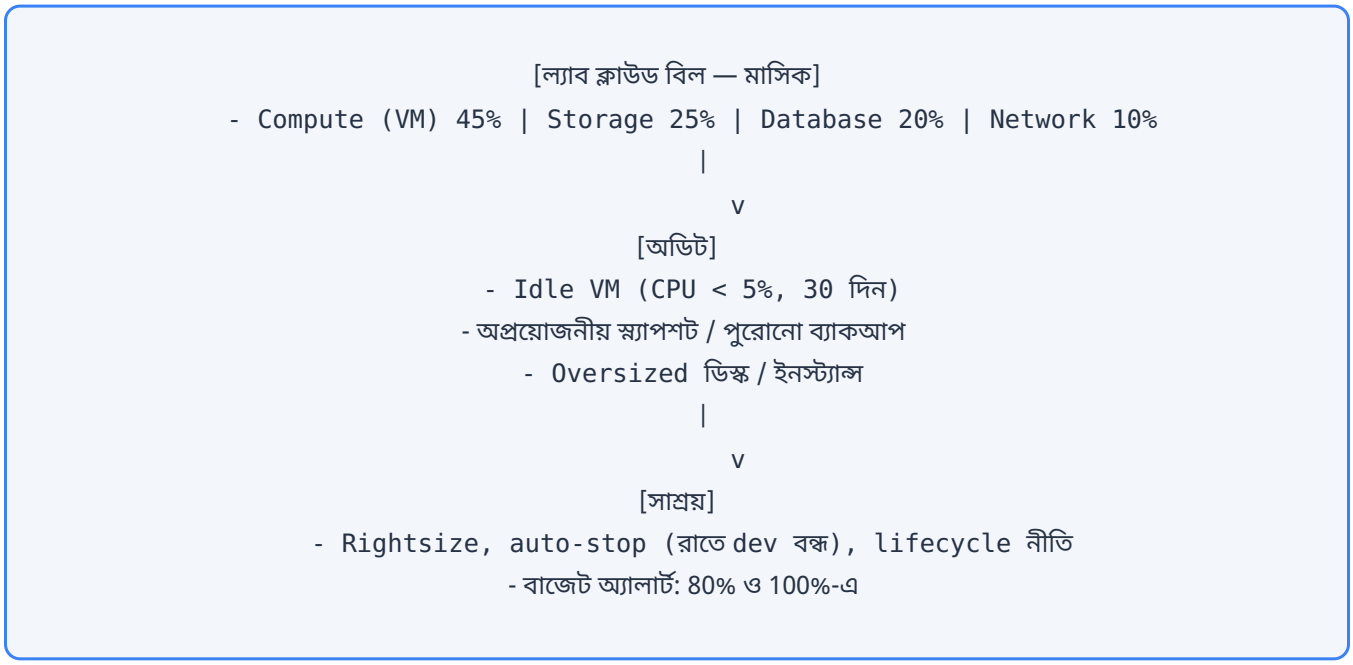
## ২. পূর্বশর্ত (Prerequisites)

- Lab 14 সম্পন্ন; ক্লাউড বিলিং ধারণা; স্প্রেডশিট।

## ৩. আনুমানিক সময়

৩ ঘণ্টা।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

খরচ অডিট read-only কাজ — কোনো সম্পদ মুছবেন না যতক্ষণ না মালিক নিশ্চিত করেন। ল্যাব অ্যাকাউন্টের বিলিং ডেটায় আসল পেমেন্ট তথ্য থাকতে পারে — তা শেয়ার করবেন না।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab23-cost
cd ~/technova-lab/lab23-cost
cat > cost-audit.md <<'EOF'
# TechNova ল্যাব খরচ অডিট (নমুনা — চিত্রকল্পমূলক)
| সার্ভিস | মাসিক খরচ (USD) | মন্তব্য |
| Compute (VM) | 120 | 2টি VM, একটি রাতে অলস |
| Storage | 65 | পুরোনো স্ন্যাপশট 40 USD |
| Database | 55 | |
| Network | 25 | |
| মোট | 265 | |
EOF
cat cost-audit.md
```

## ৭. কমান্ড ও কনফিগারেশন

```
# ল্যাব VM-এর ব্যবহার দেখুন (rightsizing-এর প্রমাণ)
uptime
free -h
df -h
# CPU ব্যবহারের ইতিহাস (sysstat থাকলে)
sar -u 1 3 2>/dev/null || echo "sar নেই — top/vmstat দিয়ে পরীক্ষা করুন"
vmstat 1 3

# সাশ্রয়ের হিসাব (নমুনা)
python3 -c "
total = 265
savings = 40 + 30  # স্ন্যাপশট মুছে + রাতে dev বন্ধ
print(f'মাসিক সাশ্রয়: \${savings} ({savings/total*100:.1f}%)')
print(f'নতুন মোট: \${total - savings}')
"
```

| সুপারিশ | মাসিক সাশ্রয় (নমুনা) | ঝুঁকি |
|---|---|---|
| পুরোনো স্ন্যাপশট মুছে ফেলা | ৪০ ডলার | নিম্ন — retention নীতি মেনে |
| Dev VM রাতে বন্ধ | ৩০ ডলার | নিম্ন — স্বয়ংক্রিয় চালু |
| VM rightsizing (বড় থেকে মাঝারি) | ২৫ ডলার | মাঝারি — লোড টেস্ট করে |

## ৮. প্রত্যাশিত আউটপুট

একটি খরচ-অডিট প্রতিবেদন, অন্তত ৩টি সাশ্রয়ের সুপারিশ (প্রতিটিতে পরিমাণ, ঝুঁকি ও বাস্তবায়ন ধাপ), এবং বাজেট-সতর্কতার নীতি।

## ৯. যাচাইকরণ (Verification)

- প্রতিটি সুপারিশের কি পরিমাপযোগ্য সাশ্রয় লেখা আছে?
- সাশ্রয় কি নির্ভরযোগ্যতা (SLO) কমাবে না — তা যাচাই করা হয়েছে কি?
- ট্যাগিং (যেমন team, env, cost-center) কি সম্পূর্ণ?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **খরচের উৎস খুঁজে পাচ্ছি না:** ট্যাগ ছাড়া সম্পদ খুঁজুন — ট্যাগহীন সম্পদই সবচেয়ে বেশি লুকানো খরচ।
- **সাশ্রয় করলে পারফরম্যান্স কমে:** Lab 20-এর লোড টেস্ট দিয়ে rightsizing যাচাই করুন।

> **REAL WORLD:** FinOps মানে অর্থ ও প্রকৌশল দলের যৌথ খরচ-পরিচালনা। সবচেয়ে বড় সাশ্রয় প্রায়ই “মুছে ফেলা” থেকে আসে — ভুলে যাওয়া সম্পদ, নয় নতুন প্রযুক্তি থেকে।

## ১১. নিরাপত্তা সতর্কতা

খরচ কমাতে ব্যাকআপ, লগিং বা নিরাপত্তা মনিটরিং বন্ধ করবেন না — সাশ্রয় যেন ঝুঁকি না বাড়ায়।

## ১২. ক্লিনআপ (Cleanup)

```
ls -l ~/technova-lab/lab23-cost/
# অডিট প্রতিবেদন সংরক্ষণ করুন; কোনো সম্পদ মোছা হয়নি (সুপারিশ মাত্র)
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. খরচ বনাম নির্ভরযোগ্যতা — কখন কোনটিকে অগ্রাধিকার দেবেন?
2. ট্যাগিং কেন খরচ-পরিচালনার ভিত্তি?
3. একটি দলকে খরচ-সচেতন করতে সবচেয়ে কার্যকর একটি অভ্যাস কী?

ল্যাব ২৪

# Lab 24 — Complete DevSecOps/SRE Capstone

এটি আগের ২৩টি ল্যাবের সমন্বয় — একটি ছোট কিন্তু সম্পূর্ণ চক্র: থ্রেট মডেল থেকে শুরু করে নিরাপদ CI/CD, পর্যবেক্ষণ, ঘটনা-প্রতিক্রিয়া ও খরচ-পর্যালোচনা পর্যন্ত। এই ল্যাবটি ক্যাপস্টোনের মহড়া (rehearsal): এখানে যা শিখেছেন, ক্যাপস্টোনে তা পূর্ণাঙ্গভাবে প্রয়োগ করবেন।

## ১. উদ্দেশ্য (Objectives)

- একটি MERN অ্যাপে end-to-end DevSecOps পাইপলাইন তৈরি করা।
- SLI/SLO, অ্যালার্ট ও ড্যাশবোর্ড একসঙ্গে চালানো।
- একটি অনুকরণীয় ঘটনা মোকাবিলা করে postmortem লেখা।
- Production readiness পর্যালোচনা (checklist) সম্পন্ন করা।

## ২. পূর্বশর্ত (Prerequisites)

- Lab 1–23 সম্পন্ন (বা সমতুল্য অভিজ্ঞতা); Docker, Kubernetes, Prometheus ধারণা।

## ৩. আনুমানিক সময়

৮ ঘণ্টা (দুই দিনে ভাগ করে করা যায়)।

## ৪. আর্কিটেকচার



## ৫. নিরাপদ ল্যাব পরিবেশ

শুধু ল্যাব পরিবেশে কাজ করুন। সব সিক্রেট CHANGE-ME-... প্লেসহোল্ডার। কোনো বাস্তব ব্যবহারকারী বা প্রোডাকশন ডেটা নয়। Chaos ধাপ শুধু Lab 22-এর সীমা মেনে।

## ৬. সেটআপ

```
mkdir -p ~/technova-lab/lab24-capstone
cd ~/technova-lab/lab24-capstone
cat > checklist.md <<'EOF'
# Production Readiness Checklist (Lab 24)
- [ ] থ্রেট মডেল হালনাগাদ (Lab 1)
- [ ] OWASP পরীক্ষা পাস (Lab 2)
- [ ] SSH ও ফায়ারওয়াল শক্ত (Lab 3)
- [ ] IAM least-privilege (Lab 4)
- [ ] সিক্রেট Vault-এ (Lab 5)
- [ ] SAST/SCA/Image স্ক্যান পাস (Lab 6, 8, 9)
- [ ] K8s অডিট পাস (Lab 10)
- [ ] CI গেট চালু (Lab 11)
- [ ] SBOM + স্বাক্ষর (Lab 12)
- [ ] নীতি-কোড পাস (Lab 13)
- [ ] CSPM চেক পাস (Lab 14)
- [ ] দুর্বলতা রেজিস্টার হালনাগাদ (Lab 15)
- [ ] SLO নির্ধারিত (Lab 16)
- [ ] ড্যাশবোর্ড চালু (Lab 17)
- [ ] অ্যালার্ট ও অন-কল প্রস্তুত (Lab 18)
- [ ] Incident runbook প্রস্তুত (Lab 19)
- [ ] লোড টেস্ট পাস (Lab 20)
- [ ] ব্যাকআপ restore পরীক্ষিত (Lab 21)
- [ ] Chaos অনুশীলন সম্পন্ন (Lab 22)
- [ ] খরচ পর্যালোচনা সম্পন্ন (Lab 23)
EOF
cat checklist.md
```

## ৭. কমান্ড ও কনফিগারেশন

```
# ১. অ্যাপ বিল্ড ও স্ক্যান (Lab 9, 11, 12-এর পুনরাবৃত্তি)
docker build -t technova-app:capstone ~/technova-lab/lab09-docker/app 2>/dev/null || echo "Lab
9-এর অ্যাপ ব্যবহার করুন"
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:latest image --severity HIGH,CRITICAL technova-app:capstone 2>&1 | tail -20

# ২. পর্যবেক্ষণ চালু (Lab 17)
# docker compose up -d (Lab 17 ফোল্ডারে)
# ৩. লোড টেস্ট (Lab 20) — সংক্ষিপ্ত
# ৪. ঘটনা অনুশীলন (Lab 19) — একটি পড বন্ধ করে প্রতিক্রিয়া অনুশীলন
echo "প্রতিটি ধাপের প্রমাণ (স্ক্রিনশট/লগ/রিপোর্ট) checklist-এর পাশে সংরক্ষণ করুন"
```

## ৮. প্রত্যাশিত আউটপুট

একটি চলমান, স্ক্যান-পাস করা অ্যাপ; SLO ড্যাশবোর্ড; একটি ঘটনার postmortem; এবং সম্পূর্ণ production readiness checklist (সব ঘর টিক দেওয়া বা ব্যতিক্রম লেখা)।

## ৯. যাচাইকরণ (Verification)

- Checklist-এর প্রতিটি ঘর কি প্রমাণসহ সম্পন্ন?
- অসম্পূর্ণ ঘর থাকলে কি ব্যতিক্রম (exception) ও সময়সীমা লেখা আছে?
- একজন পর্যালোচক কি আপনার সিস্টেমের নিরাপত্তা ও নির্ভরযোগ্যতা যাচাই করতে পারেন?

## ১০. সমস্যা সমাধান (Troubleshooting)

- **একাধিক ধাপ একসঙ্গে ব্যর্থ:** একবারে একটি ধাপ ঠিক করুন; আগের ল্যাবের নোট দেখুন।
- **সময় শেষ হয়ে যাচ্ছে:** ক্যাপস্টোনের মতো Lab 24-কেও পর্যায়ে ভাগ করুন — নিরাপত্তা, পর্যবেক্ষণ, ঘটনা, খরচ।

> **CONCEPT:** Production readiness মানে “চালু করা যায়” নয়, “চালু রেখে নিরাপদে পরিচালনা করা যায়” — মনিটরিং, অ্যালার্ট, ব্যাকআপ, runbook ও দায়িত্ব ছাড়া কোনো সিস্টেমই প্রোডাকশন-প্রস্তুত নয়।

## ১১. নিরাপত্তা সতর্কতা

ক্যাপস্টোন মহড়াতেও একই নিয়ম: শুধু ল্যাব, শুধু অনুমোদিত পরিবেশ, আসল সিক্রেট নয়, আসল ব্যবহারকারী নয়।

## ১২. ক্লিনআপ (Cleanup)

```
docker compose down -v 2>/dev/null || true
kind delete cluster --name technova-lab 2>/dev/null || true
kind delete cluster --name chaos-lab 2>/dev/null || true
docker ps -a
ls ~/technova-lab/lab24-capstone/
```

## ১৩. প্রতিফলন প্রশ্ন (Reflection Questions)

1. ২৩টি ল্যাবের মধ্যে কোনটি সবচেয়ে বেশি শিখিয়েছে — এবং কেন?
2. একটি ছোট দলে DevSecOps ও SRE-র দায়িত্ব কীভাবে ভাগ করবেন?
3. আপনার ক্যাপস্টোনে Lab 24-এর কোন অংশটি সবচেয়ে কঠিন হবে?

অ্যাসাইনমেন্ট ১

# Assignment 1 — TechNova Threat Model Report

**উদ্দেশ্য:** TechNova MERN অ্যাপের জন্য একটি পূর্ণাঙ্গ থ্রেট মডেল প্রতিবেদন তৈরি করা (Lab 1-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. DFD আঁকুন — সব trust boundary চিহ্নিত করুন।
2. STRIDE দিয়ে কমপক্ষে ১৫টি থ্রেট লিখুন।
3. প্রতিটি থ্রেটের likelihood, impact ও ঝুঁকি নির্ধারণ করুন।
4. শীর্ষ ৫ ঝুঁকির জন্য mitigation ও অবশিষ্ট ঝুঁকি লিখুন।

## ২. ডেলিভারেবল (Deliverables)

- PDF/Markdown প্রতিবেদন (DFD + STRIDE টেবিল + ঝুঁকি র‍্যাঙ্কিং)।
- শীর্ষ ৫ mitigation-এর বাস্তবায়ন পরিকল্পনা।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| DFD-র সম্পূর্ণতা ও trust boundary | ২৫% |
| STRIDE কভারেজ ও গভীরতা | ৩০% |
| ঝুঁকি মূল্যায়নের যৌক্তিকতা | ২৫% |
| Mitigation-এর বাস্তবতা ও স্পষ্টতা | ২০% |

অ্যাসাইনমেন্ট ২

# Assignment 2 — OWASP Top 10 Assessment Report

**উদ্দেশ্য:** Juice Shop বা নিজের ল্যাব অ্যাপের OWASP Top 10 মূল্যায়ন প্রতিবেদন লেখা (Lab 2-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. কমপক্ষে ৭টি OWASP শ্রেণির বাস্তব উদাহরণ খুঁজুন।
2. প্রতিটির প্রমাণ (ধাপ, স্ক্রিনশট), প্রভাব ও CVSS-ধাঁচের তীব্রতা লিখুন।
3. প্রতিটির প্রতিকার (কোড উদাহরণসহ) লিখুন।
4. একটি সংক্ষিপ্ত নির্বাহী সারাংশ (executive summary) যোগ করুন।

## ২. ডেলিভারেবল (Deliverables)

- মূল্যায়ন প্রতিবেদন (PDF/Markdown)।
- প্রতিকার-কোডের নমুনা (diff বা snippet)।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| ফলাফলের সংখ্যা ও নির্ভুলতা | ৩০% |
| প্রমাণের গুণমান | ২৫% |
| প্রতিকারের সঠিকতা | ৩০% |
| প্রতিবেদনের স্পষ্টতা | ১৫% |

অ্যাসাইনমেন্ট ৩

# Assignment 3 — Linux Hardening Playbook

**উদ্দেশ্য:** Ubuntu 24.04 সার্ভারের জন্য একটি পুনরাবৃত্তিযোগ্য hardening playbook লেখা (Lab 3-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. Lynis রিপোর্ট থেকে শীর্ষ ১০ সুপারিশ বেছে নিন।
2. প্রতিটির জন্য কমান্ড/কনফিগারেশন ও যাচাই ধাপ লিখুন।
3. SSH, UFW, fail2ban ও আপডেট নীতি একত্র করুন।
4. Hardening-এর আগে ও পরে স্কোর তুলনা করুন।

## ২. ডেলিভারেবল (Deliverables)

- Hardening playbook (Markdown + শেল স্ক্রিপ্ট)।
- আগে/পরে অডিট রিপোর্ট।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| Playbook-এর সম্পূর্ণতা | ৩০% |
| কমান্ডের নির্ভুলতা ও নিরাপত্তা | ৩০% |
| আগে/পরে পরিমাপ | ২০% |
| পুনরাবৃত্তিযোগ্যতা (অন্য VM-এ চলে কি না) | ২০% |

অ্যাসাইনমেন্ট ৪

# Assignment 4 — IAM Policy Design

**উদ্দেশ্য:** TechNova-র জন্য least-privilege IAM নকশা তৈরি করা (Lab 4-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. ৬টি ভূমিকা সংজ্ঞায়িত করুন (developer, SRE, CI runner, auditor, admin, break-glass)।
2. প্রতিটির জন্য নীতি (JSON/YAML) লিখুন।
3. ৩টি over-privileged উদাহরণ দেখিয়ে সংশোধন করুন।
4. ত্রৈমাসিক অ্যাক্সেস পর্যালোচনার প্রক্রিয়া লিখুন।

## ২. ডেলিভারেবল (Deliverables)

- ভূমিকা ও নীতি ফাইল।
- অ্যাক্সেস পর্যালোচনা চেকলিস্ট।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| Least-privilege-এর কঠোরতা | ৩৫% |
| নীতির সিনট্যাক্স ও নির্ভুলতা | ২৫% |
| Over-privilege বিশ্লেষণ | ২০% |
| পর্যালোচনা প্রক্রিয়া | ২০% |

অ্যাসাইনমেন্ট ৫

# Assignment 5 — Secrets Management Migration Plan

**উদ্দেশ্য:** একটি অ্যাপকে হার্ডকোড করা সিক্রেট থেকে Vault-ভিত্তিক ব্যবস্থাপনায় নেওয়ার পরিকল্পনা লেখা (Lab 5-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. কোডবেসে সিক্রেটের তালিকা করুন (gitleaks ধাঁচের স্ক্যান)।
2. প্রতিটি সিক্রেটের জন্য Vault পাথ ও নীতি নির্ধারণ করুন।
3. Rotation সময়সূচি ও ফাঁস-প্রতিক্রিয়া পরিকল্পনা লিখুন।
4. অ্যাপ থেকে Vault পড়ার কোড-নমুনা লিখুন।

## ২. ডেলিভারেবল (Deliverables)

- মাইগ্রেশন পরিকল্পনা (পর্যায়, ঝুঁকি, rollback)।
- Vault নীতি ও rotation সময়সূচি।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| সিক্রেট তালিকার সম্পূর্ণতা | ২৫% |
| Vault নকশা | ৩০% |
| Rotation ও ফাঁস-প্রতিক্রিয়া | ২৫% |
| মাইগ্রেশনের বাস্তবতা | ২০% |

অ্যাসাইনমেন্ট ৬

# Assignment 6 — SAST Triage Exercise

**উদ্দেশ্য:** একটি বড় SAST রিপোর্ট ত্রিয়াজ করে আসল দুর্বলতা আলাদা করা (Lab 6-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. কমপক্ষে ২০টি SAST ফলাফল সংগ্রহ করুন।
2. প্রতিটিকে শ্রেণিবদ্ধ করুন: আসল / false positive / স্বীকারযোগ্য ঝুঁকি।
3. ৫টি আসল দুর্বলতা ঠিক করে পুনঃস্ক্যানের প্রমাণ দিন।
4. False positive কমাতে নিয়ম-সমন্বয়ের সুপারিশ লিখুন।

## ২. ডেলিভারেবল (Deliverables)

- ত্রিয়াজ টেবিল (সিদ্ধান্ত ও যুক্তিসহ)।
- ঠিক করা কোড ও পুনঃস্ক্যান রিপোর্ট।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| ত্রিয়াজের নির্ভুলতা | ৩৫% |
| সংশোধনের গুণমান | ৩০% |
| যুক্তির স্পষ্টতা | ২০% |
| নিয়ম-সমন্বয় সুপারিশ | ১৫% |

অ্যাসাইনমেন্ট ৭

# Assignment 7 — DAST Report and Remediation

**উদ্দেশ্য:** ZAP স্ক্যানের ফলাফল বিশ্লেষণ করে প্রতিকার পরিকল্পনা লেখা (Lab 7-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. Baseline ও (ল্যাবে) active স্ক্যান চালান।
2. High/Medium ফলাফল যাচাই করে false positive বাদ দিন।
3. প্রতিটি আসল ফলাফলের জন্য প্রতিকার ও সময়সীমা লিখুন।
4. Helmet/CSP কনফিগারেশন প্রয়োগ করে পুনঃস্ক্যান করুন।

## ২. ডেলিভারেবল (Deliverables)

- DAST প্রতিবেদন ও প্রতিকার পরিকল্পনা।
- পুনঃস্ক্যানের তুলনা (আগে/পরে)।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| ফলাফল যাচাইয়ের নির্ভুলতা | ৩০% |
| প্রতিকারের সঠিকতা | ৩০% |
| পুনঃস্ক্যানের প্রমাণ | ২৫% |
| প্রতিবেদনের গুণমান | ১৫% |

অ্যাসাইনমেন্ট ৮

# Assignment 8 — Dependency Risk Register

**উদ্দেশ্য:** একটি MERN প্রজেক্টের dependency ঝুঁকি রেজিস্টার তৈরি করা (Lab 8-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. npm audit + Trivy দিয়ে পূর্ণ স্ক্যান করুন।
2. প্রতিটি High/Critical দুর্বলতার জন্য CVE, CVSS, প্রভাব লিখুন।
3. আপডেট, প্রতিস্থাপন বা ব্যতিক্রম — প্রতিটির সিদ্ধান্ত লিখুন।
4. Dependabot-ধাঁচের স্বয়ংক্রিয় আপডেট নীতি লিখুন।

## ২. ডেলিভারেবল (Deliverables)

- Dependency ঝুঁকি রেজিস্টার।
- আপডেট পরিকল্পনা ও নীতি।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| রেজিস্টারের সম্পূর্ণতা | ৩০% |
| অগ্রাধিকার নির্ধারণ | ২৫% |
| সিদ্ধান্তের যৌক্তিকতা | ২৫% |
| স্বয়ংক্রিয় আপডেট নীতি | ২০% |

অ্যাসাইনমেন্ট ৯

# Assignment 9 — Secure Container Design

**উদ্দেশ্য:** TechNova অ্যাপের জন্য নিরাপদ, ছোট ও স্বাক্ষরিত কনটেইনার ইমেজ ডিজাইন করা (Lab 9 ও 12-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. মাল্টি-স্টেজ Dockerfile লিখুন (non-root, read-only FS, dropped capability)।
2. Trivy স্ক্যানে HIGH/CRITICAL শূন্য বা ব্যতিক্রমসহ নথিভুক্ত করুন।
3. SBOM তৈরি করে Cosign স্বাক্ষরের পরিকল্পনা লিখুন।
4. ইমেজ আকার (size) আগে/পরে তুলনা করুন।

## ২. ডেলিভারেবল (Deliverables)

- Dockerfile, স্ক্যান রিপোর্ট, SBOM।
- স্বাক্ষর ও যাচাই পরিকল্পনা।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| Dockerfile নিরাপত্তা | ৩০% |
| স্ক্যান ফলাফল | ২৫% |
| SBOM ও স্বাক্ষর | ২৫% |
| আকার ও দক্ষতা | ২০% |

অ্যাসাইনমেন্ট ১০

# Assignment 10 — Kubernetes Security Review

**উদ্দেশ্য:** একটি Kubernetes ম্যানিফেস্ট সেটের নিরাপত্তা পর্যালোচনা করা (Lab 10 ও 13-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. কমপক্ষে ৫টি ম্যানিফেস্ট (Deployment, Service, RBAC, NetworkPolicy, Secret) পরীক্ষা করুন।
2. OPA নীতি দিয়ে স্বয়ংক্রিয় চেক চালান।
3. প্রতিটি ত্রুটির জন্য সংশোধিত ম্যানিফেস্ট লিখুন।
4. RBAC-এর least-privilege বিশ্লেষণ লিখুন।

## ২. ডেলিভারেবল (Deliverables)

- পর্যালোচনা প্রতিবেদন ও সংশোধিত ম্যানিফেস্ট।
- OPA নীতি ফাইল।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| ত্রুটি শনাক্তকরণ | ৩০% |
| সংশোধনের নির্ভুলতা | ৩০% |
| OPA নীতির গুণমান | ২০% |
| RBAC বিশ্লেষণ | ২০% |

অ্যাসাইনমেন্ট ১১

# Assignment 11 — CI/CD Security Gate Design

**উদ্দেশ্য:** TechNova-র জন্য সম্পূর্ণ CI/CD নিরাপত্তা গেট নকশা করা (Lab 11-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. পাইপলাইনের প্রতিটি ধাপের গেট (SAST, SCA, image scan, SBOM, policy) সংজ্ঞায়িত করুন।
2. প্রতিটি গেটের fail শর্ত ও ব্যতিক্রম প্রক্রিয়া লিখুন।
3. GitHub Actions ওয়ার্কফ্লো লিখে ল্যাব রেপোতে পরীক্ষা করুন।
4. Hotfix-এর জন্য নিরাপদ বাইপাস পদ্ধতি লিখুন।

## ২. ডেলিভারেবল (Deliverables)

- গেট নকশা নথি ও ওয়ার্কফ্লো ফাইল।
- পরীক্ষার প্রমাণ (পাস/ফেল লগ)।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| গেট কভারেজ | ৩০% |
| Fail নীতির স্পষ্টতা | ২৫% |
| ওয়ার্কফ্লোর কার্যকারিতা | ২৫% |
| ব্যতিক্রম ও বাইপাস নীতি | ২০% |

অ্যাসাইনমেন্ট ১২

# Assignment 12 — SLI/SLO and Error Budget Design

**উদ্দেশ্য:** TechNova-র ৩টি গুরুত্বপূর্ণ সেবার জন্য SLI/SLO ও error budget নকশা করা (Lab 16-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. প্রতিটি সেবার ব্যবহারকারীর যাত্রা চিহ্নিত করুন।
2. প্রতিটির জন্য ২–৩টি SLI ও SLO লক্ষ্য নির্ধারণ করুন।
3. Error budget গণনা করুন (৯৯.৯৯% = ৩০ দিনে ৪ মিনিট ১৯ সেকেন্ড — সঠিক গণনাসহ)।
4. Burn-rate অ্যালার্ট নীতি লিখুন।

## ২. ডেলিভারেবল (Deliverables)

- SLO নকশা নথি (টেবিল ও গণনাসহ)।
- Burn-rate অ্যালার্ট নীতি।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| SLI-এর উপযোগিতা | ৩০% |
| SLO গণনার নির্ভুলতা | ৩০% |
| Error budget নীতি | ২০% |
| Burn-rate অ্যালার্ট | ২০% |

অ্যাসাইনমেন্ট ১৩

# Assignment 13 — Observability Dashboard and Alerts

**উদ্দেশ্য:** Prometheus + Grafana-তে SLO ড্যাশবোর্ড ও Alertmanager নিয়ম তৈরি করা (Lab 17 ও 18-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. অ্যাপে /metrics যোগ করে Prometheus-এ scrape করুন।
2. Grafana-তে availability, latency, error budget প্যানেল তৈরি করুন।
3. কমপক্ষে ৪টি অ্যালার্ট নিয়ম লিখুন (critical ও warning মিলিয়ে)।
4. একটি অ্যালার্ট ইচ্ছাকৃতভাবে ট্রিগার করে প্রতিক্রিয়া নথিভুক্ত করুন।

## ২. ডেলিভারেবল (Deliverables)

- ড্যাশবোর্ড স্ক্রিনশট/JSON, অ্যালার্ট নিয়ম ফাইল।
- অ্যালার্ট পরীক্ষার লগ।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| ড্যাশবোর্ডের সম্পূর্ণতা | ৩০% |
| PromQL-এর নির্ভুলতা | ২৫% |
| অ্যালার্ট নিয়মের গুণমান | ২৫% |
| পরীক্ষার প্রমাণ | ২০% |

অ্যাসাইনমেন্ট ১৪

# Assignment 14 — Incident Postmortem

**উদ্দেশ্য:** একটি অনুকরণীয় ঘটনার blameless postmortem লেখা (Lab 19-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. একটি ঘটনা দৃশ্য বেছে নিন (যেমন ডেটাবেস বিভ্রাট, deployment regression)।
2. সময়রেখা, প্রভাব ও প্রশমন লিখুন।
3. Five Whys দিয়ে মূল কারণ বিশ্লেষণ করুন।
4. কমপক্ষে ৪টি action item (দায়িত্ব ও সময়সীমাসহ) লিখুন।

## ২. ডেলিভারেবল (Deliverables)

- Postmortem প্রতিবেদন।
- Action item ট্র্যাকার।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| সময়রেখার সম্পূর্ণতা | ২৫% |
| মূল কারণ বিশ্লেষণের গভীরতা | ৩০% |
| Blameless সুর | ১৫% |
| Action item-এর গুণমান | ৩০% |

অ্যাসাইনমেন্ট ১৫

# Assignment 15 — DR Test and Cost Review

**উদ্দেশ্য:** ব্যাকআপ restore পরীক্ষা ও ক্লাউড খরচ পর্যালোচনা একত্রে করা (Lab 21 ও 23-এর সম্প্রসারণ)।

## ১. কাজ (Tasks)

1. MongoDB ব্যাকআপ নিয়ে আলাদা পরিবেশে restore করুন; RTO মাপুন।
2. RPO/RTO লক্ষ্য নির্ধারণ করে DR runbook লিখুন।
3. ল্যাব খরচ অডিট করে ৩টি সাশ্রয়ের সুপারিশ লিখুন।
4. সাশ্রয় যেন নির্ভরযোগ্যতা না কমায় — তা যাচাই করুন।

## ২. ডেলিভারেবল (Deliverables)

- DR runbook ও restore পরীক্ষার প্রমাণ।
- খরচ পর্যালোচনা প্রতিবেদন।

## ৩. মূল্যায়ন (Evaluation Criteria)

| মানদণ্ড | ওজন |
|---|---|
| Restore পরীক্ষার প্রমাণ | ৩০% |
| DR runbook-এর সম্পূর্ণতা | ২৫% |
| খরচ বিশ্লেষণ | ২৫% |
| নির্ভরযোগ্যতা যাচাই | ২০% |

প্রজেক্ট ০১

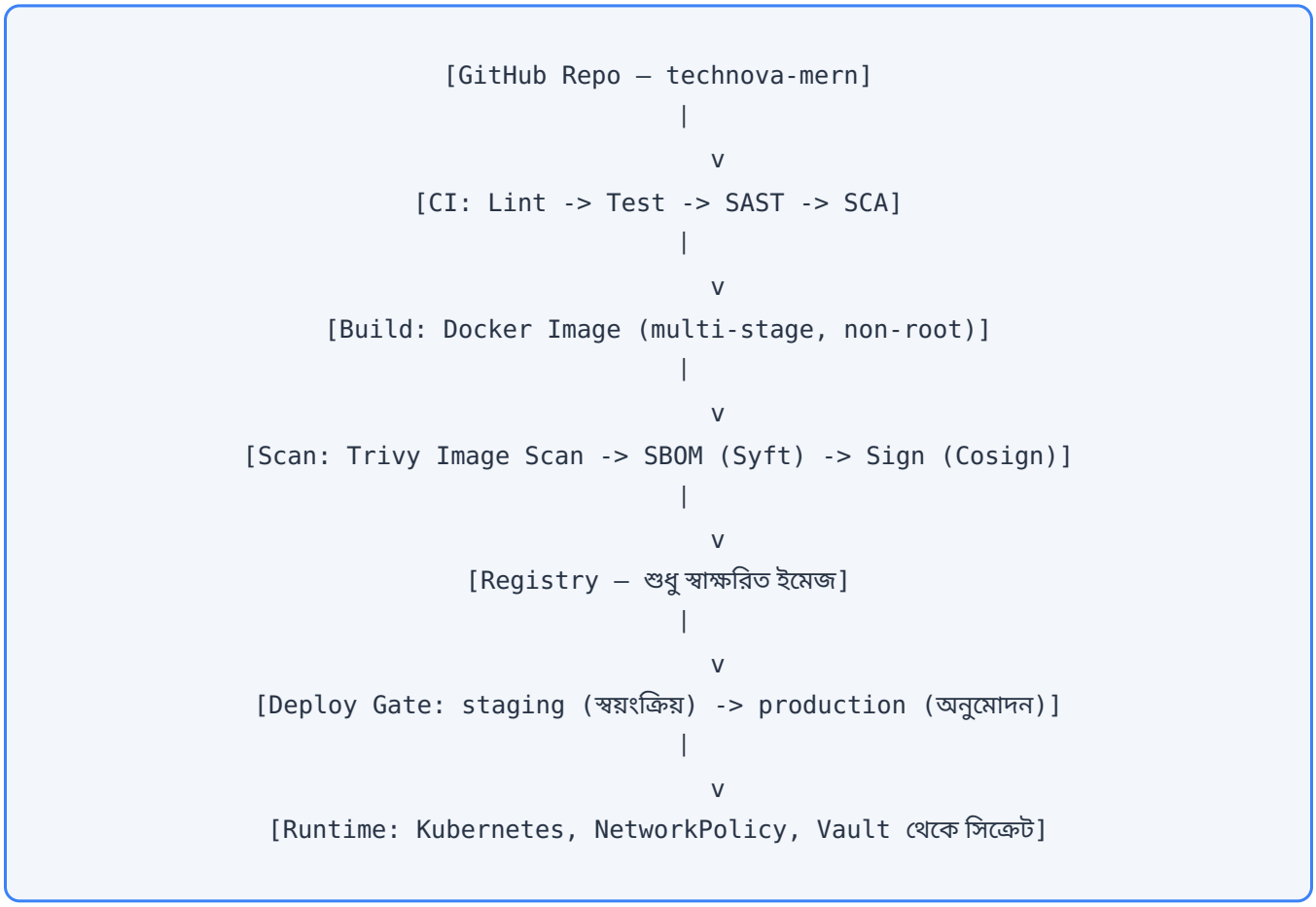
# Project 01 — Secure MERN DevSecOps Pipeline

এই প্রজেক্টে আপনি TechNova MERN অ্যাপের জন্য একটি সম্পূর্ণ নিরাপদ CI/CD পাইপলাইন তৈরি করবেন — কোড কমিট থেকে নিরাপদ deployment পর্যন্ত প্রতিটি ধাপে নিরাপত্তা পরীক্ষা থাকবে।

## ১. প্রয়োজনীয়তা (Requirements)

- GitHub repository — MERN অ্যাপ (React + Express + MongoDB), branch protection (main-এ সরাসরি push নিষিদ্ধ, PR + review বাধ্যতামূলক)।
- CI/CD — GitHub Actions (বা সমতুল্য)।
- Tests — ইউনিট ও ইন্টিগ্রেশন টেস্ট; কভারেজ লক্ষ্য ৭০%+।
- SAST — Semgrep/SonarQube; Critical পেলে পাইপলাইন ব্যর্থ।
- SCA — npm audit / Trivy; High+ পেলে ব্যর্থ।
- Docker image scanning — Trivy; Critical পেলে ব্যর্থ।
- SBOM — Syft (SPDX/CycloneDX) প্রতিটি রিলিজে।
- Artifact signing — Cosign দিয়ে ইমেজ স্বাক্ষর ও যাচাই।
- Secrets management — Vault বা GitHub Secrets; কোডে কোনো সিক্রেট নয় (gitleaks চেক)।
- Secure deployment gates — সব গেট পাস করলে তবেই staging, তারপর অনুমোদনসহ production।

## ২. আর্কিটেকচার




## ৩. পর্যায় (Stages)

1. **পর্যায় ১ — ভিত্তি:** রেপো, branch protection, টেস্ট, lint।
2. **পর্যায় ২ — স্ক্যান:** SAST, SCA, gitleaks গেট যোগ।
3. **পর্যায় ৩ — ইমেজ:** মাল্টি-স্টেজ বিল্ড, Trivy স্ক্যান, SBOM।
4. **পর্যায় ৪ — স্বাক্ষর ও গেট:** Cosign স্বাক্ষর, deployment গেট, staging deploy।
5. **পর্যায় ৫ — সিক্রেট ও প্রোডাকশন:** Vault সংযোগ, production অনুমোদন, পর্যবেক্ষণ।

## ৪. বাস্তবায়ন নির্দেশনা (Implementation Instructions)

```
# ১. রেপো কাঠামো
technova-mern/
  client/        # React
  server/        # Express + Node.js
  .github/workflows/security.yml   # Lab 11-এর ওয়ার্কফ্লো
  Dockerfile     # Lab 9-এর মাল্টি-স্টেজ
  docker-compose.yml

# ২. নিরাপত্তা গেটের ক্রম (ওয়ার্কফ্লোতে)
# lint -> test -> semgrep -> npm audit -> build -> trivy -> syft -> cosign

# ৩. স্বাক্ষর যাচাই করে deploy (নমুনা)
```

```
# cosign verify --key cosign.pub technova.example/technova-app:TAG
# kubectl set image deployment/technova-app app=technova.example/technova-app:TAG
```

## ৫. যাচাইকরণ (Validation)

- ইচ্ছাকৃত দুর্বল কোড push করলে পাইপলাইন ব্যর্থ হয় কি?
- স্বাক্ষর ছাড়া ইমেজ deploy করতে গেলে তা আটকায় কি?
- সব গেট পাস করা কোড staging-এ deploy হয় কি?
- টেস্ট কভারেজ ৭০%+ কি?

## ৬. নিরাপত্তা পর্যালোচনা (Security Review)

থ্রেট মডেল (Lab 1) দিয়ে পাইপলাইন পর্যালোচনা করুন: CI সিক্রেটের অ্যাক্সেস, তৃতীয় পক্ষের অ্যাকশনের সংস্করণ পিন, branch protection, SBOM সংরক্ষণ। প্রতিটি ফলাফল Lab 15-এর রেজিস্টারে লিখুন।

## ৭. খরচ পর্যালোচনা (Cost Review)

CI মিনিট, রেজিস্ট্রি স্টোরেজ ও staging পরিবেশের মাসিক খরচের হিসাব দিন। অপ্রয়োজনীয় পুরোনো ইমেজ মুছে ফেলার retention নীতি লিখুন।

## ৮. চূড়ান্ত ডেলিভারেবল (Final Deliverables)

- GitHub রেপো লিংক ও ওয়ার্কফ্লো ফাইল।
- SAST/SCA/Trivy রিপোর্ট, SBOM, স্বাক্ষর যাচাইয়ের প্রমাণ।
- Deployment গেটের প্রমাণ (পাস/ফেল লগ)।
- নিরাপত্তা ও খরচ পর্যালোচনা নথি।

প্রজেক্ট ০২

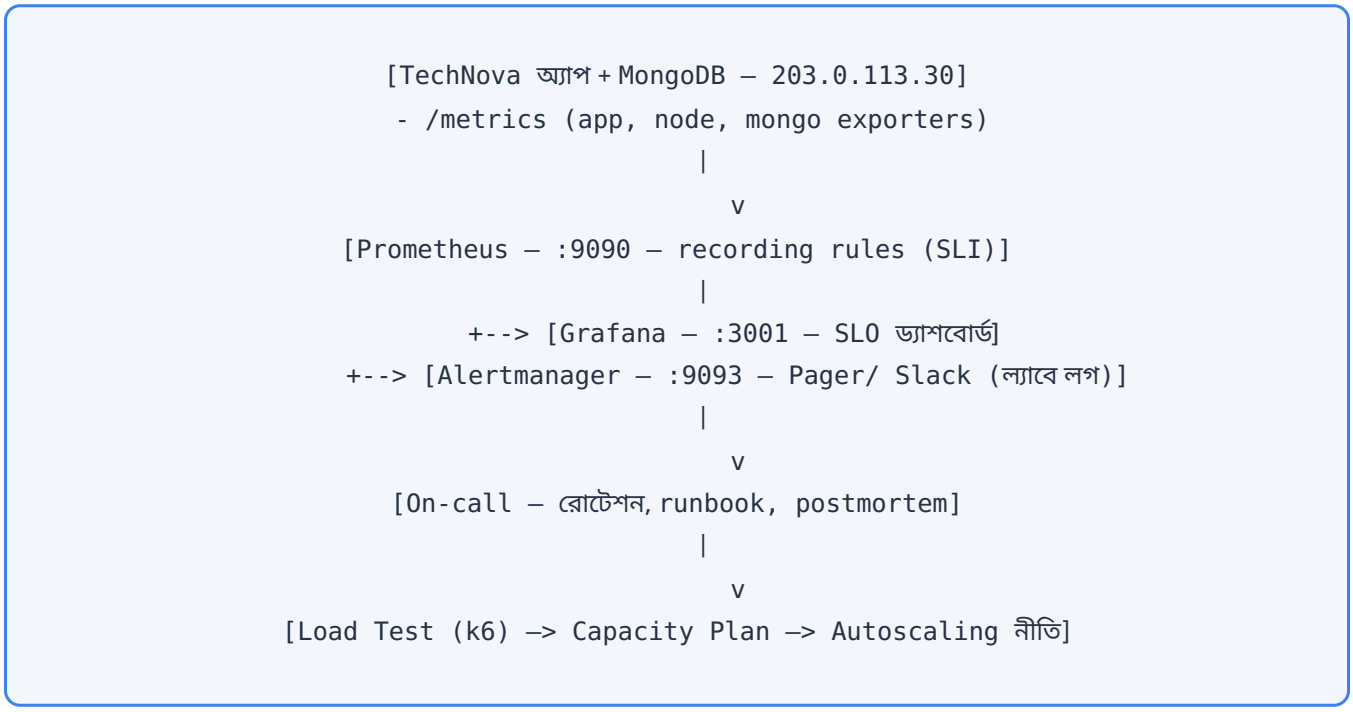
# Project 02 — Production SRE and Reliability Platform

এই প্রজেক্টে আপনি TechNova অ্যাপের জন্য একটি প্রোডাকশন-মানের SRE প্ল্যাটফর্ম তৈরি করবেন — পরিমাপ, সতর্কতা, ঘটনা-প্রতিক্রিয়া ও ধারণক্ষমতা পরিকল্পনা একসঙ্গে।

## ১. প্রয়োজনীয়তা (Requirements)

- Prometheus — মেট্রিক সংগ্রহ (অ্যাপ, Node, MongoDB exporter)।
- Grafana — SLO ড্যাশবোর্ড (availability, latency, error budget)।
- SLI/SLO — কমপক্ষে ৩টি সেবার জন্য সংজ্ঞা ও লক্ষ্য (Lab 16)।
- Error budget — গণনা ও নীতি (বাজেট শেষ হলে feature freeze)।
- Alerting — Alertmanager, burn-rate ভিত্তিক নিয়ম, রাউটিং।
- On-call workflow — রোটেশন, স্বীকৃতি সময় (MTTA), এসকেলেশন।
- Load testing — k6 দিয়ে baseline ও breaking point।
- Capacity planning — লোড টেস্ট থেকে ইনস্ট্যান্স/CPU সুপারিশ।
- Incident response — runbook, ভূমিকা, যোগাযোগ টেমপ্লেট।
- Postmortem — blameless টেমপ্লেট ও action item ট্র্যাকিং।

## ২. আর্কিটেকচার



## ৩. পর্যায় (Stages)

1. **পর্যায় ১ — পরিমাপ:** exporter, Prometheus, Grafana চালু।
2. **পর্যায় ২ — SLO:** SLI/SLO সংজ্ঞা, recording rule, ড্যাশবোর্ড।
3. **পর্যায় ৩ — সতর্কতা:** burn-rate অ্যালার্ট, রাউটিং, on-call রোটেশন।
4. **পর্যায় ৪ — ভার ও ধারণক্ষমতা:** k6 লোড টেস্ট, capacity পরিকল্পনা।
5. **পর্যায় ৫ — ঘটনা:** অনুশীলন, runbook, postmortem।

## ৪. বাস্তবায়ন নির্দেশনা (Implementation Instructions)

```
# ১. Lab 17-এর docker-compose সম্প্রসারণ করুন:
#    - node-exporter ও mongo-exporter যোগ করুন
#    - prometheus.yml-এ recording rule যোগ করুন (SLI গণনা)
# ২. Grafana-তে ৩টি ড্যাশবোর্ড: SLO Overview, Service Detail, Error Budget
# ৩. Lab 18-এর alerts.yml-এ burn-rate অ্যালার্ট যোগ করুন:
#    - 1 ঘণ্টায় 14.4x burn -> critical (page)
#    - 6 ঘণ্টায় 6x burn -> warning (ticket)
# ৪. k6 লোড টেস্ট (Lab 20) চালিয়ে capacity সুপারিশ লিখুন
# ৫. একটি ঘটনা অনুশীলন করে postmortem লিখুন (Lab 19)
```

## ৫. যাচাইকরণ (Validation)

- ড্যাশবোর্ডে SLI কি বাস্তব ট্রাফিকের সঙ্গে মেলে?
- ইচ্ছাকৃত error বাড়ালে অ্যালার্ট কি সময়মতো আসে?
- লোড টেস্টের সুপারিশ কি পরিমাপ-ভিত্তিক?
- Postmortem-এর action item কি বাস্তবায়নযোগ্য?

## ৬. নিরাপত্তা পর্যালোচনা (Security Review)

Grafana/Prometheus-এর প্রমাণীকরণ, /metrics-এর অ্যাক্সেস নিয়ন্ত্রণ, অ্যালার্টে সংবেদনশীল ডেটা না থাকা — যাচাই করুন। ড্যাশবোর্ড শুধু ল্যাব নেটওয়ার্কে রাখুন।

## ৭. খরচ পর্যালোচনা (Cost Review)

মনিটরিং স্ট্যাকের (VM, স্টোরেজ, retention) মাসিক খরচ ও retention নীতি (যেমন ৩০ দিন) লিখুন। অপ্রয়োজনীয় high-cardinality মেট্রিক বাদ দিন।

## ৮. চূড়ান্ত ডেলিভারেবল (Final Deliverables)

- Prometheus/Grafana কনফিগারেশন ও ড্যাশবোর্ড JSON।
- SLO নথি, অ্যালার্ট নিয়ম, on-call runbook।
- লোড টেস্ট রিপোর্ট ও capacity পরিকল্পনা।
- ঘটনা অনুশীলনের postmortem।

প্রজেক্ট ০৩

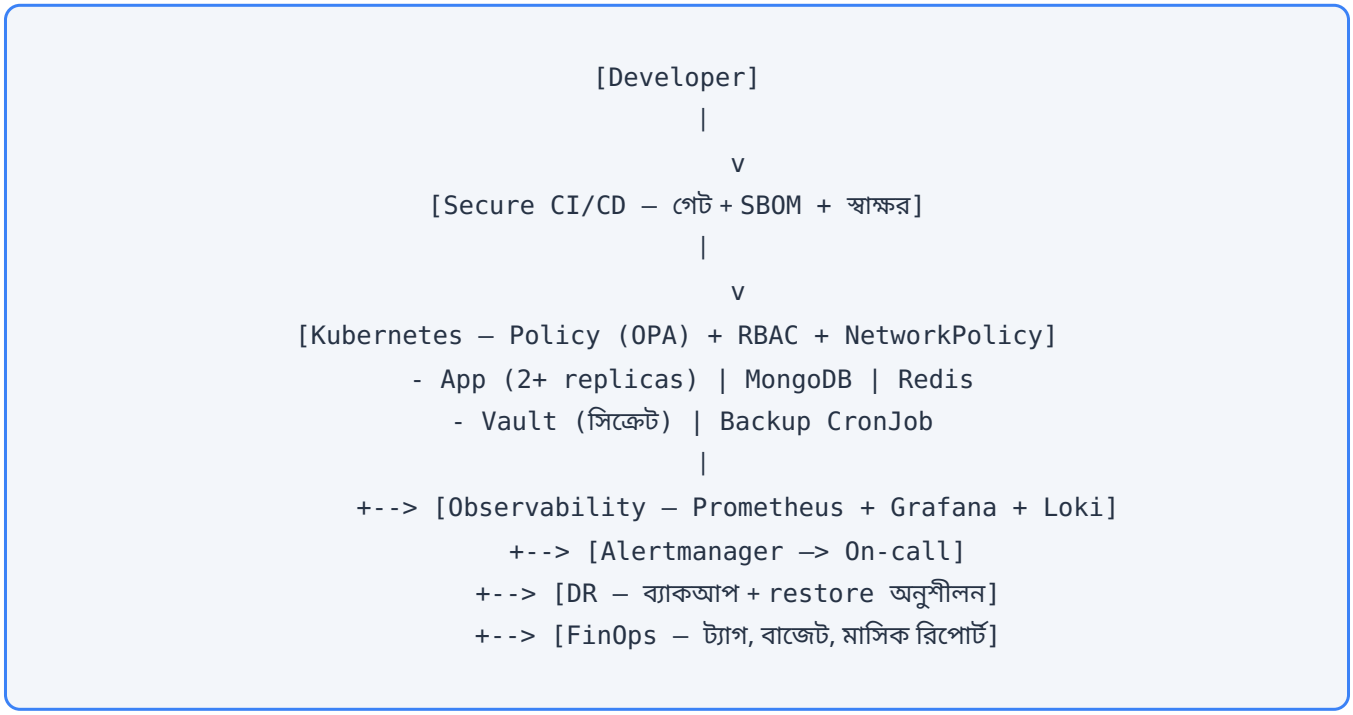
# Project 03 — TechNova Enterprise Security and Reliability Platform

এই প্রজেক্টে আপনি Project 01 ও 02-কে একত্র করে একটি এন্টারপ্রাইজ-মানের প্ল্যাটফর্ম তৈরি করবেন — নিরাপত্তা, নির্ভরযোগ্যতা, পর্যবেক্ষণ, পুনরুদ্ধার ও খরচ-শাসন (governance) একসঙ্গে।

## ১. প্রয়োজনীয়তা (Requirements)

- Secure CI/CD — Project 01-এর সব গেট (SAST, SCA, image scan, SBOM, signing, secrets)।
- Infrastructure policy enforcement — OPA/Gatekeeper দিয়ে Kubernetes নীতি; Terraform নীতি-পরীক্ষা।
- Kubernetes security — RBAC least-privilege, Pod Security Standards, NetworkPolicy, secret ব্যবস্থাপনা।
- Centralized observability — লগ (Loki বা সমতুল্য), মেট্রিক (Prometheus), ট্রেস — এক ড্যাশবোর্ডে।
- Incident response — runbook, on-call, postmortem প্রক্রিয়া।
- Backup and DR — স্বয়ংক্রিয় ব্যাকআপ, restore পরীক্ষা, RPO/RTO।
- Cloud cost governance — ট্যাগিং নীতি, বাজেট অ্যালার্ট, মাসিক পর্যালোচনা।
- Production readiness — Lab 24-এর checklist অনুযায়ী পর্যালোচনা।

## ২. আর্কিটেকচার




## ৩. পর্যায় (Stages)

1. **পর্যায় ১ — নিরাপদ ভিত্তি:** CI/CD গেট, K8s hardening, Vault।
2. **পর্যায় ২ — নীতি:** OPA/Gatekeeper, Terraform নীতি, NetworkPolicy।

3. **পর্যায় ৩ — পর্যবেক্ষণ:** কেন্দ্রীয় লগ/মেট্রিক/ট্রেস, SLO ড্যাশবোর্ড।
4. **পর্যায় ৪ — স্থিতিস্থাপকতা:** ব্যাকআপ/DR, chaos অনুশীলন, incident runbook।
5. **পর্যায় ৫ — শাসন:** খরচ-শাসন, production readiness পর্যালোচনা, চূড়ান্ত প্রতিবেদন।

## ৪. বাস্তবায়ন নির্দেশনা (Implementation Instructions)

```
# ১. Project 01-এর পাইপলাইন পুনঃব্যবহার করুন; OPA নীতি (Lab 13) CI-তে যোগ করুন
# ২. kind ক্লাস্টারে Lab 10-এর নিরাপদ ম্যানিফেস্ট deploy করুন
# ৩. Lab 17-এর স্ট্যাকে Loki (লগ) যোগ করে Grafana-তে লগ প্যানেল তৈরি করুন
# ৪. CronJob দিয়ে দৈনিক mongodump ব্যাকআপ নিন (Lab 21); restore পরীক্ষা করুন
# ৫. Lab 22-এর সীমা মেনে একটি chaos অনুশীলন করুন; ফলাফল নথিভুক্ত করুন
# ৬. Lab 23-এর খরচ অডিট দিয়ে FinOps রিপোর্ট লিখুন
```

## ৫. যাচাইকরণ (Validation)

- নীতি-বিরোধী ম্যানিফেস্ট কি CI/K8s-এ আটকায়?
- সব গেট পাস করা রিলিজই কি deploy হয়?
- ড্যাশবোর্ডে লগ, মেট্রিক ও অ্যালার্ট কি একসঙ্গে দেখা যায়?
- Restore পরীক্ষা কি RTO লক্ষ্যের মধ্যে সম্পন্ন হয়?

## ৬. নিরাপত্তা পর্যালোচনা (Security Review)

পূর্ণাঙ্গ পর্যালোচনা: থ্রেট মডেল, OWASP পরীক্ষা, CSPM চেক, দুর্বলতা রেজিস্টার, K8s অডিট — সব একত্র করে একটি নিরাপত্তা প্রতিবেদন লিখুন। প্রতিটি High ফলাফলের জন্য action item থাকতে হবে।

## ৭. খরচ পর্যালোচনা (Cost Review)

প্ল্যাটফর্মের মাসিক খরচের ভাঙন, ট্যাগিং সম্পূর্ণতা, বাজেট অ্যালার্ট, এবং ৩টি সাশ্রয়ের সুপারিশ — নির্ভরযোগ্যতা না কমিয়ে।

## ৮. চূড়ান্ত ডেলিভারেবল (Final Deliverables)

- আর্কিটেকচার নথি ও ডায়াগ্রাম।
- CI/CD, K8s, OPA, observability কনফিগারেশন।
- নিরাপত্তা প্রতিবেদন, DR runbook, FinOps রিপোর্ট।
- Production readiness checklist (স্বাক্ষরিত)।
- চূড়ান্ত উপস্থাপনা (demo + প্রতিবেদন)।

ক্যাপস্টোন

# Capstone — TechNova: Secure, Reliable and Cost-Efficient Production Platform

TechNova একটি কাল্পনিক প্রোডাকশন-ধাঁচের MERN প্ল্যাটফর্ম (technova.example) তৈরি করছে — যেখানে ব্যবহারকারীরা পণ্য ব্রাউজ করে, অর্ডার দেয় ও পেমেন্ট করে। আপনার কাজ: এই প্ল্যাটফর্মটিকে end-to-end DevSecOps ও SRE অনুশীলনসহ নিরাপদ, নির্ভরযোগ্য ও খরচ-সাশ্রয়ী করে তোলা — এবং তা প্রমাণসহ উপস্থাপন করা। এটি এই বইয়ের সব ল্যাব, অ্যাসাইনমেন্ট ও প্রজেক্টের সমন্বিত চূড়ান্ত কাজ।

> **CONCEPT:** ক্যাপস্টোনে “কাজ করে” যথেষ্ট নয় — “নিরাপদে কাজ করে, মাপা যায়, ভাঙলে সেরে ওঠে, আর খরচ নিয়ন্ত্রণে থাকে” — এই চারটি প্রমাণ করতে হবে। প্রতিটি দাবির পাশে প্রমাণ (লগ, রিপোর্ট, ড্যাশবোর্ড, পরীক্ষা) রাখুন।

## ১. প্রেক্ষাপট ও সুযোগ (Context and Scope)

- **কোম্পানি:** TechNova (কাল্পনিক); **অ্যাপ:** MERN — React ফ্রন্টএন্ড, Express/Node.js API, MongoDB, Redis ক্যাশ; **ডোমেইন:** technova.example; **ল্যাব নেটওয়ার্ক:** 203.0.113.x; **OS:** Ubuntu Server 24.04 LTS।
- **ব্যবহারকারী যাত্রা:** হোমপেজ → লগইন → পণ্য ব্রাউজ → চেকআউট → পেমেন্ট → অর্ডার নিশ্চিতকরণ।
- **সীমা:** শুধু ল্যাব পরিবেশ (লোকাল VM/কনটেইনার, kind/Minikube, Juice Shop)। আসল ব্যবহারকারী, আসল পেমেন্ট বা প্রোডাকশন ডেটা নয়। সব সিক্রেট CHANGE-ME-... প্লেসহোল্ডার।

## ২. থ্রেট মডেল (Threat Model)

Lab 1-এর পদ্ধতিতে পূর্ণাঙ্গ থ্রেট মডেল তৈরি করুন: DFD (সব trust boundary-সহ), STRIDE দিয়ে কমপক্ষে ২০টি থ্রেট, ঝুঁকি র‍্যাঙ্কিং, এবং শীর্ষ ৮ ঝুঁকির mitigation। থ্রেট মডেলটি জীবন্ত নথি — প্রতিটি বড় পরিবর্তনে হালনাগাদ করুন।

## ৩. নিরাপত্তা প্রয়োজনীয়তা (Security Requirements)

| ক্ষেত্র | প্রয়োজনীয়তা |
|---|---|
| প্রমাণীকরণ | bcrypt পাসওয়ার্ড হ্যাশ, JWT স্বল্প মেয়াদ, MFA (অ্যাডমিন), rate limiting |
| অনুমোদন | RBAC — ব্যবহারকারী শুধু নিজের ডেটা দেখে; অ্যাডমিন আলাদা ভূমিকা |
| ইনপুট | সব ইনপুট যাচাই (validation), parameterized query, output encoding |
| ডেটা | At-rest ও in-transit এনক্রিপশন; PII ন্যূনতম সংগ্রহ |
| সিক্রেট | Vault-এ সংরক্ষণ; কোডে হার্ডকোড নিষিদ্ধ; rotation |
| অবকাঠামো | SSH কী-ভিত্তিক, ফায়ারওয়াল, least-privilege IAM |

## ৪. নিরাপদ উন্নয়ন কর্মপ্রবাহ (Secure Development Workflow)

- Branch protection: main-এ সরাসরি push নিষিদ্ধ; PR, কমপক্ষে ১টি review, status check পাস বাধ্যতামূলক।
- Pre-commit: lint, gitleaks, Semgrep (দ্রুত নিয়ম)।
- কোড রিভিউ চেকলিস্ট: ইনপুট যাচাই, authorization, সিক্রেট, লগে সংবেদনশীল ডেটা নেই।
- Dependency নীতি: নতুন প্যাকেজ যোগের আগে পর্যালোচনা; lock ফাইল কমিট।

## ৫. CI/CD নিরাপত্তা গেট (CI/CD Security Gates)

Project 01-এর পাইপলাইন পূর্ণাঙ্গ করুন: lint → test (৭০%+ কভারেজ) → SAST → SCA → gitleaks → Docker build → Trivy image scan → SBOM → Cosign স্বাক্ষর → policy check (OPA) → staging → অনুমোদন → production। কোনো Critical গেট ব্যর্থ হলে deploy বন্ধ (fail-closed)।

## ৬. দুর্বলতা ব্যবস্থাপনা (Vulnerability Management)

Lab 15-এর workflow প্রয়োগ করুন: রেজিস্টার, CVSS-ভিত্তিক SLA (Critical ৭ দিন, High ৩০ দিন, Medium ৯০ দিন), ত্রিয়াজ, পুনঃস্ক্যানের প্রমাণ, ব্যতিক্রম প্রক্রিয়া। ক্যাপস্টোন শেষে কোনো খোলা Critical দুর্বলতা থাকতে পারবে না — থাকলে লিখিত ব্যতিক্রম ও ক্ষতিপূরণ নিয়ন্ত্রণ লাগবে।

## ৭. SBOM ও Artifact Provenance

প্রতিটি রিলিজে Syft SBOM (SPDX) তৈরি করুন, Cosign দিয়ে ইমেজে স্বাক্ষর করুন, এবং deploy-এর সময় স্বাক্ষর যাচাই করুন। SBOM সংরক্ষণ করুন যাতে ভবিষ্যতে কোনো CVE এলে ৫ মিনিটে বলা যায় — “এই রিলিজে ওই প্যাকেজ আছে কি না”।

## ৮. কনটেইনার ও Kubernetes নিরাপত্তা (Container and Kubernetes Security)

- মাল্টি-স্টেজ, Alpine/slim বেস, non-root, read-only FS, dropped capability।
- Pod Security Standards (restricted), RBAC least-privilege, NetworkPolicy, ResourceQuota/Limit।
- OPA/Gatekeeper নীতি: privileged নিষিদ্ধ, latest ট্যাগ নিষিদ্ধ, সিক্রেট environment-এ নয়।

## ৯. সিক্রেট ব্যবস্থাপনা (Secrets Management)

Vault (Lab 5) থেকে অ্যাপ সিক্রেট পড়ুন; rotation সময়সূচি; ফাঁস-প্রতিক্রিয়া runbook। কোনো সিক্রেট Git, লগ বা ড্যাশবোর্ডে থাকবে না।

## ১০. SLI/SLO নকশা (SLI/SLO Design)

Lab 16-এর পদ্ধতিতে ৩টি সেবার SLI/SLO নির্ধারণ করুন। মনে রাখুন: ৯৯.৯৯% SLO-তে ৩০ দিনের মাসে অনুমোদিত downtime ৪ মিনিট ১৯ সেকেন্ড (৪.৩২ মিনিট) — ৪ মিনিট ২৩ সেকেন্ড নয়। Error budget নীতি লিখুন: বাজেট ৫০%-এর নিচে নামলে সতর্কতা, শেষ হলে feature freeze।

## ১১. পর্যবেক্ষণযোগ্যতা (Observability)

Prometheus (মেট্রিক), Grafana (SLO ড্যাশবোর্ড), Loki (লগ) — তিনটি একসঙ্গে। প্রতিটি সেবার জন্য: availability, latency (p95), error rate, error budget প্যানেল। ড্যাশবোর্ডে কোনো ব্যক্তিগত ডেটা নয় — শুধু সমষ্টিগত মেট্রিক।

## ১২. ঘটনা-প্রতিক্রিয়া (Incident Response)

On-call রোটেশন, এসকেলেশন, যোগাযোগ টেমপ্লেট, runbook (কমপক্ষে ৫টি: ডেটাবেস বিভ্রাট, অ্যাপ ক্র্যাশ, credential মেয়াদোত্তীর্ণ, deployment regression, latency spike)। প্রতিটি runbook-এ: লক্ষণ, নির্ণয় ধাপ, প্রশমন, যাচাই।

## ১৩. লোড টেস্টিং (Load Testing)

k6 দিয়ে baseline লোড টেস্ট চালান; p95 latency, error rate ও breaking point নথিভুক্ত করুন; তার ভিত্তিতে capacity পরিকল্পনা ও autoscaling নীতি লিখুন। লোড টেস্ট শুধু ল্যাব অ্যাপে।

## ১৪. ব্যাকআপ ও দুর্যোগ পুনরুদ্ধার (Backup and Disaster Recovery)

স্বয়ংক্রিয় MongoDB ব্যাকআপ (দৈনিক), 3-2-1 নিয়ম, RPO/RTO লক্ষ্য, এবং আলাদা পরিবেশে restore পরীক্ষার প্রমাণ। DR runbook-এ ধাপে ধাপে পুনরুদ্ধার ও দায়িত্ব লেখা থাকবে।

## ১৫. Chaos Engineering

Lab 22-এর সীমা মেনে কমপক্ষে ২টি নিয়ন্ত্রিত পরীক্ষা চালান (যেমন পড বন্ধ, latency যোগ)। প্রতিটিতে: অনুমান, blast radius, abort শর্ত, ফলাফল, শেখা বিষয় ও উন্নতির কাজ। প্রোডাকশনে নয় — শুধু ল্যাবে।

## ১৬. খরচ অপ্টিমাইজেশন (Cost Optimization)

খরচের ভাঙন, ট্যাগিং নীতি, বাজেট অ্যালার্ট, rightsizing সুপারিশ, এবং মাসিক FinOps পর্যালোচনা। সাশ্রয় যেন SLO বা নিরাপত্তা না কমায় — প্রতিটি সুপারিশে ঝুঁকি লিখুন।

## ১৭. বাস্তবসম্মত ব্যর্থতা অনুকরণ (Realistic Failure Simulations)

নিচের ৭টি দৃশ্যের প্রতিটি অনুশীলন করে প্রতিক্রিয়া নথিভুক্ত করুন — প্রতিটিতে: কী ঘটল, কীভাবে ধরা পড়ল, কী করলেন, কত সময় লাগল, কী শিখলেন:

1. **ডেটাবেস বিভ্রাট (Database outage):** MongoDB বন্ধ করে দিন — অ্যাপ কি সুন্দরভাবে ব্যর্থ হয় (graceful degradation)? অ্যালার্ট কি আসে? ব্যাকআপ থেকে পুনরুদ্ধার কত সময় নেয়?
2. **অ্যাপ্লিকেশন ক্র্যাশ (Application crash):** একটি পড/প্রসেস বন্ধ করুন — ReplicaSet কি পুনরুদ্ধার করে? ব্যবহারকারী কি ক্ষতিগ্রস্ত হয়?
3. **Credential মেয়াদোত্তীর্ণ (Credential expiry):** একটি JWT secret / API কী ঘোরান (rotate) — পুরোনো credential কি প্রত্যাখ্যাত হয়? নতুন credential-এ সেবা কি স্বাভাবিক থাকে?
4. **দুর্বল নির্ভরতা (Vulnerable dependency):** একটি জানা-দুর্বল প্যাকেজ সংস্করণ যোগ করুন — SCA/CI গেট কি তা আটকায়? প্যাচ করে পুনঃস্ক্যানের প্রমাণ দিন।

5. **সম্পদ নিঃশেষ (Resource exhaustion):** CPU/মেমরি সীমা ছোট করে ভার দিন — অ্যাপ কি সীমা মেনে চলে? অ্যালার্ট ও autoscaling কি কাজ করে?
6. **Deployment regression:** একটি ভাঙা রিলিজ deploy করুন — স্বাস্থ্য পরীক্ষা (health check) কি তা ধরে? Rollback কত সময়ে সম্পন্ন হয়?
7. **Latency spike:** কৃত্রিম বিলম্ব (delay) যোগ করুন — p95 অ্যালার্ট কি আসে? ব্যবহারকারীর যাত্রা কি প্রভাবিত হয়? কারণ নির্ণয় করে ঠিক করুন।

> **WARNING:** এই ৭টি অনুকরণ শুধু ল্যাব পরিবেশে, একবারে একটি করে, পর্যবেক্ষণ ও rollback পরিকল্পনাসহ চালান। একসঙ্গে একাধিক বিভ্রাট ঘটাবেন না। প্রতিটি অনুশীলনের আগে দলকে জানান।

## ১৮. Production Readiness Review

Lab 24-এর checklist ব্যবহার করে আনুষ্ঠানিক পর্যালোচনা করুন। প্রতিটি ঘরে প্রমাণ সংযুক্ত করুন। কোনো ঘর অসম্পূর্ণ থাকলে ব্যতিক্রম (exception) লিখুন — দায়িত্বপ্রাপ্ত ব্যক্তি ও সময়সীমাসহ। পর্যালোচনা ছাড়া “প্রোডাকশন-প্রস্তুত” দাবি করবেন না।

## ১৯. Runbook

কমপক্ষে ৫টি runbook লিখুন (উপরে ১২ নম্বরে তালিকাভুক্ত)। প্রতিটি runbook এক পৃষ্ঠায় পড়ে অনুসরণ করা যায় এমন হতে হবে — লক্ষণ, নির্ণয়, প্রশমন, যাচাই, এসকেলেশন।

## ২০. Architecture Decision Records (ADR)

কমপক্ষে ৫টি ADR লিখুন — যেমন: “কেন Vault?”, “কেন Prometheus?”, “কেন ৯৯.৯% SLO (৯৯.৯৯% নয়)?”, “কেন kind/Minikube ল্যাবে?”, “কেন Cosign?” প্রতিটি ADR-এ: প্রেক্ষাপট, সিদ্ধান্ত, বিকল্প, পরিণতি।

## ২১. Postmortem Reports

কমপক্ষে ২টি blameless postmortem লিখুন (ব্যর্থতা অনুকরণ থেকে)। প্রতিটিতে: সময়রেখা, প্রভাব, Five Whys, অবদানকারী কারণ, action item (দায়িত্ব ও সময়সীমাসহ)।

## ২২. চূড়ান্ত গ্রেডিং রুব্রিক (Final Grading Rubric)

| ক্ষেত্র | ওজন | উৎকৃষ্ট (৯০%+) |
|---|---|---|
| নিরাপত্তা (থ্রেট মডেল, গেট, SBOM, K8s) | ৩০% | সব গেট পাস, কোনো খোলা Critical নেই, প্রমাণসহ |
| নির্ভরযোগ্যতা (SLO, ড্যাশবোর্ড, অ্যালার্ট) | ২৫% | SLO মাপা যায়, অ্যালার্ট পরীক্ষিত, error budget নীতি আছে |
| ঘটনা ও পুনরুদ্ধার (runbook, DR, postmortem) | ২০% | ৭টি অনুকরণ সম্পন্ন, restore পরীক্ষিত, blameless postmortem |
| খরচ ও শাসন (FinOps, ADR, readiness) | ১৫% | খরচ পর্যালোচনা, ADR, readiness পর্যালোচনা সম্পন্ন |
| উপস্থাপনা ও নথি | ১০% | স্পষ্ট, প্রমাণসহ, পুনরাবৃত্তিযোগ্য |

## ২৩. চূড়ান্ত ডেলিভারেবল (Final Deliverables)

- GitHub রেপো, CI/CD পাইপলাইন, SBOM, স্বাক্ষর যাচাই।
- থ্রেট মডেল, নিরাপত্তা প্রতিবেদন, দুর্বলতা রেজিস্টার।
- SLO নথি, Grafana ড্যাশবোর্ড, অ্যালার্ট নিয়ম।
- ৫টি runbook, ৫টি ADR, ২টি postmortem।
- DR runbook + restore প্রমাণ, FinOps রিপোর্ট, readiness checklist।
- ৭টি ব্যর্থতা অনুকরণের প্রতিবেদন ও চূড়ান্ত উপস্থাপনা।

ক্যাপস্টোন — সমাধান নির্দেশিকা

# Capstone Solution Guide — TechNova Production Platform (Reference Outline)

> **CONCEPT:** প্রথমে নিজে চেষ্টা করুন (attempt-first)। এই নির্দেশিকা একটি রেফারেন্স রূপরেখা — সম্পূর্ণ সমাধান নয়। নিজের সমাধান জমা দেওয়ার পর এখানে দেওয়া পদ্ধতির সঙ্গে তুলনা করুন; যেখানে পার্থক্য আছে, সেখানেই শেখা সবচেয়ে বেশি। সরাসরি নকল করলে শেখা হবে না।

এই নির্দেশিকা ক্যাপস্টোনের প্রতিটি অংশের একটি গ্রহণযোগ্য পদ্ধতি তুলে ধরে — যাতে আপনি নিজের কাজ যাচাই করতে পারেন।

## ১. রেফারেন্স পদ্ধতি — সংক্ষেপে

- **থ্রেট মডেল:** DFD-তে ৪টি trust boundary (ইন্টারনেট/DMZ, অ্যাপ, ডেটা, পেমেন্ট গেটওয়ে); শীর্ষ ঝুঁকি — credential stuffing, NoSQL injection, JWT জালিয়াতি, SSRF (ফাইল আপলোড), অতিরিক্ত IAM অনুমতি।
- **CI/CD:** GitHub Actions — lint → test → Semgrep → npm audit → Trivy → Syft → Cosign → OPA → staging → অনুমোদন → production। Critical-এ fail-closed।
- **SLO (উদাহরণ):** চেকআউট availability ৯৯.৯% (৩০ দিনে ৪৩ মিনিট ১২ সেকেন্ড); API p95 < ৩০০ms; error budget ০.১%। ৯৯.৯৯% শুধু তখনই, যখন পরিমাপ তা সমর্থন করে — মনে রাখুন ৯৯.৯৯% = ৪ মিনিট ১৯ সেকেন্ড।
- **পর্যবেক্ষণ:** Prometheus + Grafana + Loki; burn-rate অ্যালার্ট (১৪.৪× / ১ ঘণ্টা = page, ৬× / ৬ ঘণ্টা = ticket)।
- **DR:** দৈনিক mongodump, RPO ১ ঘণ্টা, RTO ৪ ঘণ্টা, restore পরীক্ষা মাসিক।
- **খরচ:** ট্যাগিং, বাজেট অ্যালার্ট ৮০%/১০০%, dev পরিবেশ রাতে বন্ধ, পুরোনো স্ন্যাপশট lifecycle নীতি।

## ২. ব্যর্থতা অনুকরণ — প্রত্যাশিত প্রতিক্রিয়া (সংক্ষেপে)

| দৃশ্য | প্রত্যাশিত প্রতিক্রিয়া |
|---|---|
| ডেটাবেস বিভ্রাট | অ্যালার্ট আসে, অ্যাপ graceful error দেয়, ব্যাকআপ থেকে restore; RTO মাপা হয় |
| অ্যাপ ক্র্যাশ | ReplicaSet নতুন পড তৈরি করে; error rate সাময়িক বাড়ে, দ্রুত স্বাভাবিক হয় |
| Credential মেয়াদোত্তীর্ণ | পুরোনো credential প্রত্যাখ্যাত হয়; rotation runbook অনুযায়ী নতুন credential চালু হয় |
| দুর্বল নির্ভরতা | CI গেট আটকায়; প্যাচের পর পুনঃস্ক্যান পাস হয় |
| সম্পদ নিঃশেষ | Resource limit ও অ্যালার্ট কাজ করে; HPA/সুপারিশ নথিভুক্ত হয় |
| Deployment regression | Health check ব্যর্থ হয়; rollback ১০ মিনিটের মধ্যে সম্পন্ন হয় |
| Latency spike | p95 অ্যালার্ট আসে; কারণ (ধীর কুয়েরি/ডাউনস্ট্রিম) নির্ণয় করে ঠিক করা হয় |

## ৩. নিজের কাজ যাচাই করুন (Self-Check)

1. আপনার থ্রেট মডেলের শীর্ষ ৫ ঝুঁকি কি এই রূপরেখার সঙ্গে মেলে? না মিললে যুক্তি লিখুন — ভিন্ন উত্তরও সঠিক হতে পারে।
2. আপনার SLO গণনা কি সঠিক? ৯৯.৯৯% = ৪ মিনিট ১৯ সেকেন্ড যাচাই করুন।
3. ৭টি অনুকরণের প্রতিটিতে কি MTTA/MTTR মাপা হয়েছে?
4. আপনার ADR-গুলো কি বিকল্প ও পরিণতি ব্যাখ্যা করে?
5. Readiness checklist-এর প্রতিটি ঘরে কি প্রমাণ আছে?

## ৪. সাধারণ ভুল (Common Mistakes)

- সব গেট পাস না করেও deploy করা — fail-open রাখা।
- SLO নির্ধারণ করা কিন্তু মাপার ব্যবস্থা না করা।
- Postmortem-এ ব্যক্তিকে দোষ দেওয়া।
- ব্যাকআপ নেওয়া কিন্তু restore পরীক্ষা না করা।
- খরচ কমাতে মনিটরিং বা ব্যাকআপ বন্ধ করা।
- ৯৯.৯৯%-এর downtime ৪ মিনিট ২৩ সেকেন্ড লেখা — সঠিক মান ৪ মিনিট ১৯ সেকেন্ড।

এই নির্দেশিকা শেষ। আপনার ক্যাপস্টোন জমা দেওয়ার পর একজন পর্যালোচকের সঙ্গে রুব্রিক (ক্যাপস্টোন ২২ নম্বর) মিলিয়ে নম্বর দিন — এবং শেখা বিষয়গুলো পরের প্রজেক্টে প্রয়োগ করুন।